INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

Señales en telecomunicaciones

Sistema de telefonía inalámbrica en topología de estrella con nodo central

mediante módulos NRF24L01 PA+LNA y microcontroladores ESP32

Documentación grupal del proyecto: proceso, versiones, prácticas, protocolos, pruebas y demostración

Formato IEEE · Semestre agosto–diciembre 2026

Profesor: Ricardo Alejandro Rodríguez Jiménez

Aguascalientes, Ags.

Fecha de entrega: octubre de 2026

Repositorio del equipo: github.com/maztherzanch/STLC_SYBER_ITA_26_NRFL-ESP_Communication

RESUMEN

Resumen— Este documento reúne el proceso completo seguido por el equipo para construir un sistema de telefonía inalámbrica de corto alcance con módulos de radio NRF24L01 PA+LNA y placas ESP32, en topología de estrella con una central. El proyecto avanzó en tres etapas: un generador de señales con temporizador 555, un enlace transmisor y receptor (TX/RX) controlado por un botón y, como proyecto final, una central con varios teléfonos. Se documentan los problemas de alimentación y configuración que surgieron, un nodo concentrador con cifrado XOR y la evolución del sistema hacia teléfonos que se registran en la central, establecen llamadas y envían mensajes y voz.

La versión final usa el micrófono y la bocina de cada nodo (no los de la computadora) con alimentación por USB desde la PC. Se presentan las versiones, las prácticas, los cambios e intentos, el protocolo de mensajes de 32 bytes, la construcción de los nodos, los diagramas de conexión, la matriz de pruebas (conexiones nt, tt, ntt, tt tt, nttt y tt tt tt tt, con mensajes y con llamadas), la demostración, los alcances, las limitaciones y un marco de seguridad y regulación. En las pruebas se verificaron por etapas el micrófono y la bocina de cada nodo (con programas de prueba y una página web), el registro de los teléfonos en la central y las llamadas entre nodos; los resultados de alcance, estabilidad y pérdida de paquetes se presentan en la sección XV.

Términos de indexación— NRF24L01, ESP32, topología en estrella, nodo central, telefonía inalámbrica, ADPCM, SPI, 2.4 GHz, Web Serial, Arduino.

ÍNDICE GENERAL

RESUMEN 2
ÍNDICE GENERAL 2
LISTA DE FIGURAS 3
LISTA DE TABLAS 4
GLOSARIO DE SIGLAS Y TÉRMINOS 5
I. INTRODUCCIÓN 5
A. Contexto 5
B. Objetivo general 5
C. Objetivos específicos 5
D. Justificación 5
E. Organización del documento 6
II. MARCO TEÓRICO 6
A. Fundamentos de los transmisores de radio 6
B. Evolución de los transmisores: cómo se usaban antes y ahora 6
C. El transceptor NRF24L01+ 6
D. La placa ESP32 6
E. Bus SPI 7
F. Topología en estrella 7
G. Compresión ADPCM 7
H. El temporizador 555 como generador de señales 7
III. COMPARATIVA CON OTRAS TECNOLOGÍAS 7
IV. PLANEACIÓN Y REQUISITOS 8
A. Lista de tareas del proyecto 8
B. Requisitos 8
V. MATERIALES Y HERRAMIENTAS 8
A. Hardware 8
B. Software 8
VI. PROCESO Y METODOLOGÍA 8
A. Etapas del proyecto 8
B. Historial del repositorio y ramas 9
C. Herramientas de colaboración 9
D. Declaración de herramientas de apoyo 9
VII. PRÁCTICAS 9
A. Práctica 1: generador de señales con temporizador 555 9
B. Práctica 2: enlace TX/RX con botón 10
C. Práctica 3: diagnóstico de alimentación y reinicios 10
D. Práctica 4: nodo concentrador (hub o router) 11
E. Práctica 5: central y teléfonos en estrella 11
F. Práctica 6: micrófono y bocina en el nodo 11
G. Práctica 7: doble central redundante 12
H. Otras prácticas 12
VIII. VERSIONES DEL SISTEMA 12
A. v0 a v2: generador, TX/RX y concentrador 12
B. v3: teléfono con central 12
C. v3.1 y v3.2: mensajes, ruido y micrófono 12
D. v4 y v5: redundancia y audio por la PC 12
E. v6: versión final 12
IX. CAMBIOS E INTENTOS 12
A. Bitácora de cambios 12
B. Intentos fallidos y qué se aprendió 13
C. Cambios pendientes o propuestos 13
X. ARQUITECTURA FINAL 13
A. Central 13
B. Teléfonos y canales 14
C. Máquina de estados 14
D. Comandos e indicadores del teléfono 14
E. Doble central 14
XI. CÓMO SE HICIERON LOS NODOS 14
A. Hardware 14

B. Firmware 14
C. Registro en la central 14
D. Lista de verificación 15
XII. PROTOCOLOS Y MENSAJES 15
A. Configuración de radio 15
B. Estructura del paquete 15
C. Tipos de mensaje 15
D. Flujo de una llamada 16
E. Temporizadores 16
F. Nivel de voz (versión con micrófono) 16
G. Ejemplos de mensajes en el Monitor Serie 17
H. Selección de canal y coexistencia con Wi-Fi 17
I. Tiempo en el aire y reintentos 17
J. Protocolo del nodo concentrador (router) 17
K. Formato de ruta propuesto frente al implementado 17
XIII. MICRÓFONO, BOCINA Y VOZ 17
A. Arquitectura final 17
B. Micrófono 17
C. Bocina con amplificador PAM8403 18
D. Nivel de voz en el LED (versión del repositorio) 18
E. Audio comprimido por la PC (versión alterna) 18
F. Prueba de verificación del micrófono y la bocina 19
G. Circuito anterior con transistor (descartado) 19
XIV. DIAGRAMAS DE CONEXIONES 19
A. Nodo: ESP32 y NRF24L01 19
B. Micrófono y bocina 20
C. Alimentación por USB y capacitor 20
D. Circuito del generador 555 20
XV. PRUEBAS 20
A. Metodología 20
B. Instrumentos y condiciones 21
C. Pruebas con mensajes 21
D. Pruebas complementarias 24
E. Matriz de casos de prueba 25
XVI. DEMOSTRACIÓN 26
A. Guion 26
B. Escenarios 26
C. Resultados 26
XVII. PROBLEMAS Y SOLUCIONES 26
XVIII. RETOS, ALCANCES Y LIMITACIONES 26
A. Retos 26
B. Alcances 27
C. Limitaciones 27
XIX. MARCO DE SEGURIDAD Y REGULACIÓN 27
A. Regulación del espectro 27
B. Seguridad eléctrica 27
C. Seguridad de la información 27
D. Uso responsable 27
XX. CONTRIBUCIÓN POR EQUIPO 27
A. Declaración de contribución 28
XXI. TRABAJO FUTURO, LECCIONES APRENDIDAS Y CONCLUSIONES 28
A. Trabajo futuro 28
B. Lecciones aprendidas 28
C. Conclusiones generales 28
D. Conclusiones por equipo 28
REFERENCIAS 28
APÉNDICE A. CÓDIGO FUENTE 30
A.1. Versión v1: enlace TX/RX 30
A.2. Versión v2: hub (receptor en estrella) 30
A.3. Versión v2: nodo con botón 31
A.4. Fragmento explorado: contador de pulsos del 555 32
A.5. Router con cifrado XOR (nodo.ino, rama silviorodriguez) 32
A.6. Emisor (emisor.ino, rama radiantescachorros) 34
A.7. Teléfono con micrófono (Nodo_Telefono.ino, rama equipoqeso) 35
A.8. Central telefónica v3 (Central_Estrella.ino, rama equipoqeso) 42
A.9. Teléfono con audio ADPCM (Nodo_Telefono.ino, rama main) 50
A.10. Interfaz web (NodoTelefonoInterfaz.html) 54
A.11. Versión v6: firmware final con voz (Nodo_Telefono_Voz.ino) 60
A.12. Prueba del micrófono (Prueba_Microfono.ino) 67
A.13. Prueba de la bocina del circuito (Prueba_Bocina_Circuito.ino) 68
A.14. Prueba dúplex (Prueba_Audio_Circuito.ino) 69
APÉNDICE B. DATOS DE LOS MÓDULOS 71
APÉNDICE C. BITÁCORA DE REUNIONES 71

LISTA DE FIGURAS

Fig. 1. Topología en estrella con central (ID 0) y nodos telefónicos. 7

Fig. 2. Fotografía de los componentes utilizados 8
Fig. 3. Enunciado y circuito del generador de señales en protoboard. 9
Fig. 4. Video del circuito armado y funcionando (LED parpadeando) 10
Fig. 5. Montaje físico del TX y RX 10
Fig. 6. Video del receptor al presionar el botón 10
Fig. 7. Montaje con el capacitor de 100 µF junto al módulo 11
Fig. 8. Monitor Serie del router recibiendo y cifrando un mensaje 11
Fig. 9. Monitor Serie del teléfono tras registrarse en la central 11
Fig. 10. Micrófono y bocina conectados al nodo 11
Fig. 11. Medición del nivel del micrófono con el comando /mic 12
Fig. 12. Máquina de estados del teléfono. 14
Fig. 13. Fotografía del nodo terminado 15
Fig. 14. Fotografía de las conexiones del módulo NRF24L01 15
Fig. 15. Formato del paquete de control y de la trama de audio. 15
Fig. 16. Secuencia de una llamada entre dos nodos con la central. 16
Fig. 17. Conexión del micrófono al ESP32. 18
Fig. 18. Conexión del DAC, el PAM8403 y la bocina de 0.5 W. 18
Fig. 19. Flujo de audio de la versión alterna con la PC. 18
Fig. 20. Captura de la interfaz web con los medidores de audio 19
Fig. 21. Conexión entre el ESP32 DevKit V1 y el NRF24L01 PA+LNA. 20
Fig. 22. Fotografía del circuito del generador 555 20
Fig. 23. Topologías de prueba: nt, tt, ntt, tt tt, nttt y tt tt tt tt. 21
Fig. 24. Evidencia de la prueba M1 (conexión nt, mensajes) 21
Fig. 25. Evidencia de la prueba M2 (conexión tt, mensajes) 22
Fig. 26. Evidencia de la prueba M3 (envío de mensaje, mensajes) 22
Fig. 27. Evidencia de la prueba M4 (conexión ntt, mensajes) 23
Fig. 28. Evidencia de la prueba M7 (conexión tt tt tt tt, mensajes) 24
Fig. 29. Video de la prueba de alcance 24

LISTA DE TABLAS

SIGLAS Y TÉRMINOS USADOS 5
TRANSMISORES DE ANTES FRENTE A LOS DE AHORA 6
CARACTERÍSTICAS DEL NRF24L01+ 6
COMPARATIVA DE TECNOLOGÍAS INALÁMBRICAS 7
VENTAJAS Y DESVENTAJAS FRENTE AL PROYECTO 7
REQUISITOS DEL SISTEMA 8
COMPONENTES UTILIZADOS 8
CRONOLOGÍA DEL PROYECTO 9
RAMAS DEL REPOSITORIO 9
HISTORIAL DE CAMBIOS DEL REPOSITORIO 9
FRECUENCIA ESPERADA (EJEMPLO DE CÁLCULO) 10
EXPERIMENTOS DE DIAGNÓSTICO 10
VERSIONES DEL SISTEMA 12
DIFERENCIAS ENTRE VERSIONES 12
CAMBIOS REALIZADOS 12
PARÁMETROS DE LA CENTRAL 13
COMANDOS DE LA CENTRAL 14
COMANDOS DEL MONITOR SERIE DEL TELÉFONO 14
VERIFICACIÓN ANTES DE PROBAR 15
PARÁMETROS DE RADIO 15
CAMPOS DEL PAQUETE DE CONTROL 15
TIPOS DE MENSAJE 15
CÓDIGOS DE RESPUESTA 16
TEMPORIZADORES DEL FIRMWARE 16
SALIDAS DEL MONITOR SERIE 17
SOLAPAMIENTO ENTRE WI-FI Y CANALES DEL NRF24L01 17
TIEMPOS TEÓRICOS A 1 MBPS 17
CÁLCULO DEL FLUJO DE AUDIO (VERSIÓN ALTERNA) 18
PRUEBAS DE BOCINA Y MICRÓFONO 19
ASIGNACIÓN DE PINES DEL TELÉFONO 20
CAÍDA ESTIMADA SEGÚN EL CAPACITOR 20
CONDICIONES DE PRUEBA 21
FICHA M1: CONEXIÓN NT 21
FICHA M2: CONEXIÓN TT 22
FICHA M3: ENVÍO DE MENSAJE 22

FICHA M4: CONEXIÓN NTT 23
FICHA M5: CONEXIÓN TT TT 23
FICHA M6: CONEXIÓN NTTT 23
FICHA M7: CONEXIÓN TT TT TT TT 24
PÉRDIDA DE PAQUETES SEGÚN POTENCIA Y DISTANCIA 24
TIEMPOS DE ESTABLECIMIENTO 24
LLAMADA PROLONGADA 24
MATRIZ DE CASOS DE PRUEBA 25
ESCENARIOS DE LA DEMOSTRACIÓN 26
PROBLEMAS ENCONTRADOS 26
LIMITACIONES DEL SISTEMA 27
MATRIZ DE RIESGOS 27
APORTES VISIBLES EN EL REPOSITORIO 28
CONTRIBUCIÓN POR INTEGRANTE 28
PINES DEL NRF24L01 (MÓDULO DE 8 PINES) 71
PINES DE ARRANQUE DEL ESP32 A EVITAR 71
REGISTRO DE REUNIONES 71

GLOSARIO DE SIGLAS Y TÉRMINOS

TABLA I
SIGLAS Y TÉRMINOS USADOS

Término	Significado
ACK	Acuse de recibo de un paquete.
ADC / DAC	Convertidor analógico-digital / digital-analógico.
ADPCM	Modulación por codificación de pulsos diferencial adaptativa; aquí IMA de 4 bits.
AGC	Control automático de ganancia.
CE	Chip Enable: pin que activa el modo RX/TX del NRF24L01.
CSN	Chip Select Not: selección del NRF24L01 en el bus SPI.
ESP32	Microcontrolador de Espressif con Wi-Fi/Bluetooth, usado como nodo.
GPIO	Pin de entrada/salida de propósito general.
ISM	Banda de radio industrial, científica y médica (2.4 GHz).
LNA / PA	Amplificador de bajo ruido / amplificador de potencia.
MISO / MOSI / SCK	Líneas del bus SPI: datos al maestro, datos al esclavo y reloj.
NRF24L01	Transceptor de 2.4 GHz de Nordic Semiconductor.
PAM8403	Amplificador de audio clase D de 3 W; en el nodo maneja la bocina de 0.5 W
Pipe	Canal lógico de recepción con dirección propia (hasta 6 por chip).
Push-to-talk	Hablar por turnos manteniendo pulsado un botón (BOOT en el nodo)
RX / TX	Recepción / transmisión.
SoC	Sistema en un chip.
SPI	Serial Peripheral Interface.
UART	Transmisor-receptor asíncrono universal (puerto serie).
Web Serial	API del navegador para comunicarse con puertos serie.
Nodo / Teléfono	Placa ESP32 + NRF24L01 identificada con MI_ID (1 a 254).
Central	Nodo con ID 0 que registra teléfonos y conmuta llamadas.

I. INTRODUCCIÓN

A. Contexto

La comunicación inalámbrica es la base de tecnologías cotidianas como el Wi-Fi, el Bluetooth, la telefonía celular y las redes de sensores. En la materia de Señales en telecomunicaciones se propuso un proyecto evaluativo en tres etapas. Primero, un generador de señales construido con componentes discretos y un temporizador 555, capaz de producir señales eléctricas que pudieran usarse después para emisiones inalámbricas. Segundo, un enlace transmisor y receptor (TX/RX) con dos módulos NRF24L01 PA+LNA gobernados por placas ESP32 [1], [2]. Tercero, y como proyecto final, una red de teléfonos con nodo central en topología de estrella.

En el camino se corrigieron errores de código, se resolvieron fallas de alimentación que reiniciaban las placas, se pasó por un nodo concentrador con cifrado XOR y se llegó a un sistema en el que varios teléfonos se registran en la central, establecen llamadas entre sí, intercambian mensajes y usan su propio micrófono y bocina, alimentados por el USB de la computadora. Este documento narra y justifica ese camino.

B. Objetivo general

Diseñar, implementar y probar una red inalámbrica de teléfonos en topología de estrella con nodo central, usando módulos NRF24L01 PA+LNA y ESP32, capaz de establecer llamadas y enviar mensajes entre varios nodos.

C. Objetivos específicos

1. Construir un generador de señales con un temporizador 555 y comprender cómo se genera una señal oscilante.
2. Establecer un enlace TX/RX confiable entre dos ESP32 con NRF24L01 y comprender su configuración (direcciones, canal, potencia, reintentos).
3. Diagnosticar y resolver las fallas de alimentación y de pines que afectan al módulo de radio.
4. Implementar una central que acepte registros y abra conexiones hacia todos los teléfonos de la red.
5. Definir un protocolo de mensajes de 32 bytes para registro, llamada, conexión, texto, voz y cierre.
6. Integrar un micrófono y una bocina en cada nodo, alimentados por el USB de la PC.
7. Validar el sistema con pruebas de conexión con distintas combinaciones de centrales y teléfonos.

D. Justificación

El NRF24L01 permite construir y controlar toda la capa de enlace de una red propia a un costo de pocos dólares. Esto

hace que el proyecto sea ideal para entender los fundamentos de los transmisores de radio: modulación, canalización, potencia, direccionamiento, retransmisión y coexistencia en la banda ISM de 2.4 GHz, conceptos que de otro modo quedan ocultos en tecnologías comerciales.

E. Organización del documento

La sección II presenta el marco teórico y la III compara el NRF24L01 con otras tecnologías. Las secciones IV a VII describen la planeación, los materiales, el proceso y las prácticas. Las secciones VIII y IX recogen las versiones, los cambios y los intentos. Las secciones X a XIV documentan la arquitectura final, los nodos, los protocolos, el audio y los diagramas de conexión. Las secciones XV y XVI muestran las pruebas y la demostración, y de la XVII a la XXI se discuten problemas, retos, alcances, limitaciones, seguridad, contribuciones y conclusiones.

El proyecto se desarrolló en equipo entre septiembre y octubre de 2026 para la materia Señales en telecomunicaciones. Comenzó con el generador de señales 555 (alrededor del 12 de septiembre) y terminó con una red de teléfonos con nodo central. La motivación fue entender, construyendo un sistema propio, cómo se transmite información por radio.

Cada equipo armó su transmisor con los componentes que tuvo a su alcance, por lo que los nodos no son idénticos: no todos usaron la misma bocina, el mismo micrófono ni el mismo módulo PAM8403, y algunos emplearon el micrófono y las bocinas de la computadora portátil mediante la interfaz web en lugar de los del circuito. Esta documentación describe la configuración de referencia del nodo del autor; los resultados de otros nodos pueden variar por esas diferencias de componentes.

II. MARCO TEÓRICO

A. Fundamentos de los transmisores de radio

Un transmisor de radio convierte información (voz, datos) en ondas electromagnéticas que viajan por el espacio sin cables. Su funcionamiento se apoya en las ecuaciones de Maxwell, que demostraron que una corriente eléctrica variable genera un campo electromagnético capaz de propagarse. Un transmisor consta, en esencia, de tres bloques:

- **Oscilador:** genera la onda portadora a la frecuencia de transmisión.
- **Modulador:** combina la información con la portadora variando su amplitud (AM), frecuencia (FM) o fase; en el NRF24L01 se usa modulación GFSK, una variante de FM digital [1].
- **Amplificador y antena:** elevan la potencia y la irradian al espacio; el módulo PA+LNA añade además un amplificador de potencia en transmisión y uno de bajo ruido en recepción.

B. Evolución de los transmisores: cómo se usaban antes y ahora

Entre 1895 y 1901, Guglielmo Marconi demostró comunicaciones prácticas y logró la primera señal transatlántica. Sus transmisores de chispa (spark-gap) ocupaban salas enteras, solo emitían código Morse y servían para telegrafía entre barcos y estaciones. En 1906, Reginald Fessenden logró la primera transmisión de voz con un alternador de alta frecuencia, y ese mismo año el triodo de De Forest abrió la era de los tubos de vacío, con transmisores más estables que dieron origen a la radiodifusión en AM y, más tarde, en FM (Armstrong). El transistor (1947) y los circuitos integrados miniaturizaron los equipos, y hoy un transceptor completo cabe en un chip de pocos milímetros.

TABLA II
TRANSMISORES DE ANTES FRENTE A LOS DE AHORA

Aspecto	Antes (1900–1950)	Ahora (NRF24L01)
Tecnología	Chispa, alternador, tubos de vacío	Circuito integrado digital
Tamaño	Habitaciones o muebles	Módulo de 15 × 29 mm
Potencia	Cientos a miles de vatios	Del orden de mW
Uso	Telegrafía, radiodifusión, uso militar	IoT, control remoto, redes de sensores, telefonía de corto alcance
Señal	Analógica (Morse, AM, FM)	Digital en paquetes con ACK
Ajuste	Bobinas y capacitores manuales	Registros por SPI y software
Acceso	Gobiernos, empresas, radioaficionados	Estudiantes y aficionados

C. El transceptor NRF24L01+

El NRF24L01+ de Nordic Semiconductor es un transceptor de 2.4 GHz que opera en la banda ISM con modulación GFSK, 126 canales de 1 MHz y velocidades de 250 kbps, 1 Mbps o 2 Mbps [1]. Incluye el protocolo Enhanced ShockBurst, que gestiona direccionamiento, CRC, acuse de recibo (ACK) y retransmisión automática. Puede escuchar hasta seis direcciones simultáneas (pipes) y transporta cargas útiles de hasta 32 bytes. Se controla por SPI. La variante PA+LNA incorpora amplificadores y antena externa, lo que aumenta el alcance a costa de un mayor consumo de corriente, factor clave en las fallas que se documentan más adelante.

TABLA III
CARACTERÍSTICAS DEL NRF24L01+

Parámetro	Valor
Banda	2.400–2.525 GHz (ISM)
Canales	126, de 1 MHz (frecuencia = 2400 + canal MHz)
Velocidades	250 kbps, 1 Mbps, 2 Mbps
Modulación	GFSK
Carga útil	1 a 32 bytes por paquete
Direcciones	3 a 5 bytes; 6 pipes de recepción
Interfaz	SPI (hasta 10 Mbps)
Alimentación	1.9–3.6 V (no tolera 5 V)
Variante PA+LNA	Mayor alcance y mayor consumo en TX

D. La placa ESP32

El ESP32 es un microcontrolador de doble núcleo con SPI, UART, ADC y DAC [2]. Dos detalles resultaron decisivos en el proyecto. Primero, algunos GPIO son pines de arranque (strapping) como GPIO0, GPIO2, GPIO5, GPIO12 y GPIO15, cuyo nivel al encender determina el modo de arranque, y los GPIO6 a GPIO11 están conectados a la

memoria flash interna y no deben usarse. Segundo, el regulador de 3.3 V de la placa alimentada por USB tiene capacidad limitada, y los picos de corriente del NRF24L01 PA+LNA pueden provocar caídas de tensión (brownout) y reinicios.

E. Bus SPI

SPI es un bus síncrono maestro-esclavo de cuatro líneas: SCK (reloj), MOSI (datos del maestro al esclavo), MISO (datos del esclavo al maestro) y CSN (selección). En el proyecto el ESP32 es el maestro y el NRF24L01 el esclavo. Además se usa la línea CE, que no pertenece a SPI, para conmutar el módulo entre los modos de escucha y transmisión.

F. Topología en estrella

En una red en estrella todos los nodos se comunican con un elemento central que coordina el tráfico. Su ventaja es la sencillez de administración y la facilidad para agregar nodos; su desventaja es que la central es un punto único de falla. En este proyecto la central registra a los teléfonos, recibe sus solicitudes de llamada, avisa al destinatario y asigna un canal de radio a la pareja de nodos que hablan.

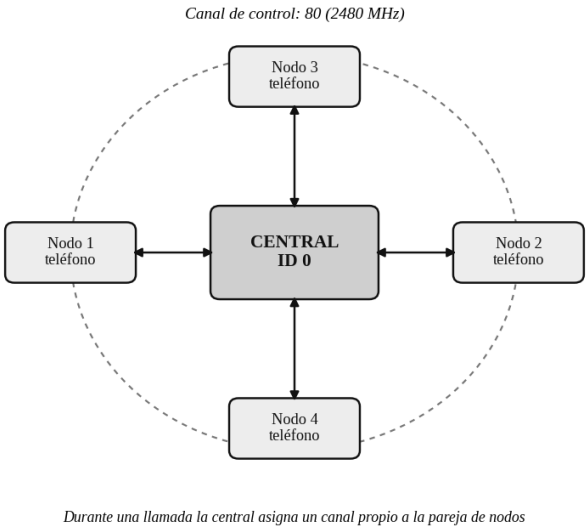


Fig. 1. Topología en estrella con central (ID 0) y nodos telefónicos.

G. Compresión ADPCM

La modulación ADPCM codifica la diferencia entre la muestra real y una predicción, usando un paso que se adapta a la señal. La variante IMA reduce cada muestra de 16 bits a 4 bits (4:1) con calidad adecuada para voz [6]. A 8 kHz resulta un flujo de 32 kbps, compatible con el ancho de banda práctico del NRF24L01.

H. El temporizador 555 como generador de señales

El circuito integrado 555 en configuración astable genera una onda cuadrada continua. Un capacitor se carga y descarga a través de dos resistencias, y dos comparadores internos conmutan la salida cuando la tensión del capacitor cruza 1/3 y 2/3 de la alimentación. La frecuencia es aproximadamente $f = 1.44 / ((R1 + 2 \cdot R2) \cdot C)$ y el ciclo de trabajo depende de la relación entre las resistencias. Un potenciómetro en R2 permite variar la frecuencia a mano. Esta señal oscilante es la materia prima de cualquier emisión: un oscilador es el primer bloque de un transmisor.

Los conceptos del marco teórico se aplicaron de forma directa en las prácticas: la frecuencia del temporizador 555 en la práctica 1, y el canal, la dirección y la potencia del NRF24L01 en las prácticas 2 a 6.

III. COMPARATIVA CON OTRAS TECNOLOGÍAS

Para justificar la elección del NRF24L01 se comparan sus características típicas con las de otras tecnologías inalámbricas que podrían haberse usado en el mismo proyecto. Los valores son típicos y varían con el modelo y las condiciones de uso [1], [2], [10], [11], [12].

TABLA IV
COMPARATIVA DE TECNOLOGÍAS INALÁMBRICAS

Tecnología	Banda	Tasa típica	Alcance típico
NRF24L01+	2.4 GHz	0.25–2 Mbps	Decenas de m; cientos con PA+LNA
Wi-Fi (802.11)	2.4 / 5 GHz	Decenas a cientos de Mbps	Decenas de m
Bluetooth	2.4 GHz	1–3 Mbps (BLE 1–2 Mbps)	10–100 m
ESP-NOW	2.4 GHz	≈1 Mbps, 250 B/paquete	Decenas a cientos de m
LoRa	Sub-GHz	0.3–50 kbps	Km

TABLA V
VENTAJAS Y DESVENTAJAS FRENTE AL PROYECTO

Tecnología	A favor	En contra
NRF24L01+	Costo mínimo, control total de la capa de enlace, sin punto de acceso, consumo bajo, ideal para aprender	Paquete de 32 B, sin seguridad integrada, interferencia con Wi-Fi
Wi-Fi	Alta velocidad, infraestructura existente	Necesita router o AP, mayor consumo y complejidad
Bluetooth	Estándar, audio nativo (perfiles)	Pocos nodos, topología restringida
ESP-NOW	Sin AP, integrado en el ESP32, latencia baja	Solo ESP32, el radio es el de Wi-Fi
LoRa	Muy largo alcance, bajo consumo	Tasa insuficiente para voz

Para voz, Wi-Fi y Bluetooth serían más cómodos pero ocultarían los detalles del enlace. LoRa no alcanza la tasa necesaria. ESP-NOW sería una alternativa técnicamente viable, pero el NRF24L01 permite además experimentar con canales, direcciones y potencia en un chip independiente de la placa. Esa fue la razón para mantenerlo en todas las versiones.

Además, el módulo es económico, fácil de conseguir y se maneja con una librería conocida (RF24), lo que permitió que cada integrante armara su propio nodo.

IV. PLANEACIÓN Y REQUISITOS

A. Lista de tareas del proyecto

El repositorio del equipo contiene un documento de tareas (archivo issues), creado y revisado el 29 de septiembre, con las expectativas del proyecto. Su contenido se resume así: crear un nodo central que, en fase de pruebas, cumpla dos objetivos: aceptar conexiones y datos de todos los elementos de la red, y abrir conexiones y enviar datos a todos ellos [8].

Una vez que la central establece ambas conexiones, el siguiente paso es especificar la ruta del mensaje. Cada mensaje debe incluir en su cadena los nodos por los que atraviesa, con el formato: código del emisor, código del receptor, código del nodo concentrador y mensaje; por ejemplo `001 -> 012 -> NC001 -> "hola"`. Se debe comprobar que el mensaje llegue y sea interpretado únicamente por el dispositivo con ese código.

Además, todos los dispositivos pueden transmitir y recibir (RX/TX) y alternan su función según el rol solicitado: RX es el estado natural, de escucha y espera; TX es el estado al que cambia al recibir la instrucción de enviar. El ciclo es: cambia de modo, genera la cadena de envío, establece conexión con el nodo central, envía el mensaje y vuelve a modo de espera [8].

B. Requisitos

TABLA VI
REQUISITOS DEL SISTEMA

ID	Requisito	Origen
RF1	La central acepta registros y datos de todos los nodos	issues
RF2	La central abre conexiones y envía datos a todos los nodos	issues
RF3	Los nodos alternan RX (espera) y TX (envío)	issues
RF4	Solo el destinatario interpreta el mensaje	issues
RF5	Un nodo puede llamar a otro y la pareja se comunica en un canal propio	Equipo
RF6	Mensajes de texto durante la llamada	Equipo
RF7	Audio de voz entre los nodos	Equipo
RNF1	Paquete de radio de 32 bytes	NRF24L01
RNF2	Funcionar con ESP32 DevKit y SuperMini C3	Código
RNF3	Costo bajo y componentes comunes	Equipo

Los criterios que se tomaron en cuenta son: funcionamiento de la red, documentación en formato IEEE, evidencias, pruebas y demostración.

V. MATERIALES Y HERRAMIENTAS

A. Hardware

TABLA VII
COMPONENTES UTILIZADOS

Componente	Cant.	Función
ESP32 DevKit V1	1 por nodo	Microcontrolador de cada nodo y de la central
NRF24L01 PA+LNA con antena	1 por nodo	Enlace de radio
Capacitor electrolítico 100 µF	3 por nodo	Desacoplo del NRF24, de la alimentación del PAM8403 y acople de la entrada de audio
Protoboard y jumpers	1 por nodo	Conexiones
Cable USB de datos y PC	1 por nodo	Programación, Monitor Serie y alimentación
LED y resistencia de 330 Ω	1 por nodo	Indicador de estado (GPIO12)
NE555, resistencias 220 Ω y 1 kΩ, potenciómetro 10 kΩ, LED, capacitor 100 µF	1 juego	Generador de señales (práctica 1)
Micrófono analógico con amplificador (tipo MAX9814)	1 por nodo	Captura de voz en el nodo
Bocina 8 Ω, 0.5 W	1 por nodo	Salida de audio
Módulo amplificador PAM8403 (HW-104)	1 por nodo	Amplificador clase D de la bocina; recibe la señal del DAC (GPIO25) y 5 V del USB
ESP32-S3 SuperMini o C3 SuperMini	Opcional	Central y teléfonos alternos

B. Software

- Arduino IDE 2.x en macOS, con el núcleo de ESP32 (versión 3.3.x) y la placa «ESP32 Dev Module».
- Librería RF24 de TMRh20 [3].
- esptool v5.3.1 para la carga del firmware.
- Navegador Chrome o Edge de escritorio, solo para la interfaz web alterna con Web Serial [7].
- GitHub para el control de versiones del código y Google Docs para la documentación [8].

Las comprobaciones se hicieron principalmente con el Monitor Serie, el Serial Plotter del IDE y las páginas web de prueba de audio (Web Serial).

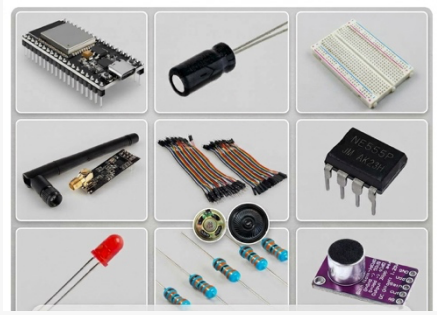


Fig. 2. Fotografía de los componentes utilizados

VI. PROCESO Y METODOLOGÍA

A. Etapas del proyecto

El trabajo avanzó por etapas, cada una construida sobre la anterior y con pruebas antes de pasar a la siguiente: generador de señales, enlace TX/RX y, al final, la central con teléfonos en estrella. Las fechas del repositorio provienen del historial de GitHub [8]; las demás son aproximadas.

TABLA VIII
CRONOLOGÍA DEL PROYECTO

Etapa	Descripción	Fecha
1	Generador de señales con 555	≈12 sep
2	Enlace TX/RX con botón y LED	≈18 sep
3	Corrección de código y de librería	≈18 sep
4	Diagnóstico de reinicios y alimentación	sep–oct
5	Creación del repositorio y tareas	24–29 sep
6	Nodo concentrador con XOR	28 sep–5 oct
7	Central y teléfonos en estrella	2–8 oct
8	Mensajes de la central, micrófono y VOZ	8–9 oct
9	Doble central y audio por la PC	12–14 oct
10	Micrófono y bocina propios, pruebas y demostración	15–20 oct

B. Historial del repositorio y ramas

El repositorio del equipo tiene una rama principal y cuatro ramas de trabajo, una por subequipo. Cada una conserva su propia versión del firmware, lo que permitió desarrollar la central, los teléfonos y las variantes en paralelo [8].

TABLA IX
RAMAS DEL REPOSITORIO

Rama	Contenido
main	Nodo_Telefono.ino con audio ADPCM, interfaz web y documento issues
equipoqeso	Central_Estrella (v2 y v3), central S3, Nodo_Telefono con micrófono
radiantescachorros	Emisor y receptor, nodo, doble central y teléfono con doble central
silviorodriguez	Nodo_Telefono con ADPCM e interfaz web
chuyemiliano	Nodo_Telefono (primera versión de los teléfonos)

TABLA X
HISTORIAL DE CAMBIOS DEL REPOSITORIO

Fecha	Usuario	Cambio
24 sep	Maztherzanch	Creación del repositorio
28–29 sep	engsecure	nodo.ino: router con cifrado XOR
29 sep	Maztherzanch	Documento de tareas (issues) y revisión
2 oct	f1sher	emisor.ino y receptor.ino
2 oct	VTorres	Central para ESP32-S3
2 oct	emilianosDev	Programa de los teléfonos
8 oct	qesin	Central v2 (mensajes y ruido)
8 oct	Dyisus05	Ajuste del nodo y Central_Telefonica_v2.zip
9 oct	qesin	Teléfono con micrófono y central v3
12 oct	f1sher	Doble central redundante
13–14 oct	engsecure	Nodo con ADPCM e interfaz web

Cada usuario de GitHub se asocia con el nombre real del integrante en la sección XX.

C. Herramientas de colaboración

El equipo trabajó con un repositorio de GitHub (código del nodo, interfaz web y documento de tareas) y con un Google Doc titulado «Central telefónica», donde se registran las pruebas de conexión. Las tareas se repartieron mediante el documento de tareas del repositorio y la coordinación se hizo por mensajería y en las sesiones de laboratorio.

D. Declaración de herramientas de apoyo

Durante el desarrollo se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo para depurar código, explicar errores de carga y de compilación, proponer el firmware de voz y los programas de prueba del micrófono y la bocina, y redactar borradores de esta documentación. El equipo revisó el contenido, armó y probó el hardware y es responsable del resultado final.

VII. PRÁCTICAS

Las prácticas se realizaron en este orden: primero el generador de señales con temporizador 555, después el enlace transmisor y receptor (TX/RX) y, como proyecto final, la central con teléfonos en topología de estrella. Cada práctica se documenta con objetivo, materiales, procedimiento y resultados, acompañados de fotografías o videos cuando se dispone de ellos.

A. Práctica 1: generador de señales con temporizador 555

Enunciado. Utilizando componentes discretos de electrónica, crear un dispositivo capaz de generar señales eléctricas que posteriormente puedan usarse para emisiones inalámbricas. **Materiales.** Protoboard, circuito integrado NE555, resistencias de 220 Ω y 1 kΩ, potenciómetro (resistencia variable) de 10 kΩ, LED, capacitor de 100 µF, cables y fuente de energía (batería de 9 V).

Fundamento. El 555 en modo astable es un oscilador: el capacitor se carga y descarga entre 1/3 y 2/3 de la tensión de alimentación a través de las resistencias, y la salida (pin 3) alterna entre nivel alto y bajo. La frecuencia es aproximadamente $f = 1.44 / ((R1 + 2 \cdot R2) \cdot C)$, por lo que al girar el potenciómetro, que forma parte de R2, cambia la frecuencia de la señal. El LED en la salida permite ver la oscilación cuando es lenta.

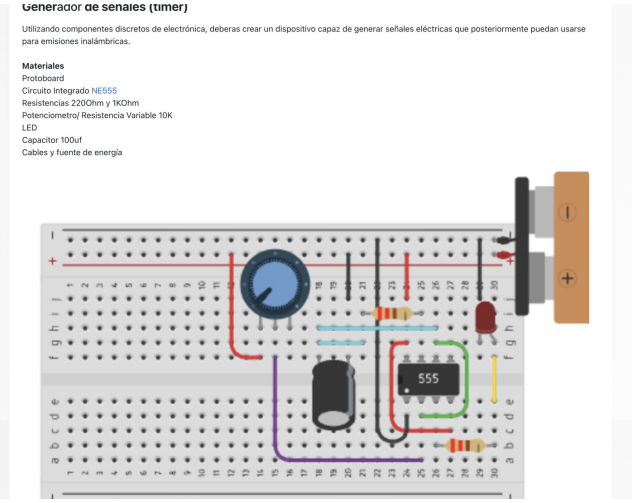


Fig. 3. Enunciado y circuito del generador de señales en protoboard.

TABLA XI
FRECUENCIA ESPERADA (EJEMPLO DE CÁLCULO)

R1	R2 (pot. + fija)	C	f aprox.
1 kΩ	1 kΩ	100 µF	4.8 Hz
1 kΩ	11 kΩ	100 µF	0.63 Hz

La práctica comprobó que la frecuencia cambia al girar el potenciómetro y que el LED parpadea más rápido o más lento según su posición.

Link de video

Fig. 4. Video del circuito armado y funcionando (LED parpadeando)

B. Práctica 2: enlace TX/RX con botón

Objetivo. Establecer comunicación entre dos ESP32 con NRF24L01: un transmisor que envía el estado de un botón y un receptor que lo interpreta. **Materiales.** Dos ESP32 DevKit, dos NRF24L01 PA+LNA, botón en GPIO25, cables y la librería RF24.

Procedimiento. (1) Conectar el módulo por SPI (SCK 18, MISO 19, MOSI 23, CSN 4) y definir CE. (2) Instalar la librería RF24 de TMRh20 y programar el transmisor con una dirección de 5 caracteres, que cambió de «00001» a «00009». (3) Programar el receptor con la misma dirección. (4) Abrir el Monitor Serie a 115200 baudios.

Hallazgos de código. El transmisor original invertía el valor del botón con una segunda negación y enviaba el estado contrario al real. La potencia en `RF24_PA_MIN` desperdiciaba la ventaja del módulo PA+LNA, `stopListening()` se repetía en cada ciclo cuando bastaba una vez en `setup()` y el botón sin resistencia podía quedar flotante, por lo que se usó `INPUT_PULLDOWN`.

Resultado: con la misma dirección, canal y potencia en el transmisor y el receptor, el receptor refleja el estado del botón. Al quitar la doble inversión el LED coincidió con el botón. Los cambios de código quedaron registrados en la tabla XV.

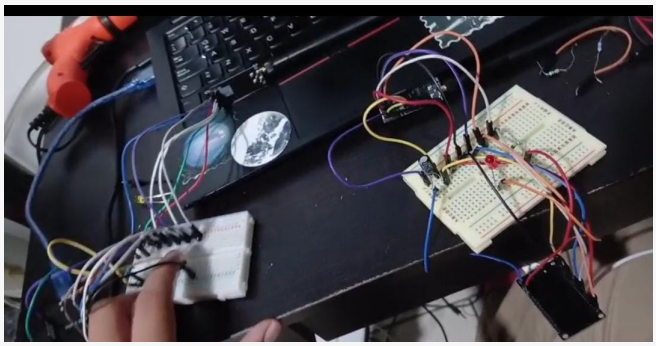


Fig. 5. Montaje físico del TX y RX

Link de video

Fig. 6. Video del receptor al presionar el botón

C. Práctica 3: diagnóstico de alimentación y reinicios

Al transmitir, el ESP32 entraba en un ciclo de reinicios con mensajes como `invalid header: 0xffffffff` y `rst:0x7 (TG0WDT_SYS_RESET)`. Se probó, de forma sistemática, si la causa era el pin CE en GPIO2 (pin de arranque), la detección del módulo por SPI o la alimentación. El análisis está en la sección XVII y la estimación de la caída de tensión en la XIV.

TABLA XII
EXPERIMENTOS DE DIAGNÓSTICO

Prueba	Configuración	Resultado
E1	begin() con verificación y retardo de 2 s, sin write()	Estable
E2	write() con PA_LOW, CE en GPIO2	Reinicio durante la transmisión
E3	write() con CE en GPIO16 (no strapping)	Reinicio: GPIO2 no es la causa
E4	CE en GPIO2 y capacitor de 100 µF	≈10 ciclos «Sin ACK» y reinicio
E5	Capacitor + setRetries(5,5)	Estable
E6	Regulador de 3.3 V externo	No fue necesario: el capacitor bastó

Resultado: la placa quedó estable con el capacitor de 100 µF junto al módulo, la potencia en PA_LOW, setRetries(5,5) y el pin CE en GPIO22. Los reinicios aparecían cuando fallaban los ACK y la corriente media subía por los reintentos, lo que confirma que el origen era la alimentación y no el pin CE.

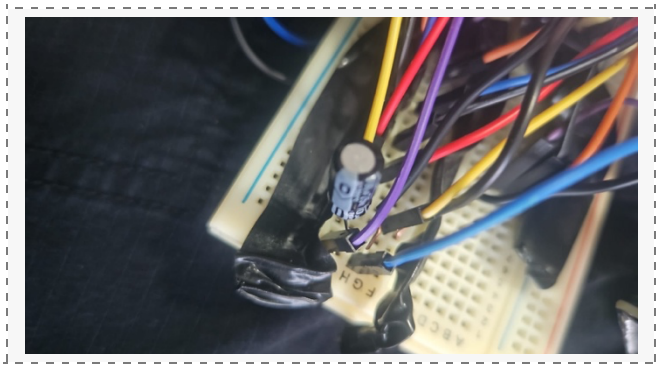


Fig. 7. Montaje con el capacitor de 100 µF junto al módulo

D. Práctica 4: nodo concentrador (hub o router)

Siguiendo el documento de tareas del repositorio, se construyó un nodo concentrador que recibe de varios nodos y reenvía al destino. Se desarrollaron dos variantes. En la primera, el hub abre una pipe de lectura por nodo y responde a cada uno; los nodos usan una dirección de subida (`NodU1`, `NodU2`…) y otra de bajada (`NodD1`…), porque en RF24 las pipes 1 a 5 deben compartir los primeros 4 bytes de la dirección.

En la segunda variante (archivos nodo.ino, emisor.ino y receptor.ino del repositorio) el router escucha en seis direcciones, define direcciones de destino y usa un paquete con identificador de destino, longitud y 16 bytes de datos que se **cifran con XOR** con una clave de texto fija, de modo que solo el destino que conoce la clave interpreta el mensaje. Un LED indica la recepción [8].

Según el historial del repositorio, la variante con una pipe por nodo y la variante router con XOR (nodo.ino, emisor.ino y receptor.ino) las desarrollaron integrantes distintos [8]; ambas se probaron con un router y dos nodos destino y llevaron al diseño de la central.

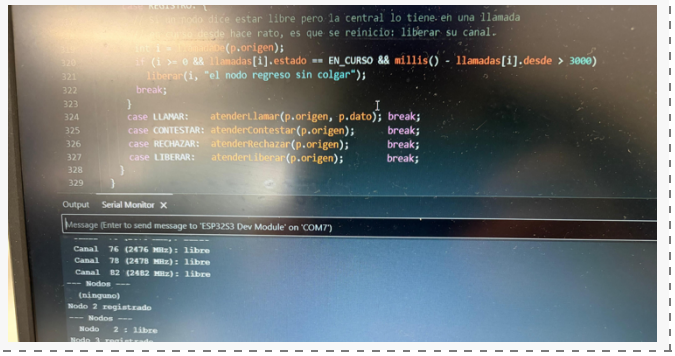


Fig. 8. Monitor Serie del router recibiendo y cifrando un mensaje

E. Práctica 5: central y teléfonos en estrella

Es el proyecto final. Se programaron una **central** (ID 0) y varios **teléfonos** (ID 1 a 254). Todos los teléfonos cargan el mismo código y solo cambian `MI_ID`. Los comandos se escriben en el Monitor Serie: `/llamar ID`, `/contestar`, `/rechazar`, `/colgar` y `/estado`. Se verificó que los pines del cableado coincidan con los del código, y la configuración final está en la sección XIV.

Resultado: la central (ID 0) registra a cada teléfono al encenderse; todos usan el mismo código con un MI_ID distinto. Las llamadas se verificaron entre pares de nodos y, para trabajar con las centrales de otros compañeros, se hicieron configurables el nombre de la red y el canal de control. Los IDs y el número de teléfonos probados se anotan en las fichas de la sección XV.

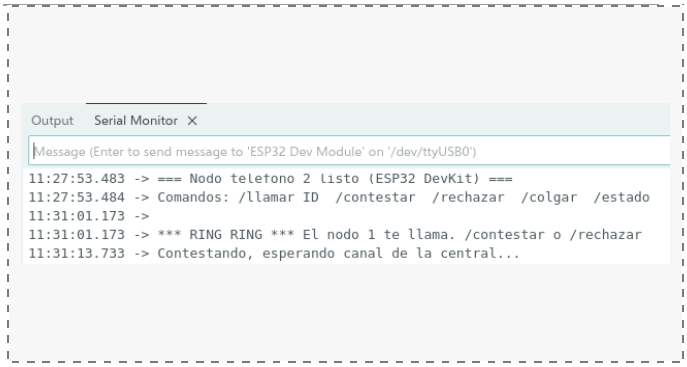


Fig. 9. Monitor Serie del teléfono tras registrarse en la central

F. Práctica 6: micrófono y bocina en el nodo

Cada teléfono incorpora un micrófono analógico (tipo MAX9814) conectado a GPIO34 y una bocina de 0.5 W y 8 Ω accionada desde el DAC (GPIO25) mediante el amplificador PAM8403. Se usan el micrófono y la bocina del propio nodo, no los de la computadora, y todo el conjunto se alimenta por el USB de la PC. El firmware final (Nodo_Telefono_Voz) captura la voz a 8 kHz, la comprime con ADPCM, la envía por radio y reproduce la que recibe; se habla por turnos con el botón BOOT. El circuito y la prueba de verificación están en las secciones XIII y XIV.

Resultado: en la primera prueba del micrófono solo se oyó estática, después un chillido breve y luego silencio. Se atribuyó a la realimentación entre la bocina de la PC y el micrófono y al control automático de ganancia del módulo, y se corrigió separando los elementos y bajando el volumen. Las pruebas de audio realizadas se resumen en la tabla XXIX.

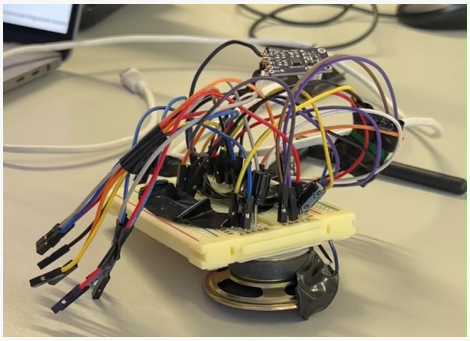


Fig. 10. Micrófono y bocina conectados al nodo

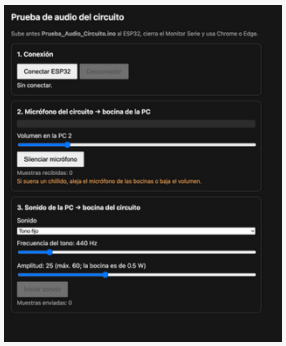


Fig. 11. Medición del nivel del micrófono con el comando /mic

G. Práctica 7: doble central redundante

Una rama del repositorio añade una segunda central con ID 100. Los teléfonos recuerdan cuál central está activa y, si una no responde, conmutan automáticamente a la otra y vuelven a registrarse. Para probar la conmutación se apaga la central 0 y se observa que el teléfono se registra en la central 100.

H. Otras prácticas

Otras actividades de la práctica final: pruebas del micrófono y la bocina del circuito (sección XIII.F), configuración del nombre de red y del canal de control del teléfono para operar con las centrales de otros compañeros, y carga de firmware con esptool (incluido el borrado completo de la flash).

VIII. VERSIONES DEL SISTEMA

El sistema evolucionó en versiones sucesivas. Todas conservan el hardware base (ESP32 + NRF24L01 + capacitor) y cambian el firmware y la arquitectura. Las fechas provienen del historial del repositorio [8].

TABLA XIII
VERSIONES DEL SISTEMA

Ver.	Descripción	Fecha
v0	Generador de señales con 555 (solo hardware)	≈12 sep
v1	Enlace TX/RX con botón	≈18 sep
v2	Nodo concentrador/router, con XOR	28–29 sep
v3	Teléfono con central (texto, canales)	2 oct
v3.1	Central v2: mensajes y monitoreo de ruido	8 oct
v3.2	Micrófono y nivel de voz; central v3	9 oct
v4	Doble central redundante (ID 0 y 100)	12 oct
v5	Audio ADPCM por interfaz web (alterna)	13–14 oct
v6	Final: micrófono y bocina propios (PAM8403), voz ADPCM, USB de la PC	15–20 oct

A. v0 a v2: generador, TX/RX y concentrador

El generador 555 fue la primera práctica. El enlace TX/RX incluía los errores de la práctica 2. El concentrador introdujo el concepto de red con varias direcciones y el cifrado XOR. El código v1 se incluye en el apéndice A.1.

B. v3: teléfono con central

El control se centraliza. Los teléfonos nunca se llaman directamente: piden a la central que establezca la comunicación, y esta asigna un canal propio a cada pareja para dejar libre el canal de control (80). Existen versiones de la central para ESP32-S3 SuperMini, ESP32-C3 SuperMini y ESP32 DevKit.

C. v3.1 y v3.2: mensajes, ruido y micrófono

La central v2 puede enviar mensajes a todos los nodos o a uno (aunque estén en llamada) y monitorea de forma continua las interferencias para asignar solo canales limpios. La v3 añade el paquete VOZ: cada teléfono mide su micrófono y envía un nivel de 0 a 255 a la pareja.

D. v4 y v5: redundancia y audio por la PC

La doble central hace tolerante al sistema a la falla de una central. La versión con interfaz web transmite audio real comprimido con ADPCM usando el micrófono y las bocinas de la computadora; se conserva como alternativa y se detalla en la sección XIII.

E. v6: versión final

La versión final usa el micrófono y la bocina del propio nodo, con alimentación por USB desde la PC, dentro de la topología de estrella con central. El firmware final se llama Nodo_Telefono_Voz (octubre de 2026): captura el micrófono a 8 kHz, comprime con ADPCM, transmite paquetes VOZ de 32 bytes sin ACK y reproduce la voz recibida por el DAC y el PAM8403. Frente a la v3.2, que solo enviaba el nivel de voz, añade voz real, el botón BOOT como push-to-talk, los comandos /tono, /nivel, /vol y /gan, y parámetros de red configurables.

TABLA XIV
DIFERENCIAS ENTRE VERSIONES

Aspecto	v1	v3	v3.2	v6
Topología	Punto a punto	Central	Central	Central
Datos	Botón	Texto	Texto, nivel de voz	Texto y voz
Direcc.	Fija	[id]NODE	[id]NODE	[id]NODE
Audio	No	No	Nivel en LED	Mic y bocina
Compresión	No	No	No	ADPCM 4 bits
Bocina	No	No	No	DAC + PAM8403
Red	Fija	NODE / 80	NODE / 80	Configurable

Entre la v3.2 y la v6 hubo versiones de prueba: Prueba_Microfono (micrófono del circuito en la bocina de la PC), Prueba_Bocina_Circuito (tonos en la bocina del circuito), Prueba_Audio_Circuito (prueba dúplex) y Nodo_Telefono_corregido (red configurable y mensajes de llamada fallida), todas durante octubre.

IX. CAMBIOS E INTENTOS

A. Bitácora de cambios

TABLA XV
CAMBIOS REALIZADOS

N.º	Cambio	Motivo / resultado
1	Quitar la doble inversión del botón	Se enviaba el estado contrario al real
2	Dirección «00001» a «00009»	Evitar coincidencias; igual en RX

N.º	Cambio	Motivo / resultado
3	PA_MIN a PA_LOW	Aprovechar el PA+LNA con menor consumo
4	stopListening() solo en setup()	Evitar llamadas redundantes
5	Botón con INPUT_PULLDOWN	Evitar pin flotante
6	Instalar librería RF24 (TMRh20)	Error «nRF24L01.h: No such file»
7	Ignorar el panel Debug (OpenOCD)	No hay adaptador JTAG
8	CE: GPIO2, 16, 27 y 22	Pruebas de pin de arranque; nodo final en GPIO22
9	Capacitor de 100 µF	Mitiga caídas de tensión al transmitir
10	setRetries(5,5)	Menos corriente por reintentos
11	Velocidad de carga y botón BOOT	Error de esptool
12	De hub/router a protocolo con central	Llamadas, canales y estados
13	Central v2: mensajes y monitoreo de ruido	Canales limpios y avisos
14	Micrófono con paquete VOZ	Nivel de voz entre nodos
15	Doble central con ID 100	Redundancia
16	Audio ADPCM por interfaz web	Voz real vía PC (alterna)
17	Micrófono y bocina propios, USB de la PC	Versión final sin audio de la computadora
18	Amplificador PAM8403 en lugar del transistor	Más volumen y menos consumo en reposo; 5 V del USB
19	Firmware con voz ADPCM en el nodo	Voz real con micrófono y bocina propios
20	NOMBRE_RED y CANAL_CONTROL configurables	Las llamadas con otras centrales fallaban por red y canal distintos (NODE/80 frente a ITA1/76)
21	Mensaje [LLAMADA FALLIDA] con el motivo	Antes la llamada fallaba en silencio
22	Definir struct Adpcm antes de la primera función	Error de compilación por los prototipos que genera el IDE
23	Amplitud del DAC limitada (AMP_MAX)	El PAM8403 amplifica unas 16 veces

B. Intentos fallidos y qué se aprendió

Intento 1: el pin CE como sospechoso

El primer diagnóstico culpó al pin CE en GPIO2, por ser un pin de arranque. Se movió a GPIO16 y a GPIO27 y el reinicio continuó, por lo que no era la causa principal.

Intento 2: reducir el consumo

Con PA_LOW, el capacitor de 100 µF y las pruebas de `radio.begin()` la placa funcionó varios ciclos, pero reiniciaba tras repetidos «Sin ACK», cuando los reintentos elevan el consumo medio. Esto apuntó a la alimentación.

Intento 3: la central no responde

Al cargar el firmware del teléfono, el nodo imprimía «Aviso: la central no responde», aunque la central veía a otros nodos. El código definía CE en GPIO2 y el cableado real estaba en GPIO22; se corrigió `PIN_CE`.

Intento 4: el micrófono solo daba estática

En la prueba del micrófono con la bocina de la PC se oyó estática, después un chillido breve y luego silencio. Se interpretó como realimentación entre la bocina de la PC y el micrófono, más la reducción de ganancia del control automático del módulo tras un sonido fuerte. Se separaron los elementos, se bajó el volumen y se repitió la prueba por etapas.

Intento 5: llamadas fallidas con la central de otro equipo

Las llamadas con nodos de otros compañeros no se establecían. El nodo usaba la red NODE y el canal 80, mientras que la central de otro equipo usaba el nombre ITA1 y el canal 76. Se hicieron configurables ambos parámetros y el nodo imprime ahora el motivo de cada llamada fallida.

Intento 6: error al cargar el firmware

esptool respondió «Unable to verify flash chip connection». Se bajó la velocidad de carga, se usó el modo de flash DIO, se mantuvo el botón BOOT, se desconectaron los periféricos y se borró la flash con erase-flash; después la carga fue correcta.

Intento 7: error de compilación del firmware de voz

El IDE de Arduino generaba los prototipos de las funciones antes de la definición del tipo Adpcm y la compilación fallaba. Se movió la definición de la estructura antes de la primera función.

C. Cambios pendientes o propuestos

Quedan como mejora un amplificador con más potencia, un filtro para reducir el siseo del DAC de 8 bits, un DAC externo por I2S, el cifrado de los paquetes y la conversación en ambos sentidos a la vez (dúplex).

X. ARQUITECTURA FINAL

A. Central

La central (ID 0) registra los teléfonos, conmuta las llamadas, asigna canales y libera recursos. Su firmware detecta la placa elegida en el IDE y soporta ESP32-S3 SuperMini, ESP32-C3 SuperMini y ESP32 DevKit. En la S3 el NRF24L01 se conecta a CE 9, CSN 10, MOSI 11, SCK 12 y MISO 13, y el LED RGB de la placa (GPIO48) indica el estado: azul tenue lista, verde llamadas en curso y rojo parpadeando error [8].

TABLA XVI
PARÁMETROS DE LA CENTRAL

Parámetro	Valor
Nodos registrados	Hasta 32
Llamadas simultáneas	Hasta 16
Nodo caduca sin registro	15 s
Timbre máximo	20 s
Canales	0 a 83 (2400–2483 MHz)
Canal de control	80; quedan 83 para llamadas
Umbral de ruido	25 % de muestras con señal
Revisión de canales	Uno cada 40 ms; barrido ≈3.4 s
Verificación previa	20 muestras antes de asignar

Al contestar una llamada, la central elige el mejor canal libre, esto es, sin interferencia detectada y lo más lejos posible de los canales ocupados, y manda a ambos teléfonos a él. Si un nodo está en llamada y la central le envía un mensaje, salta a su canal, se lo entrega y regresa al canal de control.

TABLA XVII
COMANDOS DE LA CENTRAL

Comando	Efecto
/tabla	Nodos registrados y su estado
/canales	Mapa de los 84 canales y llamadas activas
/ruido	Canales con interferencia y porcentaje
/todos texto	Mensaje a todos los nodos
/msg ID texto	Mensaje a un nodo
/ayuda	Lista de comandos

B. Teléfonos y canales

Los teléfonos son placas ESP32 con NRF24L01, micrófono, bocina, LED y el botón BOOT, alimentadas por USB desde la PC. El canal de control es el 80 (2480 MHz = 2400 + canal). Todos escuchan en una dirección de 5 bytes derivada de su ID: `[id] N O D E`. Al conectar una llamada, la central indica un canal libre en el paquete CONECTAR y ambos nodos se cambian a él; al terminar, regresan al 80 y avisan con LIBERAR.

C. Máquina de estados

Cada teléfono está en REPOSO, LLAMANDO, TIMBRANDO o EN_LLAMADA.

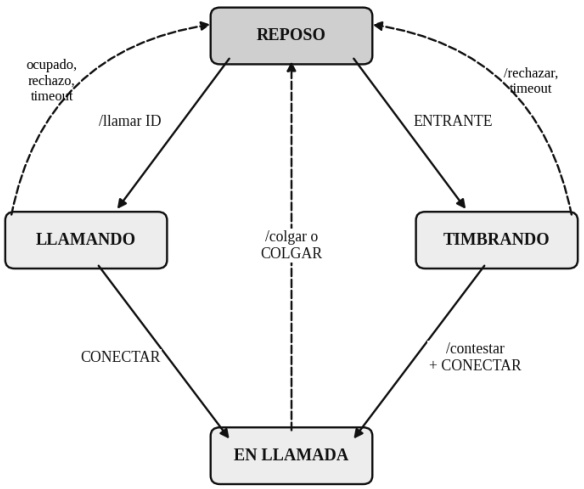


Fig. 12. Máquina de estados del teléfono.

D. Comandos e indicadores del teléfono

TABLA XVIII
COMANDOS DEL MONITOR SERIE DEL TELÉFONO

Comando	Efecto
/llamar N	Pide a la central llamar al nodo N
/contestar	Contesta una llamada
/rechazar	Rechaza una llamada entrante
/colgar	Termina o cancela la llamada
/estado	Estado, canal y pareja
/mic	Nivel del micrófono en vivo
/ganancia N	Sensibilidad del micrófono (1 a 20, por defecto 4)
/calibrar	Mide de nuevo el ruido de fondo (1 s)
texto	En llamada, se envía a la pareja

El LED parpadea rápido al timbrar y lento al llamar; en llamada brilla según la voz de la otra persona. El botón BOOT (GPIO0) contesta si está timbrando y cuelga si está en llamada.

E. Doble central

En la variante redundante hay dos centrales (ID 0 y 100). Cada teléfono guarda la central activa; si el envío a una falla, prueba con la otra y, si responde, la adopta.

XI. CÓMO SE HICIERON LOS NODOS

A. Hardware

Cada nodo es una placa ESP32 DevKit V1 con un módulo NRF24L01 PA+LNA por SPI. En el nodo del autor el pin CE va a GPIO22. Se agregan un micrófono analógico en GPIO34, una bocina de 0.5 W con un módulo PAM8403 alimentado desde el DAC en GPIO25, un LED externo en GPIO12 con 330 Ω y el botón BOOT de la placa. Todo se alimenta por el cable USB de la computadora y el pin 5V/VIN alimenta el PAM8403. En total se usan tres capacitores de 100 µF: uno en VCC/GND del NRF24L01, otro en la alimentación del PAM8403 y otro en serie con la entrada de audio.

B. Firmware

1. En Arduino IDE elegir «ESP32 Dev Module» (o «ESP32C3 Dev Module» con «USB CDC On Boot» activado).
2. Definir un `MI_ID` distinto en cada nodo; el código es el mismo en todos.
3. Revisar que los pines del bloque de la placa coincidan con el cableado, en especial `PIN_CE`.
4. Compilar y subir; si falla la carga, mantener BOOT y bajar la velocidad.
5. Abrir el Monitor Serie a 115200 baudios con «Nueva línea» y verificar «Nodo telefono N listo».
6. Ejecutar /tono y /nivel para probar la bocina y el micrófono del nodo.

C. Registro en la central

Al iniciar, el nodo se registra enviando REGISTRO a la central y lo repite cada 5 segundos en reposo. Si no hay ACK,

imprime «Aviso: la central no responde» una sola vez; al recuperarla, imprime «Central encontrada».

D. Lista de verificación

TABLA XIX
VERIFICACIÓN ANTES DE PROBAR

Punto	OK
Capacitor de 100 µF junto al módulo NRF24 y otro en la alimentación del PAM8403	☐
VCC del NRF24 en 3V3 (nunca 5 V)	☐
CE, CSN, SCK, MISO, MOSI coinciden con el código	☐
MI_ID distinto en cada nodo	☐
Micrófono en GPIO34 (OUT), 3V3 y GND	☐
PAM8403 con 5 V del USB y bocina en L OUT (no a GND)	☐
Capacitor de 100 µF en serie con GPIO25 (positivo hacia GPIO25)	☐
Monitor Serie a 115200	☐
Antena colocada en el módulo PA+LNA	☐

Cada integrante usó el mismo firmware con un MI_ID distinto.

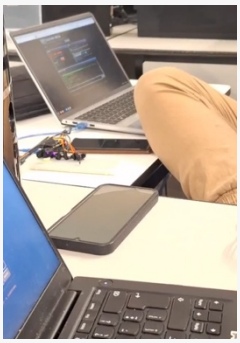


Fig. 13. Fotografía del nodo terminado

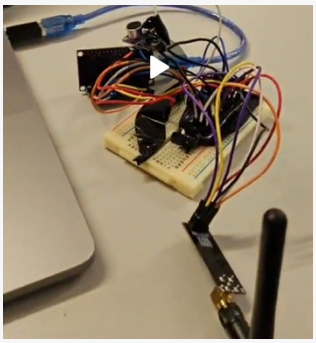


Fig. 14. Fotografía de las conexiones del módulo NRF24L01

XII. PROTOCOLOS Y MENSAJES

A. Configuración de radio

TABLA XX
PARÁMETROS DE RADIO

Parámetro	Valor
Canal de control	80 (2480 MHz)
Canales de llamada	0 a 83, elegidos por la central
Velocidad	1 Mbps
Potencia	RF24_PA_LOW
Tamaño de carga	32 bytes fijo
Auto-ACK	Activado
Reintentos	retardo 3 + (MI_ID mod 10), 15 reintentos
Dirección	[id] N O D E (5 bytes)

El retardo de reintento depende del ID para que dos nodos que colisionen no reintenten al mismo tiempo.

B. Estructura del paquete

Todos los mensajes usan un paquete de 32 bytes: 6 de cabecera y 26 de texto. La compilación lo verifica con un `static_assert`.

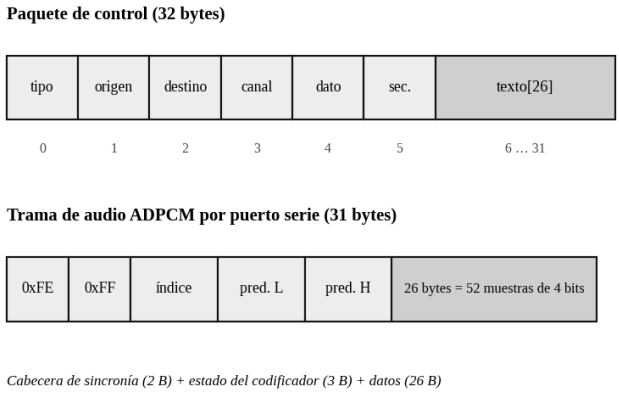


Fig. 15. Formato del paquete de control y de la trama de audio.

TABLA XXI
CAMPOS DEL PAQUETE DE CONTROL

Campo	Bytes	Uso
tipo	1	Tipo de mensaje
origen	1	ID del emisor
destino	1	ID del destinatario
canal	1	Canal asignado (CONECTAR)
dato	1	ID de pareja, código o nivel de voz
secuencia	1	Contador de paquetes
texto	26	Mensaje de texto

C. Tipos de mensaje

TABLA XXII
TIPOS DE MENSAJE

Valor	Tipo	Sentido	Descripción
1	REGISTRO	Nodo → central	Se anuncia como disponible
2	LLAMAR	Nodo → central	Pide llamar al nodo «dato»

Valor	Tipo	Sentido	Descripción
3	CONTESTAR	Nodo → central	Acepta la llamada
4	RECHAZAR	Nodo → central	Rechaza la llamada
5	LIBERAR	Nodo → central	Queda libre
6	ENTRANTE	Central → nodo	Llamada del nodo «dato»
7	CONECTAR	Central → nodos	Canal asignado a la llamada
8	RESPUESTA	Central → nodo	Resultado (ver códigos)
9	TEXTO	Nodo ↔ nodo	Mensaje en llamada
10	KEEPALIVE	Nodo ↔ nodo	Comprueba a la pareja
11	COLGAR	Nodo ↔ nodo	Termina la llamada
12	AVISO	Central → nodo	Mensaje de la central (v2)
13	VOZ	Nodo ↔ nodo	Nivel de voz 0–255 (v3.2); audio ADPCM (v5)

TABLA XXIII
CÓDIGOS DE RESPUESTA

Código	Nombre	Significado
1	R_TIMBRANDO	El destino está sonando
2	R_OCUPADO	El destino está en llamada
3	R_NO_DISPONIBLE	El destino no está registrado
4	R_RECHAZADO	El destino rechazó
5	R_NO_CONTESTA	Se agotó el tiempo
6	R_SIN_CANALES	No hay canales libres
7	R_CANCELADO	El que llamaba canceló

D. Flujo de una llamada

El nodo A pide a la central llamar al nodo B. La central avisa a B (ENTRANTE) y responde a A que está timbrando. Cuando B contesta, la central envía CONECTAR con un canal libre a ambos, que se cambian a él. Ahí intercambian texto, voz y KEEPALIVE hasta que uno cuelga. Ambos avisan con LIBERAR y vuelven a registrarse en el canal de control.

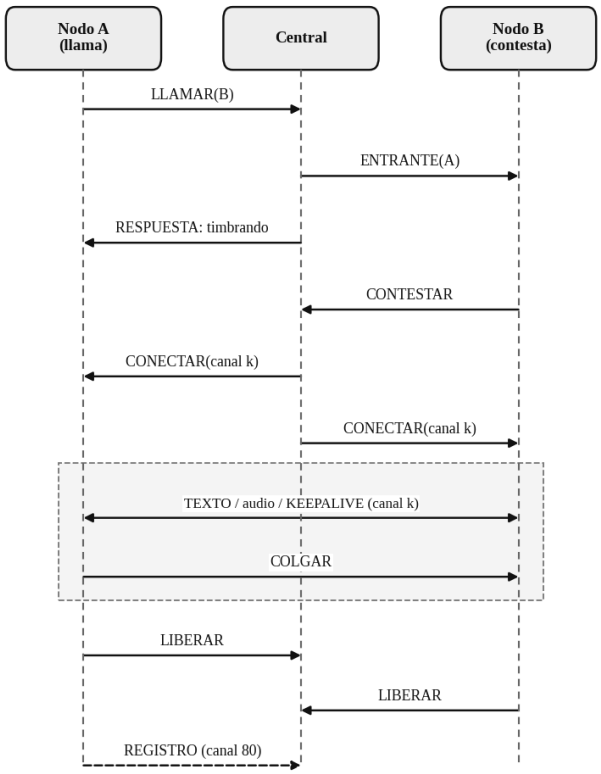


Fig. 16. Secuencia de una llamada entre dos nodos con la central.

E. Temporizadores

TABLA XXIV
TEMPORIZADORES DEL FIRMWARE

Constante	Valor	Uso
REGISTRO_CADA_MS	5000 ms	Reanuncio en reposo
ESPERA_MAX_MS	25000 ms	Máximo llamando o timbrando
KEEPALIVE_CADA_MS	2000 ms	Comprobación de la pareja
FALLOS_PARA_CORTAR	5	≈10 s sin contacto corta la llamada
VENTANA_MS	50 ms	Medición del micrófono (20 por s)

F. Nivel de voz (versión con micrófono)

Durante una llamada cada teléfono toma muestras del micrófono, calcula la amplitud pico a pico cada 50 ms, resta el ruido de fondo calibrado al encender, multiplica por la ganancia y suaviza el resultado (sube rápido y baja despacio). Ese nivel de 0 a 255 se envía a la pareja en el campo `dato` de un paquete VOZ 20 veces por segundo. Si deja de llegar voz durante 300 ms, el nivel remoto vuelve a cero. El LED se corrige con una curva gamma porque el ojo percibe mejor los cambios en niveles bajos.

G. Ejemplos de mensajes en el Monitor Serie

TABLA XXV
SALIDAS DEL MONITOR SERIE

Salida	Significado
=== Nodo telefono 10 listo (ESP32 DevKit) ===	Arranque correcto del nodo 10
Aviso: la central no responde.	Sin ACK de la central
mic: amplitud … nivel … | ####	Nivel del micrófono con /mic
Llamando… / TIMBRANDO / CONECTADO (canal N)	Avance de una llamada saliente
[LLAMADA FALLIDA] motivo	La central no respondió o rechazó la llamada; indica la razón

H. Selección de canal y coexistencia con Wi-Fi

El NRF24L01 a 1 Mbps ocupa aproximadamente 1 MHz, mientras que un canal Wi-Fi ocupa unos 22 MHz. Con los canales Wi-Fi 1, 6 y 11, los canales del NRF24L01 libres de solapamiento quedan en los huecos entre ellos y por encima del 11. El canal 80 (2480 MHz) queda fuera de esos tres canales, lo que justifica su elección como control. La central, además, mide la interferencia y evita los canales ocupados.

TABLA XXVI
SOLAPAMIENTO ENTRE WI-FI Y CANALES DEL NRF24L01

Wi-Fi	Rango (MHz)	Canales NRF
Canal 1	2401–2423	1–23
Canal 6	2426–2448	26–48
Canal 11	2451–2473	51–73
Libres	2424–2425, 2449–2450, 2474–2483	24–25, 49–50, 74–83

Calculado con frecuencias centrales de 2412, 2437 y 2462 MHz y 22 MHz de ancho (canales Wi-Fi 1, 6 y 11).

I. Tiempo en el aire y reintentos

Con Enhanced ShockBurst a 1 Mbps, un paquete de 32 bytes tiene un preámbulo de 1 byte, una dirección de 5 bytes, un campo de control de 9 bits, la carga útil y un CRC de 2 bytes. Suman 329 bits, es decir, unos 329 µs en el aire. Un ACK sin carga útil dura unos 73 µs, y a esto se suman los tiempos de conmutación [1].

TABLA XXVII
TIEMPOS TEÓRICOS A 1 MBPS

Elemento	Bits	Duración
Preámbulo	8	8 µs
Dirección (5 B)	40	40 µs
Campo de control	9	9 µs
Carga útil (32 B)	256	256 µs
CRC (2 B)	16	16 µs
Paquete completo	329	**329 µs**
ACK sin carga	73	73 µs

Cada reintento repite el paquete completo tras el retardo configurado, con 15 reintentos como máximo. Por eso un nodo sin ACK mantiene el transmisor encendido mucho más tiempo, lo que se relaciona con el consumo descrito en la sección XIV.

J. Protocolo del nodo concentrador (router)

La variante router usa un paquete distinto: identificador de destino (1 byte), longitud de datos (1 byte) y 16 bytes de datos. Los datos se cifran con XOR usando una clave de texto fija: cada byte se combina con el byte correspondiente de la clave, repetida cíclicamente, y solo se cifra la longitud exacta. Para depurar, los bytes cifrados se imprimen en hexadecimal. El router escucha en seis direcciones y reenvía a las de destino. Es un cifrado de demostración, no seguro.

K. Formato de ruta propuesto frente al implementado

El documento de tareas propone cadenas de la forma `001 -> 012 -> NC001 -> mensaje`. En la implementación con central, la ruta la resuelve la central mediante el ID de destino del paquete LLAMAR y la asignación de canal. No se implementó el formato explícito de ruta.

XIII. MICRÓFONO, BOCINA Y VOZ

A. Arquitectura final

En la versión final cada teléfono lleva su propio micrófono y su propia bocina, y no se usan los de la computadora. La PC solo suministra la alimentación por USB y el Monitor Serie. El micrófono analógico entra por GPIO34 (ADC1) y se muestrea a 8 kHz; el ESP32 comprime la voz con ADPCM y la envía en paquetes VOZ. El nodo que recibe descomprime y reproduce con el DAC (GPIO25), cuya señal pasa por un capacitor de 100 µF a la entrada L del módulo PAM8403 (HW-104) y de ahí a la bocina de 8 Ω y 0.5 W. El audio se atiende en una tarea del núcleo 0 con un temporizador de 8 kHz y se habla por turnos manteniendo pulsado el botón BOOT (push-to-talk). Esta sección describe ambos circuitos, la prueba de verificación y las versiones anteriores.

Como se indicó en la sección I-E, los nodos de los equipos no son idénticos; esta sección describe la configuración de referencia del nodo del autor.

B. Micrófono

El micrófono es un módulo analógico con amplificador (tipo MAX9814): VDD va a 3V3, GND a GND y OUT a GPIO34, que pertenece al ADC1 y es solo de entrada. La lectura es de 12 bits (0 a 4095). El MAX9814 incorpora control automático de ganancia (AGC): con el pin GAIN unido a VDD la ganancia es de 40 dB, con GAIN a GND de 50 dB y sin conectar de 60 dB [4]; para voz cercana conviene empezar con 40 dB. Por el AGC, tras un sonido fuerte el módulo baja su ganancia unos instantes. Si se usa un módulo KY-037 o KY-038, la conexión es la misma pero con la salida AO.

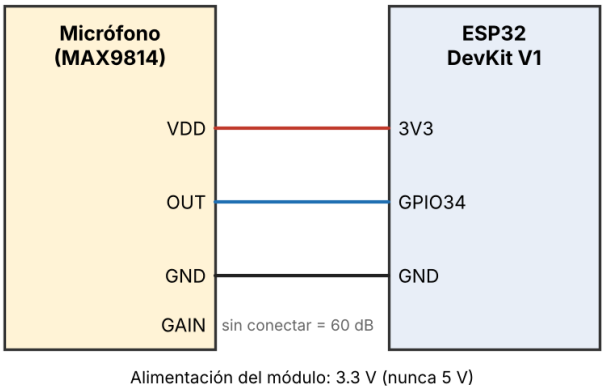


Fig. 17. Conexión del micrófono al ESP32.

C. Bocina con amplificador PAM8403

La bocina de 0.5 W y 8 Ω se maneja con el módulo PAM8403 (HW-104), un amplificador clase D. La señal del DAC (GPIO25) pasa por un capacitor de 100 µF, con el positivo hacia GPIO25, a la entrada L del módulo; el pin G se une a GND; el borne «+» va a VIN (5 V del USB) con otro capacitor de 100 µF a GND y el borne «−» a GND. La bocina se conecta entre L OUT − y L OUT + y nunca a GND. La entrada R y la salida R OUT no se usan.

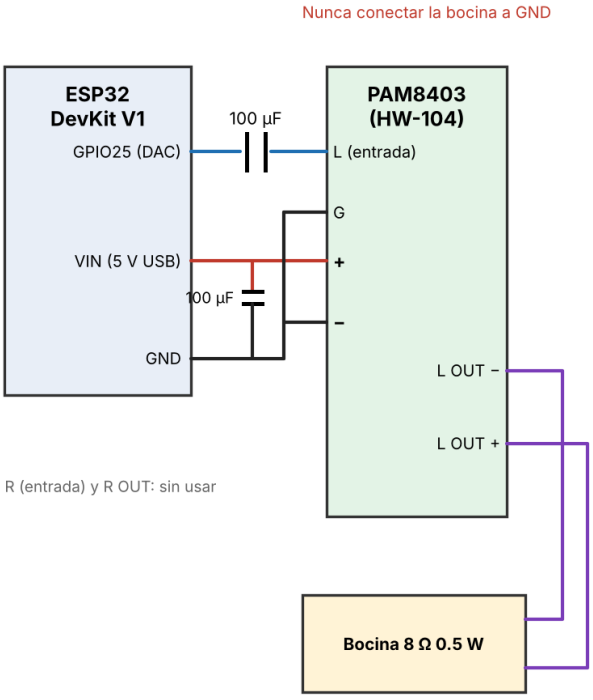


Fig. 18. Conexión del DAC, el PAM8403 y la bocina de 0.5 W.

El PAM8403 amplifica unas 16 veces, por lo que el firmware limita la amplitud del DAC (AMP_MAX 20; para un tono continuo, unas 14 unidades) y no se rebasan los 0.5 W de la bocina. Como el DAC es de 8 bits se percibe algo de siseo, que se reduce con menos amplitud o con un atenuador (divisor o potenciómetro de 10 kΩ) en la entrada. El módulo se alimenta de VIN (5 V del USB) y nunca de 3V3. Con esto el nodo usa tres capacitores de 100 µF: NRF24L01, PAM8403 y acople de la entrada de audio.

D. Nivel de voz en el LED (versión del repositorio)

La versión del repositorio con micrófono (v3.2) mide el volumen de cada persona y lo envía a la otra en paquetes VOZ; el LED de la pareja brilla según ese nivel. Se calibra el ruido de fondo al encender (1 s de silencio) y la sensibilidad se ajusta con /ganancia. La descripción del algoritmo está en la sección XII.F. Esta versión se superó con el firmware final, que transmite voz real.

E. Audio comprimido por la PC (versión alterna)

Una versión posterior del repositorio transmite voz real usando la computadora. La interfaz web, abierta en Chrome o Edge de escritorio, captura el micrófono de la PC a 8 kHz, comprime con ADPCM de 4 bits y envía tramas al nodo por Web Serial [7]; el ESP32 funciona como módem de radio y la PC receptora decodifica y reproduce. Se conserva como alternativa y su formato de paquete es el mismo que usa el firmware final.

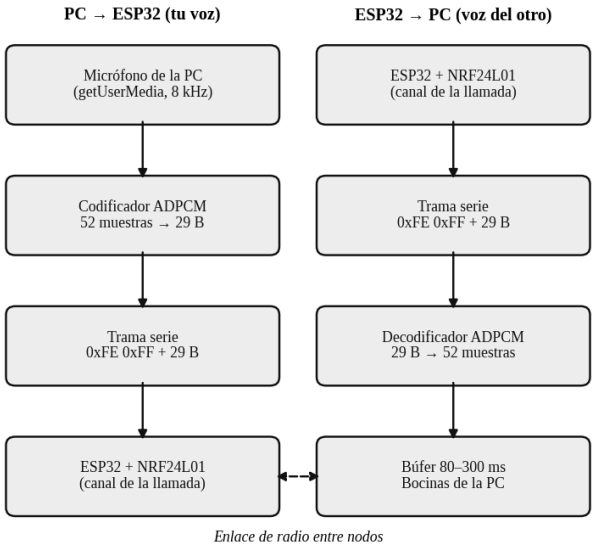


Fig. 19. Flujo de audio de la versión alterna con la PC.

Cada paquete contiene 52 muestras en 26 bytes más 3 bytes con el estado del codificador (índice y predictor), por lo que el decodificador se recupera solo si se pierde uno y el audio puede enviarse sin ACK. El paquete de radio mide 32 bytes: tipo VOZ, origen, secuencia y 29 bytes de audio.

TABLA XXVIII
CÁLCULO DEL FLUJO DE AUDIO (VERSIÓN ALTERNA)

Magnitud	Valor
Muestras por paquete	52 a 8 kHz = 6.5 ms
Paquetes por segundo	≈153.8
Carga de radio por paquete	32 B
Flujo de radio	≈39 kbit/s
Trama serie	31 B (0xFE 0xFF + 29 B)
Flujo serie	≈47.7 kbit/s (≈10 % de 460800)
Ocupación del aire	≈330 µs por paquete, ≈5 %

Valores teóricos calculados a partir de los parámetros del firmware.

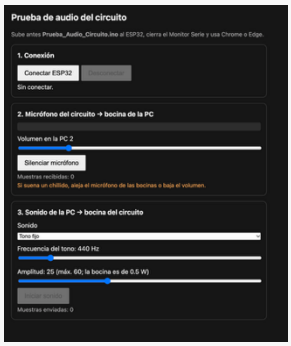


Fig. 20. Captura de la interfaz web con los medidores de audio

Voz real en la versión final. El firmware final (Nodo_Telefono_Voz) captura el micrófono por ADC a 8 kHz, comprime con ADPCM de 4 bits (52 muestras en 26 bytes más 3 bytes de estado), transmite sin ACK y en el nodo que recibe descomprime y reproduce por el DAC, el PAM8403 y la bocina propia. La voz viaja por turnos: se pulsa BOOT para hablar. El códec se verificó en computadora con una relación señal a ruido de unos 28 dB.

F. Prueba de verificación del micrófono y la bocina

Antes de las pruebas de red se verificó cada etapa de audio por separado, con programas de prueba y una página web (Chrome o Edge, Web Serial, 460 800 baudios). El objetivo fue comprobar el micrófono y la bocina del circuito, no los de la computadora.

Prueba 1, micrófono. El programa Prueba_Microfono.ino lee GPIO34 a 8 kHz y envía las muestras a la PC; la página Prueba_Audio.html las reproduce en la bocina de la computadora. Si al hablar cerca del micrófono del circuito se oye la voz en la PC, el micrófono funciona.

Prueba 2, bocina del circuito. El programa Prueba_Bocina_Circuito.ino genera tonos de 440, 880 y 660 Hz por el DAC (GPIO25), el PAM8403 y la bocina del circuito. La amplitud se mantiene baja porque el PAM8403 amplifica unas 16 veces.

Prueba 3, dúplex. El programa Prueba_Audio_Circuito.ino y su página permiten probar ambas partes a la vez: el micrófono del circuito suena en la bocina de la PC y, al mismo tiempo, la PC envía un tono, un barrido o una melodía que suena en la bocina del circuito.

Incidente real. En la primera prueba del micrófono solo se oyó estática; por un momento se escuchó un chillido y después solo silencio. Se interpretó como realimentación entre la bocina de la PC y el micrófono y como la bajada de ganancia del AGC del módulo. Se alejaron los elementos, se bajó el volumen (o se usaron audífonos en la PC) y se repitió. También se aclaró que la bocina se prueba con la bocina del circuito y no con la de la computadora.

Verificación final. Con el firmware final cargado se comprobaron los comandos /tono (tono en la bocina del nodo) y /nivel (nivel del micrófono) y, después, una llamada completa con el botón BOOT como push-to-talk. La tabla XXIX resume las pruebas de audio.

TABLA XXIX
PRUEBAS DE BOCINA Y MICRÓFONO

N.º	Prueba	Resultado	Obs.
1	Micrófono en silencio (nivel)	☐ OK	Esperado: ruido bajo y estable
2	Micrófono hablando a 10 cm	☐ OK	Esperado: el nivel sube claramente
3	Micrófono → bocina de la PC (primer intento)	Estática, chillido breve y silencio	Realimentación y AGC; se separaron y se bajó el volumen
4	Micrófono → bocina de la PC (repetición)	☐ OK	Esperado: se entiende la voz
5	Tono en la bocina del circuito (PC → DAC → PAM8403)	☐ OK	Esperado: tono de 440 Hz claro, amplitud baja
6	Barrido y melodía en la bocina del circuito	☐ OK ☐ Falla	Esperado: sin cortes ni distorsión fuerte
7	Volumen y distorsión (amplitud AMP_MAX)	☐ OK ☐ Falla	Bajar la amplitud si satura
8	Prueba dúplex (micrófono y bocina a la vez)	☐ OK ☐ Falla	Esperado: ambos funcionan sin chillido
9	Eco con la bocina cerca del micrófono	☐ OK ☐ Falla	Separar bocina y micrófono
10	Comandos /tono y /nivel del firmware final	☐ OK ☐ Falla	Esperado: tono y nivel correctos
11	Llamada con push-to-talk (BOOT)	☐ OK ☐ Falla	Esperado: se oye la voz del otro nodo
12	Retraso estimado de la voz		Medir con cronómetro o con la interfaz

Conclusión: probar por etapas (micrófono, bocina y luego ambos) permitió aislar las fallas. El primer intento mostró el acople entre la bocina de la PC y el micrófono, que se corrigió separando los elementos y bajando el volumen.

G. Circuito anterior con transistor (descartado)

En las primeras pruebas se contempló un amplificador con transistor 2N2222 (polarización con dos resistencias de 4.7 kΩ, 22 Ω de emisor y capacitores de 10 µF y 470 µF). Entregaba poco volumen y consumía unos 80 mA en reposo, por lo que se sustituyó por el módulo PAM8403 (clase D, 3 W), alimentado a 5 V [5] y con la amplitud limitada por firmware para no exceder los 0.5 W de la bocina.

XIV. DIAGRAMAS DE CONEXIONES

A. Nodo: ESP32 y NRF24L01

La figura muestra la conexión del módulo de radio. Se alimenta con 3.3 V y el capacitor de 100 µF se coloca lo más cerca posible de sus pines de alimentación.

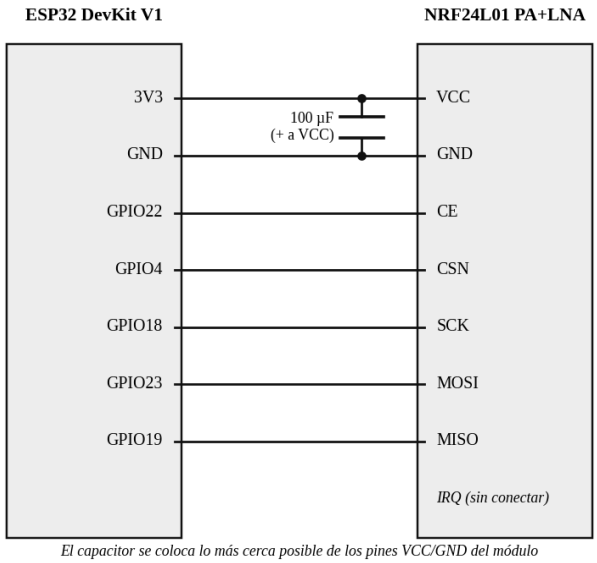


Fig. 21. Conexión entre el ESP32 DevKit V1 y el NRF24L01 PA+LNA.

TABLA XXX
ASIGNACIÓN DE PINES DEL TELÉFONO

Señal	ESP32 DevKit	C3 SuperMini
CE	GPIO22	GPIO3
CSN	GPIO4	GPIO7
SCK	GPIO18	GPIO4
MISO	GPIO19	GPIO5
MOSI	GPIO23	GPIO6
LED	GPIO12 + 330 Ω	GPIO8 (integrado)
Botón	GPIO0 (BOOT)	GPIO9 (BOOT)
Micrófono	GPIO34 (ADC1)	GPIO1 (ADC1)
Bocina (DAC)	GPIO25 → 100 µF → PAM8403 (L)	Sin DAC
PAM8403 (+ / −)	VIN (5 V) / GND	5 V / GND
VCC / GND	3V3 / GND	3V3 / GND

El ESP32-C3 no tiene DAC, por lo que la bocina con DAC requiere la placa ESP32 DevKit.

B. Micrófono y bocina

Las conexiones del micrófono y del PAM8403 con la bocina están en las figuras de la sección XIII. Ningún pin de estos circuitos coincide con los del NRF24L01 (22, 4, 18, 19 y 23).

C. Alimentación por USB y capacitor

Todo el nodo se alimenta por el USB de la computadora, que entrega unos 500 mA. El NRF24L01 PA+LNA tiene picos de corriente al transmitir; el capacitor de 100 µF amortigua los picos. El PAM8403 se alimenta del pin VIN (5 V del USB) con su propio capacitor de 100 µF, y la entrada de audio lleva otro de 100 µF en serie; en total son tres. En los electrolíticos, la pata larga es la positiva. Como medida adicional se contempló un regulador externo de 3.3 V con tierra común.

Estimación de la caída de tensión. Si el regulador de la placa no entrega el pico, el capacitor suministra la carga $Q = I \times t$ y la tensión cae $\Delta V = Q / C$. Con un pico adicional de 100 mA durante un paquete (329 µs) la carga es de unos 33 µC; con una ráfaga de reintentos de unos 5 ms es de 0.5 mC. La tabla muestra el peor caso, sin contar la aportación del regulador.

TABLA XXXI
CAÍDA ESTIMADA SEGÚN EL CAPACITOR

Capacitor	ΔV (1 paquete)	ΔV (ráfaga 5 ms)
100 µF	0.33 V	5.0 V
220 µF	0.15 V	2.3 V
470 µF	0.07 V	1.1 V
1000 µF	0.03 V	0.5 V

Estimación de orden de magnitud con 100 mA de pico adicional; el pico real se consulta en la hoja de datos del módulo [1]. Explica por qué 100 µF alarga el funcionamiento pero no evita los reinicios con muchos reintentos.

D. Circuito del generador 555

El generador usa un NE555 en modo astable, un potenciómetro de 10 kΩ, resistencias de 220 Ω y 1 kΩ, un capacitor de 100 µF y un LED, con una batería de 9 V. Si su salida se llevara al ESP32 debe pasar por un divisor de voltaje (por ejemplo, 10 kΩ y 5.6 kΩ) con tierra común, porque los GPIO no toleran más de 3.3 V.

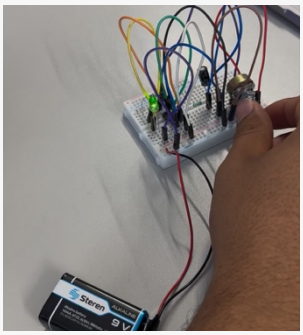


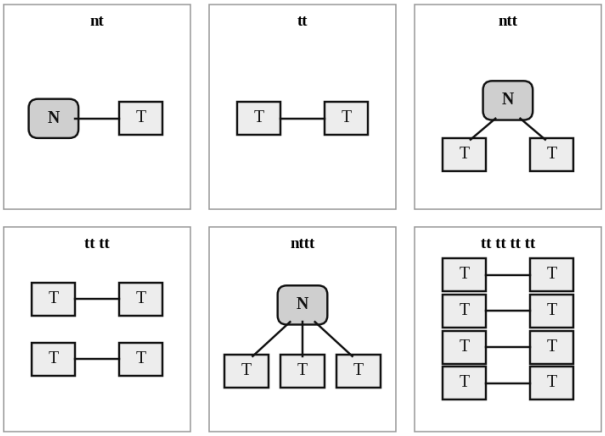
Fig. 22. Fotografía del circuito del generador 555

XV. PRUEBAS

A. Metodología

Las pruebas siguen una progresión de topologías. Se denominan con letras: n es el nodo central y t es un teléfono; así, nt es la central con un teléfono, tt dos teléfonos en llamada, ntt la central con dos teléfonos, tt tt dos llamadas simultáneas, nttt la central con tres teléfonos y tt tt tt tt cuatro llamadas simultáneas. Cada topología se prueba dos veces: una enviando mensajes de texto y otra con llamadas (audio). Se registra el resultado esperado y el obtenido.

Como los nodos de los integrantes no son idénticos (bocina, micrófono, PAM8403 o audio de la laptop), los resultados pueden variar ligeramente entre equipos.



N: nodo central T: teléfono (nodo)
— : llamada o mensaje entre nodos

Fig. 23. Topologías de prueba: nt, tt, ntt, tt tt, nttt y tt tt tt tt.

Interpretación de las abreviaturas: n es el nodo central y t un teléfono, igual que en el documento «Central telefónica» del equipo.

B. Instrumentos y condiciones

TABLA XXXII
CONDICIONES DE PRUEBA

Dato	Valor
Lugar	Laboratorio de redes
Distancia entre nodos	1 m
Potencia (PA)	RF24_PA_LOW (ajustable a HIGH)
Canal de control	80
Alimentación	USB de la PC (5 V)
Interferencia (Wi-Fi cercana)	No

C. Pruebas con mensajes

TABLA XXXIII
FICHA M1: CONEXIÓN NT

Código	M1
Prueba	Conexión nt
Topología	Central y un teléfono
Tipo	Mensajes de texto
Fecha / responsable	2 Octubre 2026
Objetivo	Verificar que un teléfono se registra en la central y recibe su confirmación.
Procedimiento	Encender la central (ID 0). Conectar un teléfono con un MI_ID único y abrir el Monitor Serie a 115200. Esperar «Nodo telefono N listo» y ejecutar /estado en la central.
Resultado esperado	El teléfono aparece en la tabla de la central y no muestra «la central no responde».
Resultado obtenido	Hay comunicación entre el nodo y la central; la conexión se generó adecuadamente.
Estado	Aprobada



Fig. 24. Evidencia de la prueba M1 (conexión nt, mensajes)

TABLA XXXIV
FICHA M2: CONEXIÓN TT

Código	M2
Prueba	Conexión tt
Topología	Dos teléfonos
Tipo	Mensajes de texto
Fecha / responsable	2 Octubre 2026
Objetivo	Verificar que dos teléfonos establecen comunicación entre sí.
Procedimiento	Con la central encendida, registrar dos teléfonos. Desde el A ejecutar /llamar B, contestar en B y comprobar CONECTADO en ambos.
Resultado esperado	Ambos teléfonos quedan conectados en el mismo canal.
Resultado obtenido	El esperado
Estado	Aprobada

TABLA XXXV
FICHA M3: ENVÍO DE MENSAJE

Código	M3
Prueba	Envío de mensaje
Topología	Dos teléfonos con la central
Tipo	Mensajes de texto
Fecha / responsable	2 Octubre 2026
Objetivo	Verificar que un mensaje de texto llega solo al destinatario.
Procedimiento	Con la llamada A–B establecida, escribir un texto de hasta 25 caracteres en el Monitor Serie de A y revisar el de B y el de un tercer nodo registrado.
Resultado esperado	El texto aparece en B y no en el tercer nodo.
Resultado obtenido	El esperado
Estado	Aprobada



Fig. 25. Evidencia de la prueba M2 (conexión tt, mensajes)



Fig. 26. Evidencia de la prueba M3 (envío de mensaje, mensajes)

TABLA XXXVI
FICHA M4: CONEXIÓN NTT

Código	M4
Prueba	Conexión ntt
Topología	Central y dos teléfonos
Tipo	Mensajes de texto
Fecha / responsable	
Objetivo	Comprobar la central con dos teléfonos registrados y una llamada entre ellos.
Procedimiento	Encender la central y dos teléfonos. Verificar los dos registros con /estado y llamar de uno al otro.
Resultado esperado	Dos teléfonos registrados y llamada establecida.
Resultado obtenido	El esperado
Estado	Aprobada

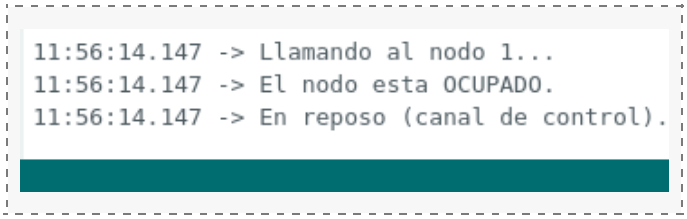


Fig. 27. Evidencia de la prueba M4 (conexión ntt, mensajes)

TABLA XXXVII
FICHA M5: CONEXIÓN TT TT

Código	M5
Prueba	Conexión tt tt
Topología	Dos pares de teléfonos
Tipo	Mensajes de texto
Fecha / responsable	2 Octubre 2026
Objetivo	Comprobar dos conversaciones simultáneas sin interferencia.
Procedimiento	Registrar cuatro teléfonos. Llamar A–B y C–D al mismo tiempo y enviar un texto en cada pareja.
Resultado esperado	La central asigna un canal distinto a cada pareja y los mensajes no se cruzan.
Estado	Aprobada

TABLA XXXVIII
FICHA M6: CONEXIÓN NTTT

Código	M6
Prueba	Conexión nttt
Topología	Central y tres teléfonos
Tipo	Mensajes de texto
Fecha / responsable	2 Octubre 2026
Objetivo	Comprobar tres registros con una llamada de dos teléfonos y el tercero libre.
Procedimiento	Registrar tres teléfonos, llamar A–B y, con la llamada en curso, llamar de C a A.
Resultado esperado	C recibe «ocupado» y A–B no se interrumpe.
Estado	Aprobada

TABLA XXXIX
FICHA M7: CONEXIÓN TT TT TT TT

Código	M7
Prueba	Conexión tt tt tt tt
Topología	Cuatro pares de teléfonos
Tipo	Mensajes de texto
Fecha / responsable	
Objetivo	Probar el límite de conversaciones simultáneas con ocho teléfonos.
Procedimiento	Registrar ocho teléfonos y establecer cuatro llamadas simultáneas; observar el comportamiento de la central.
Resultado esperado	Cuatro conversaciones simultáneas o aviso de canales agotados (R_SIN_CANALES).
Resultado obtenido	El esperado
Estado	Parcial
Observaciones	El nodo central solo sabe que un canal está liberado cuando los nodos regresan al canal de control; no conoce el estado de los canales en uso.

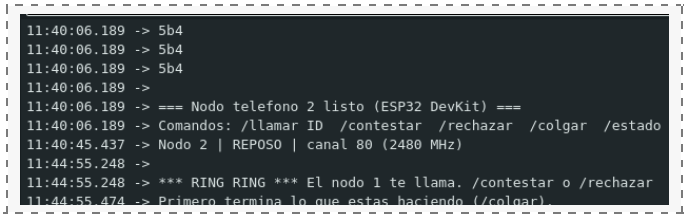


Fig. 28. Evidencia de la prueba M7 (conexión tt tt tt tt, mensajes)

D. Pruebas complementarias

Además de las pruebas por topología se realizaron pruebas que caracterizan el enlace; sus resultados se registran en las tablas siguientes.

1) Alcance según la potencia

TABLA XL
PÉRDIDA DE PAQUETES SEGÚN POTENCIA Y DISTANCIA

PA	1 m	5 m	10 m	20 m
MIN	No	Sí	Sí	Sí
LOW	No	No	Sí	Sí
HIGH	No	No	No	Sí
MAX	No	No	No	No

Link video

Fig. 29. Video de la prueba de alcance

2) Tiempo de establecimiento de llamada

TABLA XLI
TIEMPOS DE ESTABLECIMIENTO

Intento	/llamar (s)	CONECTADO (s)
1	10	Sí
2	8	Sí
3	4	Sí

3) Duración y estabilidad

TABLA XLII
LLAMADA PROLONGADA

Duración	Cortes	Paquetes perdidos	Observaciones
5 min	No	No	Estable
10 min	No	No	Estable
30 min	Sí	Sí	Interferencia

E. Matriz de casos de prueba

La matriz resume cada topología con los dos tipos de prueba (mensajes y llamadas), el resultado esperado y el estado obtenido.

TABLA XLIII
MATRIZ DE CASOS DE PRUEBA

ID	Topología	Elementos	Tipo	Resultado esperado	Estado
M1	nt	Central + 1 teléfono	Mensajes	Se registra en la central	OK
M2	tt	2 teléfonos	Mensajes	Los dos nodos establecen comunicación	OK
M3	Envío msj	2 teléfonos + central	Mensajes	El mensaje llega solo al destinatario	OK
M4	ntt	Central + 2 teléfonos	Mensajes	Dos teléfonos registrados, llamada entre ellos	OK
M5	tt tt	4 teléfonos	Mensajes	Dos conversaciones simultáneas sin interferirse	OK
M6	nttt	Central + 3 teléfonos	Mensajes	Tres registros; llamada de dos y el tercero libre	OK
M7	tt tt tt tt	8 teléfonos	Mensajes	Cuatro conversaciones simultáneas o aviso de canales agotados	Parcial
L1	nt	Central + 1 teléfono	Llamadas	Se registra en la central	OK
L2	tt	2 teléfonos	Llamadas	Los dos nodos establecen comunicación	OK
L3	Envío de voz	2 teléfonos + central	Llamadas	La voz llega solo al destinatario	OK
L4	ntt	Central + 2 teléfonos	Llamadas	Dos teléfonos registrados, llamada entre ellos	OK
L5	tt tt	4 teléfonos	Llamadas	Dos conversaciones simultáneas sin interferirse	OK
L6	nttt	Central + 3 teléfonos	Llamadas	Tres registros; llamada de dos y el tercero libre	OK
L7	tt tt tt tt	8 teléfonos	Llamadas	Cuatro conversaciones simultáneas o aviso de canales agotados	OK

XVI. DEMOSTRACIÓN

A. Guion

1. Encender la central y verificar que imprime su mensaje de inicio.
2. Conectar los teléfonos y comprobar que cada uno muestra «Nodo telefono N listo» sin avisos de central perdida.
3. Abrir la interfaz web en Chrome o Edge, elegir el firmware y la velocidad, y pulsar «Conectar ESP32».
4. Desde el nodo A ejecutar `/llamar B`; el nodo B muestra «El nodo A te llama».
5. Contestar en B (comando o botón BOOT) y comprobar «CONECTADO» y el canal asignado en ambos.
6. Enviar un mensaje de texto y hablar; verificar los medidores de audio y las estadísticas.
7. Colgar y verificar que ambos regresan a reposo.
8. Repetir con más teléfonos para mostrar llamadas simultáneas.

B. Escenarios

TABLA XLIV
ESCENARIOS DE LA DEMOSTRACIÓN

N.º	Escenario	Resultado
1	Registro de teléfonos	OK
2	Llamada entre dos teléfonos	OK
3	Mensaje de texto en llamada	OK
4	Llamada con voz	OK
5	Rechazo y ocupado	OK
6	Llamadas simultáneas	OK

C. Resultados

Los resultados de cada escenario del guion se resumen en la tabla XLIV; todos los escenarios se completaron correctamente.

XVII. PROBLEMAS Y SOLUCIONES

TABLA XLV
PROBLEMAS ENCONTRADOS

N.º	Problema o síntoma	Causa	Solución
1	Error «nRF24L01.h: No such file or directory»	Falta la librería RF24	Instalar RF24 de TMRh20 desde el gestor o ZIP
2	«OpenOCD: GDB Server Quit Unexpectedly»	Panel Debug sin adaptador JTAG	Ignorarlo; usar Upload y Monitor Serie
3	`invalid header: 0xffffffff` en bucle	Caída de tensión por picos del NRF24 PA+LNA	Capacitor, PA_LOW, setRetries, regulador externo
4	`rst:0x7 TG0WDT_SYS_RESET` al transmitir	Misma causa de alimentación	Descartado el pin CE: se probó GPIO16 y GPIO27
5	Botón enviaba el estado contrario	Doble inversión en el código	Quitar la segunda negación
6	«Sin ACK» constante	Receptor ausente o con otra dirección	Igualar dirección, canal y ID; encender el receptor
7	esptool «Unable to verify flash chip connection»	Fallo de comunicación o de velocidad	Mantener BOOT, bajar Upload Speed, cambiar cable o puerto
8	«La central no responde»	CE definido en GPIO2 y cableado en GPIO22	Definir PIN_CE en el pin real
9	Micrófono con lecturas inestables o muy bajas	Ruido de fondo y ganancia mal ajustada	/calibrar en silencio y ajustar /ganancia
10	Bocina muy débil con el DAC	El DAC entrega muy poca corriente	Amplificador PAM8403 con 5 V del USB
11	5 V o 9 V en un GPIO	Los GPIO soportan solo 3.3 V	Divisor de voltaje y revisión previa
12	Un nodo no ve a la central en el canal	Pin CE, canal o ID distintos	Revisar pines, canal 80 e ID únicos
13	Solo estática, un chillido y silencio al probar el micrófono	Realimentación con la bocina de la PC y AGC del módulo	Separar los elementos, bajar el volumen y repetir por etapas
14	Llamadas fallidas con la central de otro equipo	Nombre de red y canal distintos (NODE/80 frente a ITA1/76)	NOMBRE_RED y CANAL_CONTROL configurables y mensajes con el motivo
15	Error de compilación con el tipo Adpcm	El IDE genera prototipos antes del tipo definido por el usuario	Definir la estructura antes de la primera función
16	Siseo en la bocina	DAC de 8 bits y ganancia ×16 del PAM8403	Limitar AMP_MAX y atenuar la entrada

Lección general. Si compila y `begin()` funciona, el módulo está detectado por SPI, pero eso no prueba que transmita bien: las fallas durante la transmisión suelen ser de alimentación, y los errores de configuración se evidencian al comparar pin por pin el código con el cableado.

Otros problemas del audio: el siseo del DAC de 8 bits amplificado por el PAM8403 se reduce limitando la amplitud; y el micrófono con AGC baja su ganancia tras un sonido fuerte, por lo que puede parecer que «se apaga» unos instantes.

XVIII. RETOS, ALCANCES Y LIMITACIONES

A. Retos

- **Alimentación:** lograr estabilidad con USB y un módulo PA+LNA.
- **Coherencia de configuración:** pines, direcciones, canales e IDs iguales en todas las placas.
- **Integración:** que los nodos de cada integrante trabajen con la central de otro compañero.
- **Tiempo real:** audio fluido con un enlace de 32 bytes por paquete.
- **Trabajo en equipo:** coordinar versiones de firmware, hardware y documentación.

Retos propios: la voz en tiempo real con un ADC de 12 bits y un DAC de 8 bits se resolvió con compresión ADPCM y una tarea de audio dedicada en el núcleo 0 del ESP32; la

compatibilidad con la central de otros compañeros, con parámetros de red configurables.

Otro reto fue la diversidad de componentes: cada equipo construyó su transmisor con lo que tuvo disponible y algunos usaron el audio de la laptop, por lo que el firmware y la configuración debían funcionar con variantes de hardware.

B. Alcances

- Registro de teléfonos en una central y llamadas entre nodos.
- Mensajes de texto de hasta 25 caracteres durante la llamada.
- Audio de voz comprimido (ADPCM) entre dos nodos, con micrófono y bocina propios; la interfaz web queda como alternativa.
- Asignación de canales para varias llamadas simultáneas.
- Detección de pérdida de pareja mediante KEEPALIVE.
- Soporte de dos placas: ESP32 DevKit y ESP32-C3 SuperMini.

C. Limitaciones

TABLA XLVI
LIMITACIONES DEL SISTEMA

Limitación	Consecuencia
Paquete de 32 bytes	Mensajes cortos; audio con compresión
Velocidad de 1 Mbps	Calidad de voz de 8 kHz
Canales finitos	Hasta 16 llamadas y 32 nodos por central; R_SIN_CANALES
Central única	Punto único de falla; la doble central lo mitiga
Bocina con PAM8403 y DAC de 8 bits	Volumen limitado y algo de siseo; la amplitud se limita por firmware
DAC solo en ESP32 DevKit	El C3 SuperMini no tiene DAC
Alimentación por USB de la PC	El nodo depende de un cable y de una computadora
Sin cifrado en las llamadas	Solo la variante router usó XOR
Coexistencia con Wi-Fi	Interferencia en 2.4 GHz
Potencia PA_LOW	Alcance reducido
Conversación por turnos	Se habla manteniendo pulsado el botón BOOT (push-to-talk)
Nodos con componentes distintos	Bocina, micrófono, PAM8403 o audio de la laptop varían entre equipos, y con ello el volumen y la calidad

Los valores reales de alcance y de número máximo de nodos dependen de la prueba de alcance (sección XV-D) y de la prueba M7 (sección XV-C); las cifras teóricas son las de las tablas anteriores.

XIX. MARCO DE SEGURIDAD Y REGULACIÓN

A. Regulación del espectro

El proyecto opera en la banda ISM de 2.4 GHz, de uso libre con ciertas condiciones de potencia. El NRF24L01 permite 126 canales, pero en la mayoría de los países la banda ISM termina en 2.4835 GHz, por lo que los canales por encima del 83 deben evitarse. La central usa los canales 0 a 83 y el de control (80, 2480 MHz) queda dentro de la banda pero cerca de su límite superior. Los módulos PA+LNA pueden emitir potencias mayores que las de un módulo básico, por lo que deben respetar los límites de potencia radiada vigentes. Debe consultarse la normativa vigente de la autoridad de telecomunicaciones de México sobre equipos de 2.4 GHz, que fija los límites de potencia aplicables.

B. Seguridad eléctrica

- El NRF24L01 y los GPIO del ESP32 trabajan a 3.3 V; no conectar 5 V ni los 9 V del generador 555 sin un divisor de voltaje. Los 5 V del USB se usan solo para el amplificador PAM8403.
- Respetar la polaridad de los capacitores electrolíticos para evitar sobrecalentamiento.
- Compartir tierra entre fuentes distintas.
- Si se usan baterías de litio, protegerlas contra cortocircuito y sobrecarga.

C. Seguridad de la información

La variante router cifra los datos con XOR y una clave fija, lo cual es solo una demostración: es fácil de romper y la clave está en el código. El protocolo de teléfono con central envía los paquetes sin cifrar y con direcciones predecibles (`[id]NODE`), por lo que un tercero con otro NRF24L01 podría escuchar o suplantar un nodo. Para fines académicos esto es aceptable, pero un sistema real necesitaría cifrado robusto, autenticación de nodos y protección contra reenvío.

TABLA XLVII
MATRIZ DE RIESGOS

Riesgo	Prob.	Impacto	Mitigación
Caída de tensión	Alta	Medio	Capacitor, regulador externo
Daño por 5 V o 9 V en GPIO	Media	Alto	Divisor de voltaje, revisión previa
Escucha de llamadas	Media	Medio	Uso en laboratorio; cifrado futuro
Interferencia de Wi-Fi	Alta	Medio	Elegir canal libre
Exceso de potencia	Baja	Medio	Mantener PA_LOW o PA_HIGH según norma
Volumen excesivo en la bocina de 0.5 W	Media	Bajo	Limitar la amplitud en el firmware (AMP_MAX)

D. Uso responsable

El sistema se usa solo en el laboratorio y con los participantes del equipo, sin captar comunicaciones ajenas. Las pruebas de audio deben realizarse con consentimiento de las personas implicadas.

Las pruebas se hicieron en interiores y con potencia baja o media, y el volumen de la bocina se limitó por firmware para no dañarla ni molestar al oído.

XX. CONTRIBUCIÓN POR EQUIPO

Este apartado distingue el trabajo de cada integrante, necesario para una evaluación justa del esfuerzo grupal.

TABLA XLVIII
APORTES VISIBLES EN EL REPOSITORIO

Usuario	Aportes	Nombre real
Maztherzanch	Repositorio, documento de tareas	Ricardo Rdz
engsecure	Router con XOR, nodo con ADPCM e interfaz web	
f1sher	Emisor/receptor, doble central	
VTorres	Central para ESP32-S3	Victor Torres
emilianosDev	Programa de los teléfonos	Emiliano
qesin	Central v2 y v3, teléfono con micrófono	Rita
Dyisus05	Ajuste del nodo y paquete de la central v2	Jesus

Los aportes que no dejan commits (hardware, pruebas, bocina, documentación) se resumen en la tabla siguiente.

TABLA XLIX
CONTRIBUCIÓN POR INTEGRANTE

Integrante	Contribuciones principales
Chuyemiliano	Creación de código
Equipoqeso	Creación de código
Radiantescachorros	Creación de código
Silviorodrigues	Creación de código
equiporocket	Creación de código y documentación

A. Declaración de contribución

Los integrantes declaran que el trabajo se repartió según las tablas anteriores y que cada uno participó en el diseño, el armado, la programación, las pruebas y la redacción.

XXI. TRABAJO FUTURO, LECCIONES APRENDIDAS Y CONCLUSIONES

A. Trabajo futuro

- Cifrado robusto y autenticación de los paquetes (en lugar de XOR).
- Mejor calidad de audio: DAC o amplificador por I2S (por ejemplo MAX98357A), filtro contra el siseo y conversación en ambos sentidos a la vez.
- Regulador de 3.3 V dedicado y placa de circuito impreso.
- ACK selectivo o corrección de errores en el audio.
- Enrutamiento por ruta de nodos y varias centrales.
- Selección dinámica de canales libres según la interferencia.

B. Lecciones aprendidas

1. La alimentación es el primer sospechoso cuando un módulo de radio funciona y luego reinicia la placa.
2. `radio.begin()` solo valida la comunicación SPI; no demuestra que la transmisión sea correcta.
3. Documentar el cableado real evita errores al cambiar de código o de integrante.
4. Un protocolo con tipos de mensaje y estados facilita añadir funciones como el audio.
5. Cada paquete de voz con su estado hace al sistema tolerante a pérdidas.

Probar por etapas (micrófono, bocina y luego ambos) aísla las fallas; comprobar que todos los nodos usen el mismo nombre de red y canal antes de culpar al código; limitar la amplitud del audio antes de conectar un amplificador; y definir los tipos propios antes de la primera función porque el IDE de Arduino genera prototipos.

C. Conclusiones generales

El proyecto permitió pasar de un generador de señales y un enlace punto a punto a una red de teléfonos con nodo central, y comprender en la práctica los fundamentos de los transmisores de radio: portadora, canal, dirección, potencia, retransmisión y coexistencia. Los problemas de alimentación y de configuración mostraron que el diseño de un sistema de radio depende tanto del software como del hardware. En la versión final, cada nodo captura su voz, la comprime y la reproduce con su propia bocina, alimentado por el USB de la PC; el sistema cumple el objetivo general con las limitaciones de la sección XVIII.

D. Conclusiones por equipo

Equipo 1: Aprendimos a armar y alimentar correctamente un nodo de radio: el capacitor, el cableado por pin y la importancia de revisar el hardware antes que el código.

Equipo 2: Comprendimos el protocolo de 32 bytes y los estados de llamada; entender los mensajes facilitó agregar funciones como el texto y la voz.

Equipo 3: Entendimos cómo la central asigna canales y por qué todos los nodos deben compartir la red y el canal de control.

Equipo 4: Aprendimos cómo se captura, comprime y reproduce el audio con un ADC, un DAC y un amplificador, y a probar cada etapa por separado.

Equipo 5: Aprendimos a documentar las pruebas y a ordenar las evidencias, y a trabajar con versiones de código de varias personas.

REFERENCIAS

[1] Nordic Semiconductor, «nRF24L01+ Single Chip 2.4 GHz Transceiver Product Specification v1.0», 2008.
[2] Espressif Systems, «ESP32 Series Datasheet».
[3] TMRh20 et al., «RF24: Optimized high speed nRF24L01+ driver class», GitHub, github.com/nRF24/RF24.
[4] Maxim Integrated, «MAX9814: Microphone Amplifier with AGC and Low-Noise Microphone Bias», hoja de datos.

[5] Diodes Incorporated, «PAM8403: Filterless 3 W Class-D Stereo Audio Amplifier», hoja de datos.

[6] Interactive Multimedia Association, «Recommended Practices for Enhancing Digital Audio Compatibility in Multimedia Systems, rev. 3.00», 1992 (ADPCM IMA).

[7] W3C WICG, «Web Serial API», wicg.github.io/serial.

[8] Equipo, «STLC_SYBER_ITA_26_NRFL-ESP_Communication», repositorio de GitHub, github.com/maztherzanch/STLC_SYBER_ITA_26_NRFL-ESP_Communication.

[9] Espressif Systems, «arduino-esp32», GitHub, github.com/espressif/arduino-esp32.

[10] IEEE, «IEEE Std 802.11-2020, Wireless LAN Medium Access Control and Physical Layer Specifications».

[11] Bluetooth SIG, «Bluetooth Core Specification», versión 5.4.

[12] Espressif Systems, «ESP-NOW User Guide».

[13] Nordic Semiconductor, «nRF24L01+ Enhanced ShockBurst», nota de la hoja de datos [1].

APÉNDICE A. CÓDIGO FUENTE

A.1. Versión v1: enlace TX/RX

Código v1 en su versión corregida (dirección «00009», botón en GPIO25 con INPUT_PULLDOWN, PA_LOW y setRetries(5,5)). Se reconstruyó a partir de los cambios documentados en la tabla XV.

```
// TRANSMISOR v1 (corregido)
#include <SPI.h>
#include <RF24.h>
#define PIN_CE 22
#define PIN_CSN 4
#define PIN_BOTON 25
RF24 radio(PIN_CE, PIN_CSN);
const byte direccion[6] = "00009";

void setup() {
  Serial.begin(115200);
  pinMode(PIN_BOTON, INPUT_PULLDOWN);
  if (!radio.begin()) { Serial.println("NRF24 no detectado"); while (1); }
  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_1MBPS);
  radio.setRetries(5, 5);
  radio.openWritingPipe(direccion);
  radio.stopListening();
}

void loop() {
  bool estado = digitalRead(PIN_BOTON);   // sin segunda negacion
  bool ok = radio.write(&estado, sizeof(estado));
  Serial.println(ok ? "Enviado" : "Sin ACK");
  delay(100);
}

// RECEPTOR v1 (corregido)
#include <SPI.h>
#include <RF24.h>
#define PIN_CE 22
#define PIN_CSN 4
#define PIN_LED 12
RF24 radio(PIN_CE, PIN_CSN);
const byte direccion[6] = "00009";

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  if (!radio.begin()) { Serial.println("NRF24 no detectado"); while (1); }
  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_1MBPS);
  radio.openReadingPipe(1, direccion);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    bool estado;
    radio.read(&estado, sizeof(estado));
    digitalWrite(PIN_LED, estado);
    Serial.println(estado ? "Boton presionado" : "Boton suelto");
  }
}
```

A.2. Versión v2: hub (receptor en estrella)

```
#include <SPI.h>
#include <RF24.h>
```

```
RF24 radio(27, 4);  // CE, CSN

const byte dirSubida[][6] = {"NodU1", "NodU2", "NodU3"};  // nodos -> hub
const byte dirBajada[][6] = {"NodD1", "NodD2", "NodD3"};  // hub -> nodos
const int NUM_NODOS = 1;   // subir cuando se agreguen mas nodos

struct Mensaje {
  uint8_t id;
  int16_t valor;
  char texto[16];
};

unsigned long ultimoEnvio = 0;

void setup() {
  Serial.begin(115200);
  delay(2000);
  if (!radio.begin()) {
    Serial.println("NRF24 no detectado");
    while (true) delay(1000);
  }
  radio.setPALevel(RF24_PA_LOW);
  radio.setRetries(5, 5);
  for (int i = 0; i < NUM_NODOS; i++) radio.openReadingPipe(i + 1, dirSubida[i]);
  radio.startListening();
  Serial.println("Hub listo");
}

void loop() {
  uint8_t pipe;
  if (radio.available(&pipe)) {                  // recibir de cualquier nodo
    Mensaje m;
    radio.read(&m, sizeof(m));
    Serial.printf("Nodo %d (pipe %d): %s\n", m.id, pipe, m.texto);
  }

  if (millis() - ultimoEnvio > 5000) {           // responder a todos cada 5 s
    ultimoEnvio = millis();
    radio.stopListening();
    for (int i = 0; i < NUM_NODOS; i++) {
      Mensaje m = {0, (int16_t)(millis() / 1000), "Hub OK"};
      radio.openWritingPipe(dirBajada[i]);
      bool ok = radio.write(&m, sizeof(m));
      Serial.printf("Envio a nodo %d: %s\n", i + 1, ok ? "OK" : "sin ACK");
    }
    radio.startListening();
  }
}
```

A.3. Versión v2: nodo con botón

```
#include <SPI.h>
#include <RF24.h>

#define ID_NODO 1        // Si agregas mas nodos, cambia este numero en cada uno
#define BOTON 25

RF24 radio(27, 4);       // CE, CSN

const byte dirSubida[][6] = {"NodU1", "NodU2", "NodU3"};   // nodo -> hub
const byte dirBajada[][6] = {"NodD1", "NodD2", "NodD3"};   // hub -> nodo

struct Mensaje {
  uint8_t id;
  int16_t valor;
  char texto[16];
};

void setup() {
```

```
  Serial.begin(115200);
  delay(2000);
  pinMode(BOTON, INPUT_PULLDOWN);

  if (!radio.begin()) {
    Serial.println("NRF24 no detectado, revisa conexiones");
    while (true) delay(1000);
  }
  radio.setPALevel(RF24_PA_LOW);
  radio.setRetries(5, 5);

  radio.openWritingPipe(dirSubida[ID_NODO - 1]);
  radio.openReadingPipe(1, dirBajada[ID_NODO - 1]);
  radio.startListening();
  Serial.printf("Nodo %d listo\n", ID_NODO);
}

void loop() {
  if (radio.available()) {                      // mensajes del hub
    Mensaje m;
    radio.read(&m, sizeof(m));
    Serial.printf("Hub dice: %s | valor=%d\n", m.texto, m.valor);
  }

  if (digitalRead(BOTON) == HIGH) {             // boton -> hub
    Mensaje m = {ID_NODO, 1, "Boton"};
    radio.stopListening();
    bool ok = radio.write(&m, sizeof(m));
    radio.startListening();
    Serial.println(ok ? "Enviado al hub" : "Sin ACK");
    delay(300);                                 // anti-rebote
  }
}
```

A.4. Fragmento explorado: contador de pulsos del 555

```
// Fragmento explorado: medir la frecuencia de un generador de senales 555
// (entrada por divisor de voltaje 10k / 5.6k en GPIO25) y enviarla al hub.
#define PIN_SENAL 25
volatile uint32_t pulsos = 0;
unsigned long ultimoEnvio = 0;

void IRAM_ATTR contarPulso() { pulsos++; }

// en setup():
//   pinMode(PIN_SENAL, INPUT);
//   attachInterrupt(digitalPinToInterrupt(PIN_SENAL), contarPulso, RISING);

// en loop():
//   if (millis() - ultimoEnvio >= 1000) {
//     ultimoEnvio = millis();
//     noInterrupts(); uint32_t n = pulsos; pulsos = 0; interrupts();
//     Mensaje m = {ID_NODO, (int16_t)n, "Frecuencia"};
//     radio.stopListening(); bool ok = radio.write(&m, sizeof(m)); radio.startListening();
//     Serial.printf("Frecuencia: %u Hz -> %s\n", n, ok ? "OK" : "Sin ACK");
//   }
```

A.5. Router con cifrado XOR (nodo.ino, rama silviorodriguez)

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define CE_PIN 2
#define CSN_PIN 4
#define LED_INDICATOR 12

RF24 radio(CE_PIN, CSN_PIN);
```

```
const byte listenAddresses[6][5] = {
  {0x11, 0x22, 0x33, 0x44, 0x55},
  {0x66, 0x77, 0x88, 0x99, 0xAA},
  {0x66, 0x77, 0x88, 0x99, 0xBB},
  {0x66, 0x77, 0x88, 0x99, 0xCC},
  {0x66, 0x77, 0x88, 0x99, 0xDD},
  {0x66, 0x77, 0x88, 0x99, 0xEE}
};

const byte destAddresses[3][5] = {
  {0xAA, 0xAA, 0xAA, 0xAA, 0x01},
  {0xAA, 0xAA, 0xAA, 0xAA, 0x02},
  {0xAA, 0xAA, 0xAA, 0xAA, 0x03}
};

struct DataPacket {
  uint8_t targetID;
  uint8_t dataLen;
  char binaryData[16];
};

DataPacket payload;

// Función XOR que solo cifra la cantidad exacta de bytes
void processXOR(char* data, uint8_t len) {
  const char key[] = "C1Ph3rK3y";
  int keyLen = sizeof(key) - 1;
  for (uint8_t i = 0; i < len; i++) {
    data[i] ^= key[i % keyLen];
  }
}

// Función para imprimir bytes cifrados en HEX sin romper la terminal
void printHEX(char* data, uint8_t len) {
  for (uint8_t i = 0; i < len; i++) {
    if ((uint8_t)data[i] < 0x10) Serial.print("0");
    Serial.print((uint8_t)data[i], HEX);
    Serial.print(" ");
  }
  Serial.println();
}

void setup() {
  pinMode(LED_INDICATOR, OUTPUT);
  Serial.begin(115200);
  Serial.println("\n--- Nodo Enrutador Activo ---");

  radio.begin();
  radio.setPALevel(RF24_PA_MIN);
  radio.setDataRate(RF24_1MBPS);

  for(uint8_t i = 0; i < 6; i++) {
    radio.openReadingPipe(i, listenAddresses[i]);
  }

  radio.startListening();
}

void loop() {
  uint8_t pipeNum;

  if (radio.available(&pipeNum)) {
    memset(&payload, 0, sizeof(payload));
    radio.read(&payload, sizeof(payload));

    digitalWrite(LED_INDICATOR, HIGH);

    // Asegurar límite de tamaño por seguridad
    if (payload.dataLen > 15) payload.dataLen = 15;

    Serial.print("\n[ROUTER] Mensaje recibido del Pipe ");
    Serial.print(pipeNum);
```

```
    Serial.print(" -> Texto plano: ");
    Serial.println(payload.binaryData);

    // Cifrar los bytes reales
    processXOR(payload.binaryData, payload.dataLen);

    Serial.print("[ROUTER] Mensaje Cifrado (HEX): ");
    printHEX(payload.binaryData, payload.dataLen);

    // Reenviar
    radio.stopListening();

    if (payload.targetID < 3) {
      radio.openWritingPipe(destAddresses[payload.targetID]);

      if (radio.write(&payload, sizeof(payload))) {
        Serial.print("[ROUTER] Redirigido con exito a Destino ");
        Serial.println(payload.targetID);
      } else {
        Serial.println("[ROUTER] Error al reenviar");
      }
    }

    radio.startListening();
    digitalWrite(LED_INDICATOR, LOW);
  }
}
```

A.6. Emisor (emisor.ino, rama radiantescachorros)

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define CE_PIN   2
#define CSN_PIN  4
#define PIN_LED  22 // GPIO conectado al ánodo del LED verde

RF24 radio(CE_PIN, CSN_PIN);

// Debe coincidir con destAddresses[0] del router
// Si eres el destino 1 usa 0x02 al final; si eres el 2 usa 0x03
const byte myAddress[5] = {0xAA, 0xAA, 0xAA, 0xAA, 0x01};

struct DataPacket {
  uint8_t targetID;
  uint8_t dataLen;
  char binaryData[16];
};

DataPacket payload;

// Misma rutina XOR que el router para descifrar en destino
void processXOR(char* data, uint8_t len) {
  const char key[] = "C1Ph3rK3y";
  int keyLen = sizeof(key) - 1;
  for (uint8_t i = 0; i < len; i++) {
    data[i] ^= key[i % keyLen];
  }
}

void printHEX(char* data, uint8_t len) {
  for (uint8_t i = 0; i < len; i++) {
    if ((uint8_t)data[i] < 0x10) Serial.print("0");
    Serial.print((uint8_t)data[i], HEX);
    Serial.print(" ");
  }
  Serial.println();
}
```

```
void setup() {
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);

  Serial.begin(115200);
  Serial.println("\n--- Nodo Receptor (Destino 0) Activo ---");

  radio.begin();
  radio.setPALevel(RF24_PA_MIN);
  radio.setDataRate(RF24_1MBPS);

  radio.openReadingPipe(1, myAddress);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    memset(&payload, 0, sizeof(payload));
    radio.read(&payload, sizeof(payload));

    if (payload.dataLen > 15) payload.dataLen = 15;

    Serial.print("\n[RX] Paquete cifrado recibido (HEX): ");
    printHEX(payload.binaryData, payload.dataLen);

    // Descifrado simétrico
    processXOR(payload.binaryData, payload.dataLen);

    Serial.print("[RX] Mensaje descifrado: ");
    Serial.println(payload.binaryData);

    // Indicador visual de recepción
    digitalWrite(PIN_LED, HIGH);
    delay(120);
    digitalWrite(PIN_LED, LOW);
  }
}
```

A.7. Teléfono con micrófono (Nodo_Telefono.ino, rama equipoqeso)

Versión con micrófono y nivel de voz. En el nodo del autor PIN_CE se cambió a 22 para coincidir con el cableado, y MI_ID es distinto en cada nodo.

```
/*
  NODO TELEFONO - Topologia estrella con central (nRF24L01+ PA+LNA + ESP32)
  ------------------------------------------------------------------------------
  Se carga el MISMO codigo en todos los nodos; solo cambia MI_ID (1 a 254).
  Necesita que la CENTRAL (Central_Estrella.ino, ID 0) este encendida.

  Comandos en el Monitor Serial:
     /llamar 2    -> pide a la central llamar al nodo 2
     /contestar   -> contesta una llamada entrante
     /rechazar    -> rechaza una llamada entrante
     /colgar      -> termina la llamada (o cancela si aun esta timbrando)
     /estado      -> muestra en que estado y canal esta el nodo
     texto normal -> durante una llamada, se lo manda a la otra persona

     /mic         -> muestra en vivo el nivel de tu microfono (para probarlo)
     /ganancia N  -> sensibilidad del microfono (1 a 20, por defecto 4)
     /calibrar    -> vuelve a medir el ruido de fondo (guarda silencio 1 s)

  Mensajes de la central: aparecen como [CENTRAL] ... en cualquier momento,
  incluso a media llamada (v2).

  MICROFONO -> LED (v3): durante una llamada, cada telefono mide que tan fuerte
  le hablan a SU microfono y le manda ese nivel a la otra persona 20 veces por
  segundo. El LED de la otra persona brilla mas o menos segun tu voz.
  (Modulo de sonido KY-037/KY-038: se usa la salida analogica AO.)

  Boton BOOT: contesta si esta timbrando, cuelga si esta en llamada.
  LED: parpadea rapido = timbrando, lento = llamando,
       en llamada = brilla con la voz de la otra persona.
```

```
  FUNCIONA EN DOS PLACAS: el codigo detecta solo cual elegiste en
  Tools -> Board y usa los pines correctos. Solo cablea segun tu placa:

  a) ESP32 DevKit normal (ESP-WROOM-32) -> placa "ESP32 Dev Module"
     Mismo cableado que el circuito del profe:
     VCC->3V3  GND->GND  CE->GPIO2  CSN->GPIO4
     SCK->GPIO18  MISO->GPIO19  MOSI->GPIO23
     LED externo en GPIO12 (con resistencia de 330 ohm a GND)
     Boton BOOT de la placa = GPIO0
     Microfono: AO->GPIO34  +->3V3  GND->GND  (DO sin conectar)

  b) ESP32-C3 SuperMini -> placa "ESP32C3 Dev Module", "USB CDC On Boot: Enabled"
     VCC->3V3  GND->GND  CE->GPIO3  CSN->GPIO7
     SCK->GPIO4  MOSI->GPIO6  MISO->GPIO5
     LED de la placa = GPIO8, boton BOOT = GPIO9
     Microfono: AO->GPIO1  +->3V3  GND->GND  (DO sin conectar)

  En ambos: capacitor 10-100uF entre VCC y GND del nRF.
  Monitor Serial a 115200 con "Nueva linea".
*/
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

// ================== CONFIGURA AQUI ==================
#define MI_ID 1          // <-- distinto en cada nodo: 1, 2, 3 ...
// ====================================================

#if defined(CONFIG_IDF_TARGET_ESP32C3)
  // ---- ESP32-C3 SuperMini ----
  #define PIN_CE    3
  #define PIN_CSN   7
  #define PIN_SCK   4
  #define PIN_MISO  5
  #define PIN_MOSI  6
  #define PIN_LED   8
  #define PIN_BOOT  9
  #define PIN_MIC   1        // ADC1 (GPIO 0-4 son los analogicos del C3)
  #define LED_ON    LOW      // el LED de la placa enciende con LOW
  #define PLACA     "ESP32-C3 SuperMini"
#elif defined(CONFIG_IDF_TARGET_ESP32)
  // ---- ESP32 DevKit normal (ESP-WROOM-32), cableado del profe ----
  // OJO: en este ESP32 los GPIO 6 a 11 son de la memoria flash interna;
  // si se usan, la placa se reinicia sin parar.
  #define PIN_CE    2
  #define PIN_CSN   4
  #define PIN_SCK   18
  #define PIN_MISO  19
  #define PIN_MOSI  23
  #define PIN_LED   12
  #define PIN_BOOT  0
  #define PIN_MIC   34       // ADC1, solo entrada: ideal para el microfono
  #define LED_ON    HIGH     // LED externo: enciende con HIGH
  #define PLACA     "ESP32 DevKit"
#else
  #error "Placa no soportada: elige ESP32 Dev Module o ESP32C3 Dev Module en Tools -> Board"
#endif
#define LED_OFF (LED_ON == HIGH ? LOW : HIGH)

// ======================= PROTOCOLO (igual en central y nodos) =======================
#define ID_CENTRAL     0
#define CANAL_CONTROL  80

enum Tipo : uint8_t {
  REGISTRO = 1, LLAMAR, CONTESTAR, RECHAZAR, LIBERAR,
  ENTRANTE, CONECTAR, RESPUESTA, TEXTO, KEEPALIVE, COLGAR,
  AVISO,         // central -> nodo : mensaje de la central (v2)
  VOZ            // nodo <-> nodo   : nivel de voz 0-255 en 'dato' (v3)
};
enum Codigo : uint8_t {
  R_TIMBRANDO = 1, R_OCUPADO, R_NO_DISPONIBLE, R_RECHAZADO,
  R_NO_CONTESTA, R_SIN_CANALES, R_CANCELADO
```

```
};
struct Paquete {
  uint8_t tipo;
  uint8_t origen;
  uint8_t destino;
  uint8_t canal;
  uint8_t dato;
  uint8_t secuencia;
  char    texto[26];
};
// Estados del telefono. Va ANTES de la primera funcion: el IDE de Arduino
// genera prototipos automaticos arriba de ella y necesita conocer este tipo.
enum Estado { REPOSO, LLAMANDO, TIMBRANDO, EN_LLAMADA };

void direccionDe(uint8_t id, uint8_t *dir) {
  dir[0] = id; dir[1] = 'N'; dir[2] = 'O'; dir[3] = 'D'; dir[4] = 'E';
}
// ================================================================================

RF24 radio(PIN_CE, PIN_CSN);
uint8_t secuencia = 0;

Estado estado = REPOSO;
uint8_t pareja = 0;              // con quien hablo
uint8_t canalActual = CANAL_CONTROL;
unsigned long tEstado = 0;       // cuando entre al estado actual
unsigned long tUltimoEnvio = 0;
unsigned long tRegistro = 0;
uint8_t fallosSeguidos = 0;
bool centralPerdida = false;

// ---- Microfono ----
#define VENTANA_MS     50        // se mide el volumen cada 50 ms (20 veces por segundo)
int   micMin = 4095, micMax = 0; // minimo y maximo dentro de la ventana actual
unsigned long tVentana = 0;
int   picoAPico = 0;             // amplitud de la ultima ventana (0-4095)
int   ruidoBase = 10;            // amplitud del silencio (se calibra al encender)
int   ganancia = 4;              // sensibilidad: mas alto = el LED reacciona a voz mas baja
uint8_t nivelMic = 0;            // MI volumen, 0-255 (con envolvente)
bool  ventanaNueva = false;
bool  modoMic = false;           // /mic: imprimir el nivel en vivo
uint8_t nivelRemoto = 0;         // volumen de la OTRA persona, 0-255
unsigned long tVozRemota = 0;
unsigned long tContacto = 0;     // ultima vez que supimos de la pareja
#define SIN_CONTACTO_MS 10000UL  // 10 s sin saber de la pareja -> se corta

#define REGISTRO_CADA_MS   5000UL
#define ESPERA_MAX_MS      25000UL  // tiempo maximo llamando / timbrando

const char *nombreEstado[] = {"REPOSO", "LLAMANDO", "TIMBRANDO", "EN LLAMADA"};

// --------------------------------------------------------------------------------
bool enviar(uint8_t destino, uint8_t tipo, uint8_t dato, const char *texto) {
  Paquete p = {};
  p.tipo = tipo; p.origen = MI_ID; p.destino = destino;
  p.canal = canalActual; p.dato = dato; p.secuencia = secuencia++;
  if (texto) strncpy(p.texto, texto, sizeof(p.texto) - 1);
  uint8_t dir[5]; direccionDe(destino, dir);
  radio.stopListening();
  radio.openWritingPipe(dir);
  bool ok = radio.write(&p, sizeof(p));
  radio.startListening();
  tUltimoEnvio = millis();
  return ok;
}

void cambiarCanal(uint8_t c) {
  radio.stopListening();
  radio.setChannel(c);
  radio.startListening();
  canalActual = c;
}
```

```
void cambiarEstado(Estado e) {
  estado = e;
  tEstado = millis();
  tContacto = millis();
  fallosSeguidos = 0;
  nivelRemoto = 0;
}

// ---- LED con brillo (PWM). 0 = apagado, 255 = maximo ----
void ledBrillo(uint8_t v) {
  analogWrite(PIN_LED, LED_ON == HIGH ? v : 255 - v);
}

// ---- Microfono ----
// Se llama en cada vuelta del loop: toma una muestra y, cada 50 ms,
// calcula el volumen como la diferencia entre el maximo y el minimo (pico a pico).
void leerMicrofono() {
  int v = analogRead(PIN_MIC);
  if (v < micMin) micMin = v;
  if (v > micMax) micMax = v;
  if (millis() - tVentana < VENTANA_MS) return;

  tVentana = millis();
  picoAPico = micMax - micMin;
  micMin = 4095; micMax = 0;

  int n = (picoAPico - ruidoBase) * ganancia;      // quitar el ruido de fondo y amplificar
  n = constrain(n, 0, 255);
  if (n > nivelMic) nivelMic = n;                  // sube rapido...
  else nivelMic = (nivelMic * 2 + n) / 3;          // ...y baja suave (se ve mas natural)
  ventanaNueva = true;
}

// Mide 1 segundo de silencio para saber cuanto "ruido" tiene el microfono solo.
void calibrarMicrofono() {
  Serial.println("Calibrando microfono: guarda silencio 1 segundo...");
  long suma = 0; int ventanas = 0;
  unsigned long t0 = millis();
  while (millis() - t0 < 1000) {
    int mn = 4095, mx = 0;
    unsigned long tw = millis();
    while (millis() - tw < VENTANA_MS) {
      int v = analogRead(PIN_MIC);
      if (v < mn) mn = v;
      if (v > mx) mx = v;
    }
    suma += mx - mn; ventanas++;
  }
  ruidoBase = suma / ventanas + 3;
  Serial.printf("Listo. Ruido de fondo = %d (escala 0-4095), ganancia = %d\n", ruidoBase, ganancia);
}

// Brillo percibido: el ojo es mas sensible a cambios en niveles bajos (correccion gamma)
uint8_t gamma8(uint8_t v) { return (uint16_t)v * v / 255; }

// Regresa al canal de control y avisa a la central que el nodo quedo libre
void volverAReposo(bool avisarCentral) {
  if (canalActual != CANAL_CONTROL) cambiarCanal(CANAL_CONTROL);
  if (avisarCentral) enviar(ID_CENTRAL, LIBERAR, 0, nullptr);
  pareja = 0;
  cambiarEstado(REPOSO);
  enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
  tRegistro = millis();
  Serial.println("En reposo (canal de control).");
}

// ------------------------------------------------------------------------------
void procesarLinea(String l) {
  l.trim();
  if (l.length() == 0) return;
```

```
  if (l.startsWith("/llamar")) {
    if (estado != REPOSO) { Serial.println("Primero termina lo que estas haciendo (/colgar)."); return; }
    int destino = l.substring(7).toInt();
    if (destino < 1 || destino > 254 || destino == MI_ID) { Serial.println("Uso: /llamar ID"); return; }
    if (!enviar(ID_CENTRAL, LLAMAR, destino, nullptr)) { Serial.println("La central no responde."); return; }
    pareja = destino;
    cambiarEstado(LLAMANDO);
    Serial.printf("Llamando al nodo %u...\n", destino);
  }
  else if (l == "/contestar") {
    if (estado != TIMBRANDO) { Serial.println("No hay llamada entrante."); return; }
    enviar(ID_CENTRAL, CONTESTAR, 0, nullptr);
    Serial.println("Contestando, esperando canal de la central...");
  }
  else if (l == "/rechazar") {
    if (estado != TIMBRANDO) { Serial.println("No hay llamada entrante."); return; }
    enviar(ID_CENTRAL, RECHAZAR, 0, nullptr);
    Serial.println("Llamada rechazada.");
    volverAReposo(false);
  }
  else if (l == "/colgar") {
    if (estado == EN_LLAMADA) {
      enviar(pareja, COLGAR, 0, nullptr);                 // avisar a la otra persona
      Serial.println("Colgaste.");
      volverAReposo(true);
    } else if (estado == LLAMANDO || estado == TIMBRANDO) {
      Serial.println("Llamada cancelada.");
      volverAReposo(true);                     // la central avisa al otro
    } else Serial.println("No estas en llamada.");
  }
  else if (l == "/estado") {
    Serial.printf("Nodo %u | %s | canal %u (%u MHz)", MI_ID, nombreEstado[estado],
                  canalActual, 2400 + canalActual);
    if (pareja) Serial.printf(" | con nodo %u", pareja);
    Serial.println();
  }
  else if (l == "/mic") {
    modoMic = !modoMic;
    Serial.println(modoMic ? "Mostrando nivel del microfono (escribe /mic otra vez para parar)"
                           : "Listo, ya no se muestra el microfono.");
  }
  else if (l.startsWith("/ganancia")) {
    int g = l.substring(9).toInt();
    if (g < 1 || g > 20) { Serial.println("Uso: /ganancia 1..20"); return; }
    ganancia = g;
    Serial.printf("Ganancia = %d\n", ganancia);
  }
  else if (l == "/calibrar") calibrarMicrofono();
  else if (l.startsWith("/")) {
    Serial.println("Comandos: /llamar ID  /contestar  /rechazar  /colgar  /estado  /mic  /ganancia N  /calibrar");
  }
  else {                                        // texto normal
    if (estado != EN_LLAMADA) { Serial.println("No estas en llamada. Usa /llamar ID"); return; }
    String t = l.substring(0, 25);
    bool ok = enviar(pareja, TEXTO, 0, t.c_str());
    Serial.printf("[yo] %s %s\n", t.c_str(), ok ? "" : "(no llego)");
    if (l.length() > 25) Serial.println("  aviso: se corto a 25 caracteres");
  }
}

void procesarPaquete(Paquete &p) {
  p.texto[sizeof(p.texto) - 1] = '\0';

  // ---- Mensajes de la central (canal de control) ----
  if (p.origen == ID_CENTRAL) {
    centralPerdida = false;
    switch (p.tipo) {
      case AVISO:      // mensaje de la central: se muestra en cualquier estado
        Serial.printf("\n[CENTRAL] %s\n", p.texto);
        break;

      case ENTRANTE:
```

```
        if (estado != REPOSO) break;
        pareja = p.dato;
        cambiarEstado(TIMBRANDO);
        Serial.printf("\n*** RING RING *** El nodo %u te llama. /contestar o /rechazar\n", pareja);
        break;

      case CONECTAR:
        if (estado != LLAMANDO && estado != TIMBRANDO) break;
        pareja = p.dato;
        cambiarCanal(p.canal);               // <-- nos vamos a nuestra propia linea
        cambiarEstado(EN_LLAMADA);
        Serial.printf("CONECTADO con nodo %u en canal %u (%u MHz). Escribe para hablar, /colgar para terminar.\n",
                      pareja, p.canal, 2400 + p.canal);
        break;

      case RESPUESTA:
        switch (p.dato) {
          case R_TIMBRANDO:     Serial.printf("Timbrando en el nodo %u...\n", pareja); break;
          case R_OCUPADO:       Serial.println("El nodo esta OCUPADO.");        volverAReposo(false); break;
          case R_NO_DISPONIBLE: Serial.println("El nodo NO ESTA DISPONIBLE."); volverAReposo(false); break;
          case R_RECHAZADO:     Serial.println("Llamada RECHAZADA.");          volverAReposo(false); break;
          case R_NO_CONTESTA:   Serial.println("El nodo NO CONTESTA.");        volverAReposo(false); break;
          case R_SIN_CANALES:   Serial.println("Central saturada: no hay canales libres."); volverAReposo(false);
break;
          case R_CANCELADO:
            if (estado == TIMBRANDO) { Serial.println("La llamada se cancelo."); volverAReposo(false); }
            break;
        }
        break;
    }
    return;
  }

  // ---- Mensajes de la pareja (en nuestro canal de voz) ----
  if (estado != EN_LLAMADA || p.origen != pareja) return;
  fallosSeguidos = 0;
  tContacto = millis();
  switch (p.tipo) {
    case VOZ:    nivelRemoto = p.dato; tVozRemota = millis(); break;
    case TEXTO:  Serial.printf("[nodo %u] %s\n", p.origen, p.texto); break;
    case COLGAR: Serial.printf("El nodo %u colgo.\n", p.origen); volverAReposo(true); break;
    case KEEPALIVE: break;
  }
}

// Tareas periodicas segun el estado
void tareasPeriodicas() {
  unsigned long ahora = millis();

  if (estado == REPOSO && ahora - tRegistro > REGISTRO_CADA_MS) {
    tRegistro = ahora;
    bool ok = enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
    if (!ok && !centralPerdida) { Serial.println("Aviso: la central no responde."); centralPerdida = true; }
    if (ok && centralPerdida)   { Serial.println("Central encontrada.");            centralPerdida = false; }
  }

  if ((estado == LLAMANDO || estado == TIMBRANDO) && ahora - tEstado > ESPERA_MAX_MS) {
    Serial.println("Tiempo de espera agotado.");
    volverAReposo(true);
  }

  // En llamada: cada 50 ms mandar MI nivel de voz a la pareja.
  // Ese mismo envio sirve para comprobar que la pareja sigue ahi (ACK).
  if (estado == EN_LLAMADA && ventanaNueva) {
    if (enviar(pareja, VOZ, nivelMic, nullptr)) tContacto = ahora;
    if (ahora - tContacto > SIN_CONTACTO_MS) {
      Serial.println("Se perdio la conexion con la pareja. Llamada terminada.");
      volverAReposo(true);
    }
  }
  if (ahora - tVozRemota > 300) nivelRemoto = 0;   // si deja de llegar voz, apagar
```

```
  // /mic: barra con el nivel del microfono (cada 100 ms)
  static unsigned long tBarra = 0;
  if (modoMic && ventanaNueva && ahora - tBarra >= 100) {
    tBarra = ahora;
    Serial.printf("mic: amplitud %4d  nivel %3u |", picoAPico, nivelMic);
    for (int i = 0; i < nivelMic / 8; i++) Serial.print('#');
    Serial.println();
  }
  ventanaNueva = false;

  // LED
  uint8_t brillo = 0;
  if (estado == TIMBRANDO)  brillo = ((ahora / 150) % 2) ? 255 : 0;
  if (estado == LLAMANDO)   brillo = ((ahora / 600) % 2) ? 255 : 0;
  if (estado == EN_LLAMADA) brillo = max((uint8_t)6, gamma8(nivelRemoto));  // tenue = en llamada
  if (estado == REPOSO && modoMic) brillo = gamma8(nivelMic);              // prueba local
  ledBrillo(brillo);
}

// ---------------------------------------------------------------------------------
void setup() {
  Serial.begin(115200);
  delay(1500);
  pinMode(PIN_LED, OUTPUT);
  ledBrillo(0);
  pinMode(PIN_BOOT, INPUT_PULLUP);
  pinMode(PIN_MIC, INPUT);
  analogReadResolution(12);                   // 0-4095

  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CSN);
  if (!radio.begin(&SPI)) {
    Serial.println("ERROR: el nRF24L01 no responde. Revisa cables y alimentacion.");
    while (true) { ledBrillo(255); delay(100); ledBrillo(0); delay(100); }
  }
  radio.setChannel(CANAL_CONTROL);
  radio.setDataRate(RF24_1MBPS);
  radio.setPALevel(RF24_PA_LOW);
  radio.setPayloadSize(sizeof(Paquete));
  radio.setAutoAck(true);
  radio.setRetries(3 + (MI_ID % 10), 15);    // reintentos escalonados por nodo
  uint8_t dir[5]; direccionDe(MI_ID, dir);
  radio.openReadingPipe(1, dir);
  radio.startListening();

  Serial.printf("\n=== Nodo telefono %u listo (%s) ===\n", MI_ID, PLACA);
  Serial.println("Comandos: /llamar ID  /contestar  /rechazar  /colgar  /estado  /mic  /ganancia N  /calibrar");
  calibrarMicrofono();
  tRegistro = millis() - REGISTRO_CADA_MS;    // registrarse de inmediato
}

void loop() {
  static String linea = "";
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') { procesarLinea(linea); linea = ""; }
    else if (linea.length() < 100) linea += c;
  }

  // Boton BOOT: contestar o colgar
  static unsigned long tBoton = 0;
  if (digitalRead(PIN_BOOT) == LOW && millis() - tBoton > 600) {
    tBoton = millis();
    if (estado == TIMBRANDO) procesarLinea("/contestar");
    else if (estado == EN_LLAMADA) procesarLinea("/colgar");
  }

  if (radio.available()) {
    Paquete p;
    radio.read(&p, sizeof(p));
    procesarPaquete(p);
  }
```

```
  leerMicrofono();
  tareasPeriodicas();
}
```

A.8. Central telefónica v3 (Central_Estrella.ino, rama equipoqeso)

```
/*
  CENTRAL TELEFONICA v2 - Topologia estrella con nRF24L01+ PA+LNA
  ---------------------------------------------------------------
  Novedades de la v2:
   - La central puede mandar MENSAJES a todos los nodos (/todos) o a uno (/msg).
     Si un nodo esta en llamada, la central salta a su canal, se lo entrega
     y regresa al canal de control.
   - La central MONITOREA todos los canales: mide constantemente si hay
     interferencia (WiFi, Bluetooth, otros equipos) y solo asigna canales limpios.
   - Usa TODOS los canales de la banda ISM de 2.4 GHz: del 0 (2400 MHz)
     al 83 (2483 MHz). El 80 es el de control; quedan 83 para llamadas.

  Como funciona (igual que antes):
   1. Los nodos se registran con la central en el CANAL DE CONTROL.
   2. Un nodo pide llamar a otro; la central lo hace timbrar.
   3. Si contesta, la central elige el mejor canal libre (sin interferencia
      y lo mas lejos posible de los canales ocupados) y manda a los dos ahi.
   4. Al colgar, regresan al canal de control y el canal queda libre.

  Comandos en el Monitor Serial de la central:
     /todos texto     -> mensaje a TODOS los nodos (incluso los que estan en llamada)
     /msg ID texto    -> mensaje solo al nodo ID
     /tabla           -> nodos registrados y su estado
     /canales         -> mapa de los 84 canales y llamadas activas
     /ruido           -> canales con interferencia detectada (con porcentaje)
     /ayuda           -> esta lista

  PLACAS: el codigo detecta solo la placa elegida en Tools -> Board:
   a) ESP32-S3 SuperMini ("ESP32S3 Dev Module", USB CDC On Boot: Enabled)
        CE->9  CSN->10  MOSI->11  SCK->12  MISO->13   LED RGB de la placa (GPIO48)
   b) ESP32-C3 SuperMini ("ESP32C3 Dev Module", USB CDC On Boot: Enabled)
        CE->3  CSN->7  SCK->4  MOSI->6  MISO->5       LED de la placa (GPIO8)
   c) ESP32 DevKit normal ("ESP32 Dev Module")
        CE->2  CSN->4  SCK->18  MISO->19  MOSI->23    LED externo en GPIO12
   En todas: VCC->3V3, GND->GND, capacitor 10-100uF entre VCC y GND del nRF.
   Monitor Serial a 115200 con "Nueva linea".
*/
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

// ============================ PINES SEGUN LA PLACA ============================
#if defined(CONFIG_IDF_TARGET_ESP32S3)
  #define PIN_CE 9
  #define PIN_CSN 10
  #define PIN_MOSI 11
  #define PIN_SCK 12
  #define PIN_MISO 13
  #define PIN_RGB 48
  #define PLACA "ESP32-S3 SuperMini"
#elif defined(CONFIG_IDF_TARGET_ESP32C3)
  #define PIN_CE 3
  #define PIN_CSN 7
  #define PIN_SCK 4
  #define PIN_MISO 5
  #define PIN_MOSI 6
  #define PIN_LED 8
  #define LED_ON LOW
  #define PLACA "ESP32-C3 SuperMini"
#elif defined(CONFIG_IDF_TARGET_ESP32)
  #define PIN_CE 2
  #define PIN_CSN 4
  #define PIN_SCK 18
  #define PIN_MISO 19
```

```
  #define PIN_MOSI 23
  #define PIN_LED 12
  #define LED_ON HIGH
  #define PLACA "ESP32 DevKit"
#else
  #error "Placa no soportada: elige ESP32S3, ESP32C3 o ESP32 Dev Module en Tools -> Board"
#endif

// ====================== PROTOCOLO (igual en central y nodos) ======================
#define ID_CENTRAL     0
#define CANAL_CONTROL  80        // 2480 MHz: donde todos esperan y piden llamadas

// Rango de canales que la central puede asignar a las llamadas.
// 0..83 = 2400..2483 MHz = toda la banda ISM de 2.4 GHz (uso libre).
// El nRF24 llega hasta el 125 (2525 MHz), pero del 84 en adelante ya esta
// FUERA de la banda de uso libre, por eso no se usan.
#define CANAL_MIN      0
#define CANAL_MAX      83

enum Tipo : uint8_t {
  REGISTRO = 1,  // nodo -> central : "estoy aqui y libre" (cada 5 s)
  LLAMAR,        // nodo -> central : quiero llamar a 'dato'
  CONTESTAR,     // nodo -> central : acepto la llamada entrante
  RECHAZAR,      // nodo -> central : rechazo la llamada entrante
  LIBERAR,       // nodo -> central : colgue / cancelo
  ENTRANTE,      // central -> nodo : te llama 'dato'
  CONECTAR,      // central -> nodo : vete al canal 'canal' con la pareja 'dato'
  RESPUESTA,     // central -> nodo : aviso (codigo en 'dato')
  TEXTO,         // nodo <-> nodo   : mensaje dentro de la llamada
  KEEPALIVE,     // nodo <-> nodo   : "sigo aqui" durante la llamada
  COLGAR,        // nodo <-> nodo   : termino la llamada
  AVISO,         // central -> nodo : mensaje de la central (v2)
  VOZ            // nodo <-> nodo   : nivel de voz 0-255 (v3, la central no lo usa)
};
enum Codigo : uint8_t {
  R_TIMBRANDO = 1, R_OCUPADO, R_NO_DISPONIBLE, R_RECHAZADO,
  R_NO_CONTESTA, R_SIN_CANALES, R_CANCELADO
};
struct Paquete {          // 32 bytes exactos (maximo del nRF24)
  uint8_t tipo;
  uint8_t origen;
  uint8_t destino;
  uint8_t canal;
  uint8_t dato;
  uint8_t secuencia;
  char    texto[26];
};

// ============================ TABLAS DE LA CENTRAL ============================
#define MAX_NODOS       32
#define NODO_CADUCA_MS  15000UL   // sin registrarse en 15 s = desconectado
struct Nodo { uint8_t id; unsigned long visto; };

#define MAX_LLAMADAS    16        // llamadas simultaneas (con 32 nodos, maximo 16)
#define TIMBRE_MAX_MS   20000UL   // 20 s timbrando sin contestar -> "no contesta"
enum EstadoLlamada : uint8_t { LIBRE = 0, TIMBRANDO, EN_CURSO };
struct Llamada {
  EstadoLlamada estado;
  uint8_t origen, destino, canal;
  unsigned long desde;
};

// Monitoreo de interferencia
#define UMBRAL_RUIDO      25      // % de muestras con senal a partir del cual el canal se evita
#define MONITOREO_CADA_MS 40      // cada 40 ms se revisa un canal (barrido completo ~3.4 s)
#define MUESTRAS_MONITOR  4       // muestras por revision periodica
#define MUESTRAS_VERIFICA 20      // muestras al verificar un canal antes de asignarlo
// ================================================================================

void direccionDe(uint8_t id, uint8_t *dir) {   // direccion de 5 bytes de cada nodo
  dir[0] = id; dir[1] = 'N'; dir[2] = 'O'; dir[3] = 'D'; dir[4] = 'E';
}
```

```
RF24 radio(PIN_CE, PIN_CSN);
uint8_t secuencia = 0;

Nodo nodos[MAX_NODOS];
uint8_t numNodos = 0;
Llamada llamadas[MAX_LLAMADAS];
uint8_t ruido[CANAL_MAX + 1];         // % de ocupacion externa medida en cada canal
uint8_t canalMonitoreo = CANAL_MIN;
unsigned long tMonitoreo = 0;

// ---------------------------------------------------------------------------
// LED: 0 = apagado, 1 = central lista, 2 = hay llamadas, 3 = error
void ledModo(uint8_t modo) {
#ifdef PIN_RGB
  uint8_t r = 0, g = 0, b = 0;
  if (modo == 1) b = 10;
  if (modo == 2) g = 40;
  if (modo == 3) r = 40;
  #if ESP_ARDUINO_VERSION_MAJOR >= 3
    rgbLedWrite(PIN_RGB, r, g, b);
  #else
    neopixelWrite(PIN_RGB, r, g, b);
  #endif
#else
  digitalWrite(PIN_LED, modo >= 2 ? LED_ON : !LED_ON);   // LED simple: encendido si hay llamadas
#endif
}

// ---------------------------------------------------------------------------
// Envia un paquete en el canal indicado y regresa al canal de control.
bool enviarEnCanal(uint8_t canalRF, uint8_t destino, uint8_t tipo,
                   uint8_t canal, uint8_t dato, const char *texto) {
  Paquete p = {};
  p.tipo = tipo; p.origen = ID_CENTRAL; p.destino = destino;
  p.canal = canal; p.dato = dato; p.secuencia = secuencia++;
  if (texto) strncpy(p.texto, texto, sizeof(p.texto) - 1);
  uint8_t dir[5]; direccionDe(destino, dir);

  radio.stopListening();
  if (canalRF != CANAL_CONTROL) radio.setChannel(canalRF);
  radio.openWritingPipe(dir);
  bool ok = radio.write(&p, sizeof(p));
  if (canalRF != CANAL_CONTROL) radio.setChannel(CANAL_CONTROL);
  radio.startListening();
  return ok;
}

bool enviar(uint8_t destino, uint8_t tipo, uint8_t canal, uint8_t dato) {
  return enviarEnCanal(CANAL_CONTROL, destino, tipo, canal, dato, nullptr);
}

// ---- Nodos ----
int buscarNodo(uint8_t id) {
  for (int i = 0; i < numNodos; i++) if (nodos[i].id == id) return i;
  return -1;
}
bool nodoEnLinea(uint8_t id) {
  int i = buscarNodo(id);
  return i >= 0 && millis() - nodos[i].visto < NODO_CADUCA_MS;
}
void registrarNodo(uint8_t id) {
  int i = buscarNodo(id);
  if (i < 0) {
    if (numNodos >= MAX_NODOS) return;
    i = numNodos++;
    nodos[i].id = id;
    Serial.printf("Nodo %u registrado\n", id);
  }
  nodos[i].visto = millis();
}
```

```
// ---- Llamadas ----
int llamadaDe(uint8_t id) {             // llamada donde participa 'id', o -1
  for (int i = 0; i < MAX_LLAMADAS; i++)
    if (llamadas[i].estado != LIBRE &&
        (llamadas[i].origen == id || llamadas[i].destino == id)) return i;
  return -1;
}
int llamadaEnCanal(uint8_t c) {         // llamada que ocupa el canal c, o -1
  for (int i = 0; i < MAX_LLAMADAS; i++)
    if (llamadas[i].estado != LIBRE && llamadas[i].canal == c) return i;
  return -1;
}
int espacioLlamada() {
  for (int i = 0; i < MAX_LLAMADAS; i++) if (llamadas[i].estado == LIBRE) return i;
  return -1;
}
void liberar(int i, const char *motivo) {
  Serial.printf("Canal %u liberado (%u <-> %u): %s\n", llamadas[i].canal,
                llamadas[i].origen, llamadas[i].destino, motivo);
  llamadas[i].estado = LIBRE;
}

// ------------------------------------------------------------------------------
// MONITOREO DE CANALES
// testRPD() dice si en el canal hay una senal mas fuerte que -64 dBm.
// Se toman varias muestras y se cuenta en cuantas hubo senal.
uint8_t medirCanal(uint8_t c, uint8_t muestras) {
  uint8_t conSenal = 0;
  radio.stopListening();
  radio.setChannel(c);
  for (uint8_t i = 0; i < muestras; i++) {
    radio.startListening();
    delayMicroseconds(170);
    if (radio.testRPD()) conSenal++;
    radio.stopListening();
  }
  radio.setChannel(CANAL_CONTROL);
  radio.startListening();
  return conSenal * 100 / muestras;
}

// Revisa un canal a la vez, en ciclo, para no descuidar el canal de control.
void monitorearCanales() {
  if (millis() - tMonitoreo < MONITOREO_CADA_MS) return;
  tMonitoreo = millis();

  // Saltar el canal de control y los que estan en uso por nuestras llamadas
  // (ahi la "senal" son nuestros propios nodos, no interferencia).
  uint8_t c = canalMonitoreo;
  canalMonitoreo = (canalMonitoreo >= CANAL_MAX) ? CANAL_MIN : canalMonitoreo + 1;
  if (c == CANAL_CONTROL || llamadaEnCanal(c) >= 0) return;

  uint8_t medida = medirCanal(c, MUESTRAS_MONITOR);
  ruido[c] = (ruido[c] * 3 + medida) / 4;      // promedio movil: suaviza picos aislados
}

// Que tan lejos esta el canal c del canal ocupado mas cercano (control o llamadas)
int distanciaAOcupados(int c) {
  int d = abs(c - CANAL_CONTROL);
  for (int i = 0; i < MAX_LLAMADAS; i++)
    if (llamadas[i].estado != LIBRE) d = min(d, abs(c - (int)llamadas[i].canal));
  return d;
}

// Elige el mejor canal libre:
//  1) primero solo entre los que NO tienen interferencia;
//  2) entre ellos, el mas alejado de los canales ocupados (hasta 6 MHz de separacion);
//  3) a igual distancia, el de menos ruido.
// Si todos los libres tienen interferencia, usa el menos ruidoso.
int elegirCanal() {
  for (int pasada = 0; pasada < 2; pasada++) {
    int mejor = -1, mejorDist = -1, mejorRuido = 999;
```

```
    for (int c = CANAL_MIN; c <= CANAL_MAX; c++) {
      if (c == CANAL_CONTROL || llamadaEnCanal(c) >= 0) continue;
      if (pasada == 0 && ruido[c] > UMBRAL_RUIDO) continue;
      int dist = min(distanciaAOcupados(c), 6);
      if (dist > mejorDist || (dist == mejorDist && ruido[c] < mejorRuido)) {
        mejor = c; mejorDist = dist; mejorRuido = ruido[c];
      }
    }
    if (mejor >= 0) return mejor;
  }
  return -1;
}

// Elige un canal y lo mide en ese momento antes de asignarlo.
int elegirCanalVerificado() {
  for (int intento = 0; intento < 5; intento++) {
    int c = elegirCanal();
    if (c < 0) return -1;
    uint8_t medida = medirCanal(c, MUESTRAS_VERIFICA);
    ruido[c] = max(ruido[c], medida);
    if (medida <= UMBRAL_RUIDO) return c;
    Serial.printf("  Canal %d tiene interferencia (%u%%), buscando otro...\n", c, medida);
  }
  return elegirCanal();       // si todo esta ruidoso, el mejor que haya
}

// ---------------------------------------------------------------------------------
void atenderLlamar(uint8_t a, uint8_t b) {
  Serial.printf("Nodo %u quiere llamar al nodo %u\n", a, b);
  if (b == a || b == ID_CENTRAL || !nodoEnLinea(b)) {
    enviar(a, RESPUESTA, 0, R_NO_DISPONIBLE); return;
  }
  if (llamadaDe(a) >= 0 || llamadaDe(b) >= 0) {
    enviar(a, RESPUESTA, 0, R_OCUPADO); return;
  }
  int i = espacioLlamada();
  int canal = (i >= 0) ? elegirCanalVerificado() : -1;
  if (canal < 0) { enviar(a, RESPUESTA, 0, R_SIN_CANALES); return; }

  llamadas[i] = Llamada{TIMBRANDO, a, b, (uint8_t)canal, millis()};
  if (!enviar(b, ENTRANTE, 0, a)) {
    liberar(i, "el destino no respondio");
    enviar(a, RESPUESTA, 0, R_NO_DISPONIBLE);
    return;
  }
  enviar(a, RESPUESTA, 0, R_TIMBRANDO);
  Serial.printf("  Timbrando en nodo %u (canal %d reservado, ruido %u%%)\n",
                b, canal, ruido[canal]);
}

void atenderContestar(uint8_t b) {
  int i = llamadaDe(b);
  if (i < 0 || llamadas[i].estado != TIMBRANDO || llamadas[i].destino != b) return;
  uint8_t a = llamadas[i].origen, canal = llamadas[i].canal;
  llamadas[i].estado = EN_CURSO;
  llamadas[i].desde = millis();
  enviar(b, CONECTAR, canal, a);       // mandar a los dos a su propio canal
  enviar(a, CONECTAR, canal, b);
  Serial.printf("CONECTADOS %u <-> %u en canal %u (%u MHz)\n", a, b, canal, 2400 + canal);
}

void atenderRechazar(uint8_t b) {
  int i = llamadaDe(b);
  if (i < 0 || llamadas[i].estado != TIMBRANDO) return;
  enviar(llamadas[i].origen, RESPUESTA, 0, R_RECHAZADO);
  liberar(i, "rechazada");
}

void atenderLiberar(uint8_t x) {
  int i = llamadaDe(x);
  if (i < 0) return;
  if (llamadas[i].estado == TIMBRANDO) {
```

```
    uint8_t otro = (llamadas[i].origen == x) ? llamadas[i].destino : llamadas[i].origen;
    enviar(otro, RESPUESTA, 0, R_CANCELADO);
    liberar(i, "cancelada antes de contestar");
  } else {
    liberar(i, "colgaron");
  }
}

void revisarTiempos() {
  bool alguna = false;
  for (int i = 0; i < MAX_LLAMADAS; i++) {
    if (llamadas[i].estado == TIMBRANDO && millis() - llamadas[i].desde > TIMBRE_MAX_MS) {
      enviar(llamadas[i].origen, RESPUESTA, 0, R_NO_CONTESTA);
      enviar(llamadas[i].destino, RESPUESTA, 0, R_CANCELADO);
      liberar(i, "no contesto");
    }
    if (llamadas[i].estado == EN_CURSO) alguna = true;
  }
  static int ledAnterior = -1;          // solo actualizar el LED cuando cambia
  if (alguna != ledAnterior) { ledAnterior = alguna; ledModo(alguna ? 2 : 1); }
}

// ---------------------------------------------------------------------------
// MENSAJES DE LA CENTRAL
// Entrega un texto a un nodo, en el canal donde este (control o su llamada).
// Los textos largos se parten en pedazos de 25 caracteres.
bool entregarAviso(uint8_t id, const String &texto) {
  int l = llamadaDe(id);
  uint8_t canalRF = (l >= 0 && llamadas[l].estado == EN_CURSO) ? llamadas[l].canal : CANAL_CONTROL;
  bool ok = true;
  for (unsigned int i = 0; i < texto.length(); i += 25) {
    String trozo = texto.substring(i, i + 25);
    ok = enviarEnCanal(canalRF, id, AVISO, 0, 0, trozo.c_str()) && ok;
  }
  return ok;
}

void avisoATodos(const String &texto) {
  int entregados = 0, total = 0;
  for (int i = 0; i < numNodos; i++) {
    uint8_t id = nodos[i].id;
    if (!nodoEnLinea(id) && llamadaDe(id) < 0) continue;   // desconectado
    total++;
    bool ok = entregarAviso(id, texto);
    if (ok) entregados++;
    int l = llamadaDe(id);
    Serial.printf("  -> nodo %3u %s%s\n", id, ok ? "entregado" : "NO RESPONDIO",
                  (l >= 0 && llamadas[l].estado == EN_CURSO) ? " (en su canal de llamada)" : "");
  }
  if (total == 0) Serial.println("No hay nodos conectados.");
  else Serial.printf("Mensaje entregado a %d de %d nodos.\n", entregados, total);
}

// ---------------------------------------------------------------------------
void mostrarTabla() {
  Serial.println("--- Nodos ---");
  if (numNodos == 0) Serial.println("  (ninguno)");
  for (int i = 0; i < numNodos; i++) {
    uint8_t id = nodos[i].id;
    int l = llamadaDe(id);
    if (l >= 0) {
      uint8_t otro = llamadas[l].origen == id ? llamadas[l].destino : llamadas[l].origen;
      Serial.printf("  Nodo %3u : %s con %u (canal %u)\n", id,
                    llamadas[l].estado == TIMBRANDO ? "timbrando" : "en llamada",
                    otro, llamadas[l].canal);
    } else {
      Serial.printf("  Nodo %3u : %s\n", id, nodoEnLinea(id) ? "libre" : "desconectado");
    }
  }
}

void mostrarCanales() {
```

```
  Serial.printf("--- Canales %u a %u (%u a %u MHz) ---\n", CANAL_MIN, CANAL_MAX,
                2400 + CANAL_MIN, 2400 + CANAL_MAX);
  // Regla con las decenas
  Serial.print("  ");
  for (int c = CANAL_MIN; c <= CANAL_MAX; c++) Serial.print(c % 10 == 0 ? (char)('0' + (c / 10) % 10) : ' ');
  Serial.println();
  // Mapa: un caracter por canal
  int libres = 0, conRuido = 0, enUso = 0;
  Serial.print("  ");
  for (int c = CANAL_MIN; c <= CANAL_MAX; c++) {
    char s;
    if (c == CANAL_CONTROL)              s = 'C';
    else if (llamadaEnCanal(c) >= 0)   { s = 'L'; enUso++; }
    else if (ruido[c] > UMBRAL_RUIDO)  { s = 'x'; conRuido++; }
    else                               { s = '.'; libres++; }
    Serial.print(s);
  }
  Serial.println();
  Serial.println("  .=libre  L=llamada  C=control  x=interferencia (decenas arriba: 0=0-9, 1=10-19...)");
  Serial.printf("  Libres: %d   En llamada: %d   Con interferencia: %d\n", libres, enUso, conRuido);

  bool alguna = false;
  for (int i = 0; i < MAX_LLAMADAS; i++) {
    if (llamadas[i].estado == LIBRE) continue;
    if (!alguna) { Serial.println("  Llamadas:"); alguna = true; }
    Serial.printf("    canal %2u (%u MHz): %u -> %u  %s  %lu s\n", llamadas[i].canal,
                  2400 + llamadas[i].canal, llamadas[i].origen, llamadas[i].destino,
                  llamadas[i].estado == TIMBRANDO ? "timbrando" : "EN CURSO",
                  (millis() - llamadas[i].desde) / 1000);
  }
}

void mostrarRuido() {
  Serial.println("--- Interferencia medida por canal (promedio) ---");
  bool alguno = false;
  for (int c = CANAL_MIN; c <= CANAL_MAX; c++) {
    if (ruido[c] == 0 || c == CANAL_CONTROL) continue;
    alguno = true;
    Serial.printf("  canal %2d (%u MHz): %3u%% ", c, 2400 + c, ruido[c]);
    for (int k = 0; k < ruido[c] / 4; k++) Serial.print('#');
    Serial.println(ruido[c] > UMBRAL_RUIDO ? "  <- se evita" : "");
  }
  if (!alguno) Serial.println("  Ningun canal con interferencia detectada.");
}

void mostrarAyuda() {
  Serial.println("Comandos: /todos texto | /msg ID texto | /tabla | /canales | /ruido | /ayuda");
}

void procesarLinea(String l) {
  l.trim();
  if (l.length() == 0) return;
  if (l.startsWith("/todos ")) {
    String t = l.substring(7); t.trim();
    Serial.printf("Mensaje a todos: \"%s\"\n", t.c_str());
    avisoATodos(t);
  }
  else if (l.startsWith("/msg ")) {
    String resto = l.substring(5); resto.trim();
    int esp = resto.indexOf(' ');
    int id = resto.substring(0, esp).toInt();
    if (esp < 0 || id < 1 || id > 254) { Serial.println("Uso: /msg ID texto"); return; }
    bool ok = entregarAviso(id, resto.substring(esp + 1));
    Serial.printf("Mensaje al nodo %d: %s\n", id, ok ? "entregado" : "NO RESPONDIO");
  }
  else if (l == "/tabla")   mostrarTabla();
  else if (l == "/canales") mostrarCanales();
  else if (l == "/ruido")   mostrarRuido();
  else mostrarAyuda();
}

// ------------------------------------------------------------------------------
```

```
void setup() {
  Serial.begin(115200);
  delay(1500);
#ifdef PIN_LED
  pinMode(PIN_LED, OUTPUT);
#endif
  ledModo(0);

  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CSN);
  if (!radio.begin(&SPI)) {
    Serial.println("ERROR: el nRF24L01 no responde. Revisa cables y alimentacion.");
    while (true) { ledModo(3); delay(150); ledModo(0); delay(150); }
  }
  radio.setDataRate(RF24_1MBPS);
  radio.setPALevel(RF24_PA_LOW);
  radio.setPayloadSize(sizeof(Paquete));
  radio.setAutoAck(true);
  radio.setRetries(5, 15);
  uint8_t dir[5]; direccionDe(ID_CENTRAL, dir);
  radio.openReadingPipe(1, dir);

  // Barrido inicial de todos los canales para arrancar ya con el mapa de interferencia
  Serial.println("\nMidiendo interferencia en todos los canales...");
  for (int c = CANAL_MIN; c <= CANAL_MAX; c++)
    ruido[c] = (c == CANAL_CONTROL) ? 0 : medirCanal(c, MUESTRAS_VERIFICA);

  radio.setChannel(CANAL_CONTROL);
  radio.startListening();
  ledModo(1);

  Serial.printf("=== CENTRAL TELEFONICA v2 lista (%s) ===\n", PLACA);
  mostrarCanales();
  mostrarAyuda();
}

void loop() {
  // 1) Comandos del Monitor Serial
  static String linea = "";
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') { procesarLinea(linea); linea = ""; }
    else if (linea.length() < 120) linea += c;
  }

  // 2) Mensajes de los nodos
  if (radio.available()) {
    Paquete p;
    radio.read(&p, sizeof(p));
    registrarNodo(p.origen);
    switch (p.tipo) {
      case REGISTRO: {
        // Nodo que dice estar libre pero la central lo tiene en llamada = se reinicio
        int i = llamadaDe(p.origen);
        if (i >= 0 && llamadas[i].estado == EN_CURSO && millis() - llamadas[i].desde > 3000)
          liberar(i, "el nodo regreso sin colgar");
        break;
      }
      case LLAMAR:    atenderLlamar(p.origen, p.dato); break;
      case CONTESTAR: atenderContestar(p.origen);      break;
      case RECHAZAR:  atenderRechazar(p.origen);       break;
      case LIBERAR:   atenderLiberar(p.origen);        break;
    }
  }

  // 3) Tiempos de timbrado y LED
  revisarTiempos();

  // 4) Monitoreo continuo de interferencia (un canal cada 40 ms)
  monitorearCanales();
}
```

A.9. Teléfono con audio ADPCM (Nodo_Telefono.ino, rama main)

```
/*
  NODO TELEFONO - Topologia estrella con central (nRF24L01+ PA+LNA + ESP32)
  Version para la interfaz HTML (Web Serial, "Audio ADPCM binario", 460800 baudios).

  La PC manda la voz ya comprimida (ADPCM 4 bits, 8 kHz) en tramas por el puerto serie:
      0xFE 0xFF + 29 bytes   (1 byte indice + 2 bytes predictor + 26 bytes de 52 muestras)
  El ESP32 solo la reenvia por radio al otro nodo y hace lo mismo en sentido contrario.
  Cada paquete de voz mide 32 bytes: [VOZ][origen][secuencia][29 bytes de audio]
*/
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define MI_ID 1         // <-- distinto en cada nodo: 1, 2, 3 ...

#if defined(CONFIG_IDF_TARGET_ESP32C3)
  #define PIN_CE    3
  #define PIN_CSN   7
  #define PIN_SCK   4
  #define PIN_MISO  5
  #define PIN_MOSI  6
  #define PIN_LED   8
  #define PIN_BOOT  9
  #define LED_ON    LOW
  #define PLACA     "ESP32-C3 SuperMini"
#elif defined(CONFIG_IDF_TARGET_ESP32)
  #define PIN_CE    2
  #define PIN_CSN   4
  #define PIN_SCK   18
  #define PIN_MISO  19
  #define PIN_MOSI  23
  #define PIN_LED   12
  #define PIN_BOOT  0
  #define LED_ON    HIGH
  #define PLACA     "ESP32 DevKit"
#else
  #error "Placa no soportada"
#endif
#define LED_OFF (LED_ON == HIGH ? LOW : HIGH)

#define VELOCIDAD_SERIE 460800   // debe coincidir con la velocidad elegida en el HTML

#define ID_CENTRAL     0
#define CANAL_CONTROL  80
#define CUERPO         29        // bytes de audio por paquete

enum Tipo : uint8_t {
  REGISTRO = 1, LLAMAR, CONTESTAR, RECHAZAR, LIBERAR,
  ENTRANTE, CONECTAR, RESPUESTA, TEXTO, KEEPALIVE, COLGAR,
  AVISO, VOZ
};
enum Codigo : uint8_t {
  R_TIMBRANDO = 1, R_OCUPADO, R_NO_DISPONIBLE, R_RECHAZADO,
  R_NO_CONTESTA, R_SIN_CANALES, R_CANCELADO
};
struct Paquete {
  uint8_t tipo;
  uint8_t origen;
  uint8_t destino;
  uint8_t canal;
  uint8_t dato;
  uint8_t secuencia;
  char    texto[26];
};
static_assert(sizeof(Paquete) == 32, "El paquete debe medir 32 bytes");

enum Estado { REPOSO, LLAMANDO, TIMBRANDO, EN_LLAMADA };
```

```
void direccionDe(uint8_t id, uint8_t *dir) {
  dir[0] = id; dir[1] = 'N'; dir[2] = 'O'; dir[3] = 'D'; dir[4] = 'E';
}

RF24 radio(PIN_CE, PIN_CSN);
uint8_t secuencia = 0;

Estado estado = REPOSO;
uint8_t pareja = 0;
uint8_t canalActual = CANAL_CONTROL;
unsigned long tEstado = 0;
unsigned long tUltimoEnvio = 0;
unsigned long tRegistro = 0;
bool centralPerdida = false;

bool audioPC = false;            // la PC pidio recibir audio (/audio 1)
uint8_t nivelRemoto = 0;         // solo para el brillo del LED

#define REGISTRO_CADA_MS   5000UL

const char *nombreEstado[] = {"REPOSO", "LLAMANDO", "TIMBRANDO", "EN LLAMADA"};

bool enviar(uint8_t destino, uint8_t tipo, uint8_t dato, const char *texto) {
  Paquete p = {};
  p.tipo = tipo; p.origen = MI_ID; p.destino = destino;
  p.canal = canalActual; p.dato = dato; p.secuencia = secuencia++;
  if (texto) strncpy(p.texto, texto, sizeof(p.texto) - 1);
  uint8_t dir[5]; direccionDe(destino, dir);
  radio.stopListening();
  radio.openWritingPipe(dir);
  bool ok = radio.write(&p, sizeof(p));
  radio.startListening();
  tUltimoEnvio = millis();
  return ok;
}

// Voz hacia el otro nodo: sin ACK, si se pierde un paquete no se reintenta
// (cada paquete lleva su propio estado ADPCM, asi que el decodificador se recupera solo).
void enviarVoz(const uint8_t *cuerpo) {
  uint8_t b[32];
  b[0] = VOZ; b[1] = MI_ID; b[2] = secuencia++;
  memcpy(b + 3, cuerpo, CUERPO);
  uint8_t dir[5]; direccionDe(pareja, dir);
  radio.stopListening();
  radio.openWritingPipe(dir);
  radio.write(b, 32, true);
  radio.startListening();
}

// Voz hacia la PC: trama binaria 0xFE 0xFF + 29 bytes, escrita de una sola vez
void vozAlPC(const uint8_t *cuerpo) {
  uint8_t t[2 + CUERPO];
  t[0] = 0xFE; t[1] = 0xFF;
  memcpy(t + 2, cuerpo, CUERPO);
  Serial.write(t, sizeof(t));
}

void cambiarCanal(uint8_t c) {
  radio.stopListening();
  radio.setChannel(c);
  radio.startListening();
  canalActual = c;
}

void cambiarEstado(Estado e) {
  estado = e;
  tEstado = millis();
  nivelRemoto = 0;
}

void ledBrillo(uint8_t v) {
  analogWrite(PIN_LED, LED_ON == HIGH ? v : 255 - v);
```

```
}

uint8_t gamma8(uint8_t v) { return (uint16_t)v * v / 255; }

void volverAReposo(bool avisarCentral) {
  if (canalActual != CANAL_CONTROL) cambiarCanal(CANAL_CONTROL);
  if (avisarCentral) enviar(ID_CENTRAL, LIBERAR, 0, nullptr);
  pareja = 0;
  cambiarEstado(REPOSO);
  enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
  tRegistro = millis();
  Serial.println("En reposo.");
}

void procesarLinea(String l) {
  l.trim();
  if (l.length() == 0) return;

  if (l.startsWith("/audio")) {
    audioPC = l.substring(6).toInt() != 0;
    return;
  }

  if (l.startsWith("/llamar")) {
    if (estado != REPOSO) { Serial.println("[ERR] Ocupado"); return; }
    int destino = l.substring(7).toInt();
    if (destino < 1 || destino > 254 || destino == MI_ID) return;
    if (!enviar(ID_CENTRAL, LLAMAR, destino, nullptr)) {
      Serial.println("[ERR] La central no responde, no se pudo llamar");
      return;
    }
    pareja = destino;
    cambiarEstado(LLAMANDO);
    Serial.printf("Llamando al nodo %u\n", pareja);
  }
  else if (l == "/contestar") {
    if (estado != TIMBRANDO) return;
    enviar(ID_CENTRAL, CONTESTAR, 0, nullptr);
  }
  else if (l == "/rechazar") {
    if (estado != TIMBRANDO) return;
    enviar(ID_CENTRAL, RECHAZAR, 0, nullptr);
    volverAReposo(false);
  }
  else if (l == "/colgar") {
    if (estado == EN_LLAMADA) {
      enviar(pareja, COLGAR, 0, nullptr);
      volverAReposo(true);
    } else if (estado == LLAMANDO || estado == TIMBRANDO) {
      volverAReposo(true);
    }
  }
  else if (l == "/estado") {
    if (pareja) Serial.printf("Nodo %u | %s | canal %u | con nodo %u\n", MI_ID, nombreEstado[estado], canalActual,
pareja);
    else        Serial.printf("Nodo %u | %s | canal %u |\n", MI_ID, nombreEstado[estado], canalActual);
  }
  else {
    if (estado != EN_LLAMADA) return;
    String t = l.substring(0, 25);
    enviar(pareja, TEXTO, 0, t.c_str());
    Serial.printf("[YO] %s\n", t.c_str());
  }
}

void procesarPaquete(Paquete &p) {
  p.texto[sizeof(p.texto) - 1] = '\0';

  if (p.origen == ID_CENTRAL) {
    centralPerdida = false;
    switch (p.tipo) {
      case AVISO:
```

```
        Serial.printf("[CENTRAL] %s\n", p.texto);
        break;
      case ENTRANTE:
        if (estado != REPOSO) break;
        pareja = p.dato;
        cambiarEstado(TIMBRANDO);
        Serial.printf("El nodo %u te llama\n", pareja);
        break;
      case CONECTAR:
        if (estado != LLAMANDO && estado != TIMBRANDO) break;
        pareja = p.dato;
        cambiarCanal(p.canal);
        cambiarEstado(EN_LLAMADA);
        Serial.printf("CONECTADO con nodo %u (canal %u)\n", pareja, p.canal);
        break;
      case RESPUESTA:
        if (p.dato != R_TIMBRANDO) { volverAReposo(false); }
        break;
    }
    return;
  }

  if (estado != EN_LLAMADA || p.origen != pareja) return;
  switch (p.tipo) {
    case TEXTO:
      Serial.printf("[MSG %u] %s\n", p.origen, p.texto);
      break;
    case COLGAR:
      volverAReposo(true);
      break;
  }
}

// Lee del puerto serie: lineas de texto (comandos) y tramas de audio 0xFE 0xFF + 29 bytes
void leerSerie() {
  static uint8_t fase = 0, pos = 0;
  static uint8_t cuerpo[CUERPO];
  static String linea = "";
  while (Serial.available()) {
    uint8_t c = Serial.read();
    if (fase == 2) {
      cuerpo[pos++] = c;
      if (pos == CUERPO) {
        fase = 0;
        if (estado == EN_LLAMADA) enviarVoz(cuerpo);
      }
      continue;
    }
    if (fase == 1) {
      fase = 0;
      if (c == 0xFF) { fase = 2; pos = 0; continue; }
    }
    if (c == 0xFE) fase = 1;
    else if (c == '\n' || c == '\r') { procesarLinea(linea); linea = ""; }
    else if (linea.length() < 100) linea += (char)c;
  }
}

void setup() {
  Serial.setRxBufferSize(2048);     // la PC manda las tramas de voz en rafagas
  Serial.begin(VELOCIDAD_SERIE);
  delay(1500);
  pinMode(PIN_LED, OUTPUT);
  ledBrillo(0);
  pinMode(PIN_BOOT, INPUT_PULLUP);

  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CSN);
  if (!radio.begin(&SPI)) {
    while (true) { ledBrillo(255); delay(100); ledBrillo(0); delay(100); }
  }
  radio.setChannel(CANAL_CONTROL);
  radio.setDataRate(RF24_1MBPS);
```

```
  radio.setPALevel(RF24_PA_LOW);
  radio.setPayloadSize(sizeof(Paquete));
  radio.setAutoAck(true);
  radio.enableDynamicAck();         // permite mandar la voz sin ACK
  radio.setRetries(3 + (MI_ID % 10), 15);
  uint8_t dir[5]; direccionDe(MI_ID, dir);
  radio.openReadingPipe(1, dir);
  radio.startListening();

  tRegistro = millis() - REGISTRO_CADA_MS;
  Serial.printf("Nodo telefono %u listo (%s)\n", MI_ID, PLACA);
}

void loop() {
  leerSerie();

  while (radio.available()) {
    uint8_t buf[32];
    radio.read(buf, sizeof(buf));
    if (buf[0] == VOZ) {
      if (estado == EN_LLAMADA && buf[1] == pareja) {
        nivelRemoto = constrain(buf[3] * 3, 0, 255);   // el indice ADPCM sirve de medidor de volumen
        if (audioPC) vozAlPC(buf + 3);
      }
    } else {
      Paquete p;
      memcpy(&p, buf, sizeof(p));
      procesarPaquete(p);
    }
  }

  // Registro periodico en la central (solo en reposo, cuando estamos en el canal de control)
  unsigned long ahora = millis();
  if (estado == REPOSO && ahora - tRegistro >= REGISTRO_CADA_MS) {
    tRegistro = ahora;
    bool ok = enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
    if (ok) centralPerdida = false;
    else if (!centralPerdida) {
      centralPerdida = true;
      Serial.println("[ERR] La central no responde");
    }
  }

  // LED indicador
  uint8_t brillo = 0;
  if (estado == TIMBRANDO)  brillo = ((ahora / 150) % 2) ? 255 : 0;
  if (estado == LLAMANDO)   brillo = ((ahora / 600) % 2) ? 255 : 0;
  if (estado == EN_LLAMADA) brillo = max((uint8_t)6, gamma8(nivelRemoto));
  ledBrillo(brillo);
}
```

A.10. Interfaz web (NodoTelefonoInterfaz.html)

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>OCP · Teléfono nRF24</title>
<style>
:root{--steel:#9aa5af;--steel-l:#e3e8ec;--steel-d:#4b555e;--bar1:#10294a;--bar2:#2f65a0;--g:#5dff8a;--a:#ffb000;--
r:#ff5a4d;--c:#58d6ff;color-scheme:dark}
*{box-sizing:border-box}
body{margin:0;min-height:100vh;background:#030605 radial-gradient(ellipse at center,#0f1f18 0%,#030605 78%);color:var(-
-g);
  font:15px/1.5 "Lucida Console","Courier New",monospace;text-shadow:0 0 6px
rgba(93,255,138,.4);padding:clamp(12px,3vw,32px);--col:#7d8a95}
body::after{content:"";position:fixed;inset:0;pointer-events:none;z-index:10;
  background:repeating-linear-gradient(0deg,rgba(0,0,0,.26) 0,rgba(0,0,0,.26) 1px,transparent 1px,transparent 3px)}
body[data-s=REPOSO]{--col:var(--g)}
```

```
body[data-s=LLAMANDO],body[data-s=TIMBRANDO]{--col:var(--a)}
body[data-s="EN LLAMADA"]{--col:var(--r)}
.wrap{max-width:980px;margin:auto;display:grid;gap:16px}
header{display:flex;flex-wrap:wrap;gap:12px 24px;align-items:end;justify-content:space-between;border-bottom:2px solid
var(--steel-d);padding-bottom:14px}
h1{margin:0;font:700 clamp(22px,4.5vw,38px)/1.1 "Lucida Console","Courier New",monospace;text-
transform:uppercase;letter-spacing:.06em;
  color:var(--steel-l);text-shadow:0 0 10px rgba(88,214,255,.55),2px 2px 0 #1b2a38}
.ocp{display:inline-block;padding:0 8px;margin-right:8px;border:3px solid var(--steel-l);letter-spacing:.12em}
.sub{color:var(--c);margin:6px 0 0;text-shadow:0 0 6px rgba(88,214,255,.4)}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(310px,1fr));gap:16px}
.panel{background:#09110d;border:2px solid;border-color:var(--steel-l) var(--steel-d) var(--steel-d) var(--steel-
l);outline:2px solid #000;padding:0 18px 18px}
h2{margin:0 -18px 16px;padding:4px 10px;font:700 13px/1.6 "Lucida Console","Courier New",monospace;text-
transform:uppercase;letter-spacing:.14em;
  color:#fff;text-shadow:none;background:linear-gradient(90deg,var(--bar1),var(--bar2))}
button,input,select{font:inherit}
button{background:var(--steel);color:#000;text-shadow:none;border:2px solid;border-color:#fff var(--steel-d) var(--
steel-d) #fff;padding:7px 14px;cursor:pointer;
  font-weight:700;text-transform:uppercase;font-size:13px;letter-spacing:.05em}
button:active:not(:disabled){border-color:var(--steel-d) #fff #fff var(--steel-d);transform:translate(1px,1px)}
button.sec{background:#5c6670;color:#fff}
button.ok{background:#35b560} button.bad{background:#c62d24;color:#fff}
button:disabled{opacity:.4;cursor:not-allowed}
button:focus-visible,input:focus-visible,select:focus-visible{outline:2px dashed var(--a);outline-offset:2px}
input,select{background:#000;color:var(--g);border:2px solid;border-color:var(--steel-d) var(--steel-l) var(--steel-l)
var(--steel-d);padding:6px 8px;text-shadow:inherit}
select:disabled{opacity:.5}
input[type=range]{padding:0;accent-color:var(--g)}
.row{display:flex;flex-wrap:wrap;gap:10px;align-items:center}
.board{display:flex;gap:18px;align-items:center;margin-bottom:16px}
.lamp{position:relative;flex:none;width:76px;height:76px;border-radius:50%;border:2px solid var(--col);box-shadow:0 0
12px var(--col);
  background:linear-gradient(var(--col),var(--col)) 50% 50%/2px 100% no-repeat,linear-gradient(var(--col),var(--col))
50% 50%/100% 2px no-repeat}
.lamp::after{content:"";position:absolute;inset:25px;border:2px solid var(--col);border-radius:50%;background:#09110d}
[data-s=LLAMANDO] .lamp,[data-s=TIMBRANDO] .lamp{animation:par .6s steps(2) infinite}
[data-s="EN LLAMADA"] .lamp{box-shadow:0 0 24px var(--col),inset 0 0 14px var(--col)}
@keyframes par{50%{opacity:.25}}
@media (prefers-reduced-motion:reduce){.lamp{animation:none!important}}
.big{font:700 clamp(22px,4.5vw,32px)/1.1 "Lucida Console","Courier New",monospace;text-transform:uppercase;color:var(--
col);text-shadow:0 0 10px var(--col)}
.who{color:var(--c);margin-top:6px;text-shadow:0 0 6px rgba(88,214,255,.4)}
label{display:block;color:var(--c);text-shadow:none}
.meter{position:relative;height:16px;background:#000;border:2px solid;border-color:var(--steel-d) var(--steel-l) var(--
steel-l) var(--steel-d);margin:4px 0 12px;overflow:hidden}
.meter i{display:block;height:100%;width:0;background:linear-gradient(90deg,var(--g) 65%,var(--a) 85%,var(--r))}
.meter::after{content:"";position:absolute;inset:0;background:repeating-linear-gradient(90deg,transparent 0,transparent
8px,#000 8px,#000 10px)}
#log{height:210px;overflow:auto;background:#000;color:var(--g);border:2px solid;border-color:var(--steel-d) var(--
steel-l) var(--steel-l) var(--steel-d);padding:10px;font:13px/1.5 "Lucida Console","Courier New",monospace}
#log .me{color:var(--a);text-shadow:0 0 6px rgba(255,176,0,.5)} #log .err{color:var(--r);text-shadow:0 0 6px
rgba(255,90,77,.5)}
#aviso{display:none;background:#2a0a08;border:2px solid var(--r);color:#ffb3ab;padding:10px 14px}
.stats{color:var(--c);font-variant-numeric:tabular-nums;text-shadow:none}
.dir{border-top:2px dashed var(--steel-d);padding-top:12px;color:var(--a);font-size:12px;letter-spacing:.08em;text-
transform:uppercase;text-shadow:0 0 6px rgba(255,176,0,.4)}
</style>
</head>
<body data-s="DESCONECTADO">
<div class="wrap">
  <header>
    <div>
      <h1><span class="ocp">OCP</span>Teléfono nRF24</h1>
      <p class="sub">Terminal de comunicaciones tácticas · habla y escucha desde la PC.</p>
    </div>
    <div class="row">
      <label for="fw" style="display:inline">Firmware</label>
      <select id="fw">
        <option value="b64">Audio base64 (sdl)</option>
        <option value="adpcm" selected>Audio ADPCM binario</option>
      </select>
```

```
      <label for="baud" style="display:inline">Velocidad</label>
      <select id="baud"><option>115200</option><option>230400</option><option
selected>460800</option><option>500000</option><option>921600</option></select>
      <button id="bCon">Conectar ESP32</button>
    </div>
  </header>

  <div id="aviso">Este navegador no soporta Web Serial. Abre la página en Chrome o Edge de escritorio.</div>

  <div class="grid">
    <section class="panel">
      <h2>Llamada</h2>
      <div class="board">
        <div class="lamp"></div>
        <div><div class="big" id="estado">DESCONECTADO</div>
        <div class="who"><span id="nodo">Sin nodo</span> · con <span id="pareja">—</span></div></div>
      </div>
      <div class="row" style="margin-bottom:12px">
        <input id="dest" type="number" min="1" max="254" value="2" style="width:84px" aria-label="Número de nodo a
llamar">
        <button id="bLlamar" disabled>Llamar</button>
        <button id="bCol" class="bad" disabled>Colgar</button>
      </div>
      <div class="row">
        <button id="bCont" class="ok" disabled>Contestar</button>
        <button id="bRech" class="sec" disabled>Rechazar</button>
      </div>
    </section>

    <section class="panel">
      <h2>Audio</h2>
      <label>Tu voz (sale por radio)</label>
      <div class="meter"><i id="mL"></i></div>
      <label>Voz del otro (llega por radio)</label>
      <div class="meter"><i id="mR"></i></div>
      <div class="row" style="margin-bottom:12px">
        <button id="bMute" class="sec">Silenciar micrófono</button>
        <label for="vol" style="display:inline">Volumen</label>
        <input id="vol" type="range" min="0" max="2" step="0.05" value="1" style="width:120px">
      </div>
      <div class="stats" id="stats">Sin datos todavía.</div>
    </section>
  </div>

  <section class="panel">
    <h2>Consola del nodo</h2>
    <div id="log" role="log"></div>
    <form id="fCmd" class="row" style="margin-top:10px">
      <input id="cmd" style="flex:1;min-width:200px" placeholder="/estado, /llamar 2, texto para la otra persona…"
autocomplete="off" aria-label="Comando">
      <button class="sec">Enviar</button>
    </form>
  </section>
  <div class="dir">Directivas prioritarias: 1. Servir al público · 2. Proteger al inocente · 3. Hacer cumplir la ley ·
4. [Clasificado]</div>
</div>

<script>
const $=s=>document.querySelector(s), te=new TextEncoder(), td=new TextDecoder();
const esperar=ms=>new Promise(r=>setTimeout(r,ms));
const
STEP=[7,8,9,10,11,12,13,14,16,17,19,21,23,25,28,31,34,37,41,45,50,55,60,66,73,80,88,97,107,118,130,143,157,173,190,209,
230,253,279,307,337,371,408,449,494,544,598,658,724,796,876,963,1060,1166,1282,1411,1552,1707,1878,2066,2272,2499,2749,
3024,3327,3660,4026,4428,4871,5358,5894,6484,7132,7845,8630,9493,10442,11487,12635,13899,15289,16818,18500,20350,22385,
24623,27086,29794,32767];
const IDX=[-1,-1,-1,-1,2,4,6,8,-1,-1,-1,-1,2,4,6,8];
const N=52, CUERPO=29, B64N=26, clamp=(v,a,b)=>Math.max(a,Math.min(b,v));
let port,writer,reader,leyendo,ctx,cap,play,gain,mic,estado='DESCONECTADO',silencio=false;
let activo=false,cerrando=false,avisado=false;
let tx=0,rx=0,lvlL=0,lvlR=0,bufMs=0,ringT;
const modo=()=>$('#fw').value;          // 'b64' (líneas AUDIO:/A:) o 'adpcm' (tramas binarias 0xFE 0xFF)
```

```
/* ---------- ADPCM IMA 4 bits: cada paquete lleva su estado inicial (índice + predictor) ---------- */
const E={p:0,i:0}, pcm=new Int16Array(N); let nPcm=0;
function codificar(s){
  const o=new Uint8Array(CUERPO); o[0]=E.i; o[1]=E.p&255; o[2]=(E.p>>8)&255;
  for(let n=0;n<N;n++){
    let d=s[n]-E.p,sg=0; if(d<0){sg=8;d=-d}
    let st=STEP[E.i],c=0,v=st>>3;
    if(d>=st){c|=4;d-=st;v+=st} st>>=1;
    if(d>=st){c|=2;d-=st;v+=st} st>>=1;
    if(d>=st){c|=1;v+=st}
    E.p=clamp(E.p+(sg?-v:v),-32768,32767);
    E.i=clamp(E.i+IDX[c|sg],0,88);
    c|=sg; o[3+(n>>1)]|=(n&1)?c<<4:c;
  }
  return o;
}
function decodificar(b){
  let i=Math.min(88,b[0]),p=((b[1]|b[2]<<8)<<16)>>16; const f=new Float32Array(N);
  for(let n=0;n<N;n++){
    const x=b[3+(n>>1)],c=(n&1)?x>>4:x&15; let st=STEP[i],v=st>>3;
    if(c&4)v+=st; if(c&2)v+=st>>1; if(c&1)v+=st>>2;
    p=clamp(p+((c&8)?-v:v),-32768,32767); i=clamp(i+IDX[c],0,88); f[n]=p/32768;
  }
  return f;
}

/* ---------- Reproductor con búfer anti-cortes (sin AudioWorklet, así funciona también abriendo el archivo con doble
clic) ---------- */
const Q=[]; let qN=0,qO=0,sonando=false;
function encolar(f){
  Q.push(f); qN+=f.length;
  while(qN>2400){const c=Q.shift(); qN-=c.length-qO; qO=0}     // máximo ~300 ms de retraso
}
function vaciarCola(){Q.length=0;qN=qO=0;sonando=false}
function llenar(e){
  const y=e.outputBuffer.getChannelData(0); let k=0;
  if(!sonando&&qN>=640)sonando=true;                            // espera ~80 ms antes de arrancar
  while(sonando&&k<y.length){
    if(!Q.length){sonando=false;break}
    const c=Q[0],m=Math.min(c.length-qO,y.length-k);
    y.set(c.subarray(qO,qO+m),k);k+=m;qO+=m;qN-=m;
    if(qO>=c.length){Q.shift();qO=0}
  }
  for(;k<y.length;k++)y[k]=0;
  bufMs=qN/8;
}

async function iniciarAudio(){
  play=ctx.createScriptProcessor(512,1,1); play.onaudioprocess=llenar;
  gain=ctx.createGain(); gain.gain.value=$('#vol').value;
  play.connect(gain).connect(ctx.destination);
  mic=await
navigator.mediaDevices.getUserMedia({audio:{echoCancellation:true,noiseSuppression:true,autoGainControl:true,channelCou
nt:1}});
  cap=ctx.createScriptProcessor(512,1,1);
  cap.onaudioprocess=e=>alMicrofono(e.inputBuffer.getChannelData(0));
  ctx.createMediaStreamSource(mic).connect(cap); cap.connect(ctx.destination);   // la salida de cap queda en silencio
  log('Micrófono y altavoces activos a 8 kHz.');
}

/* ---------- Voz de la PC hacia el ESP32 ---------- */
const u8=new Uint8Array(B64N); let nU8=0;
function alMicrofono(f){
  const md=modo(); let m=0;
  for(let k=0;k<f.length;k++){
    const x=f[k],a=Math.abs(x); if(a>m)m=a;
    if(md==='adpcm'){
      pcm[nPcm++]=clamp(x,-1,1)*32767;
      if(nPcm===N){
        nPcm=0; const b=codificar(pcm);
        if(estado==='EN LLAMADA'&&!silencio){const fr=new
Uint8Array(CUERPO+2);fr[0]=0xFE;fr[1]=0xFF;fr.set(b,2);escribir(fr);tx++}
```

```
      }
    }else{
      u8[nU8++]=clamp(Math.floor((x+1)*128),0,255);
      if(nU8===B64N){
        nU8=0;
        if(estado==='EN LLAMADA'&&!silencio){escribir(te.encode('A:'+btoa(String.fromCharCode(...u8))+'\n'));tx++}
      }
    }
  }
  lvlL=Math.max(lvlL,m);
}

/* ---------- Voz del ESP32 hacia la PC ---------- */
function avisoModo(txt){if(avisado)return;avisado=true;log(txt,'err')}
function audioRx(b){                  // trama ADPCM binaria
  rx++; if(estado!=='EN LLAMADA'||!play)return;
  if(modo()!=='adpcm'){avisoModo('Está llegando audio ADPCM. Cambia Firmware a "Audio ADPCM binario" (460800) y vuelve
a conectar.');return}
  const f=decodificar(b); let m=0; for(const x of f)m=Math.max(m,Math.abs(x));
  lvlR=Math.max(lvlR,m); encolar(f);
}
function audioB64(s){                 // línea "AUDIO:<base64>"
  let r; try{r=atob(s.trim())}catch{return}
  rx++; if(estado!=='EN LLAMADA'||!play)return;
  if(modo()!=='b64'){avisoModo('Está llegando audio base64. Cambia Firmware a "Audio base64 (sdl)" (500000) y vuelve a
conectar.');return}
  const f=new Float32Array(r.length); let m=0;
  for(let i=0;i<r.length;i++){const x=r.charCodeAt(i)/128-1;f[i]=x;const a=Math.abs(x);if(a>m)m=a}
  lvlR=Math.max(lvlR,m); encolar(f);
}

/* ---------- Puerto serie: texto + tramas de audio (0xFE 0xFF + 29 bytes) ---------- */
let fase=0,cuerpo=new Uint8Array(CUERPO),pos=0,ln=[];
function procesar(u){
  for(const b of u){
    if(fase===2){cuerpo[pos++]=b;if(pos===CUERPO){fase=0;audioRx(cuerpo)}continue}
    if(fase===1){fase=0;if(b===0xFF){fase=2;pos=0;continue}}
    if(b===0xFE)fase=1;
    else if(b===10){const t=td.decode(new Uint8Array(ln)).trim();ln=[];if(t)alLinea(t)}
    else if(b!==13&&ln.length<600)ln.push(b);
  }
}
async function leer(){
  while(activo&&port&&port.readable){
    reader=port.readable.getReader();
    try{for(;;){const{value,done}=await reader.read();if(done)break;procesar(value)}}
    catch(e){if(activo)log('Error de lectura: '+e.message,'err');break}
    finally{reader.releaseLock()}
  }
}
function escribir(u8){if(writer)writer.write(u8).catch(()=>{})}
function cmd(t,callado){if(!callado)log('> '+t,'me');escribir(te.encode(t+'\n'))}
function activarAudio(){if(modo()==='adpcm'){cmd('/audio 1',1);cmd('/estado',1)}}

function alLinea(t){
  if(t.startsWith('AUDIO:')){audioB64(t.slice(6));return}
  log(t); let m;
  if((m=t.match(/Nodo telefono (\d+) listo/))){$('#nodo').textContent='Nodo
'+m[1];$('#dest').value=m[1]==1?2:1;setState('REPOSO');activarAudio()}
  else if(/Ruido de fondo/.test(t))activarAudio();
  else if((m=t.match(/El nodo (\d+) te llama/)))setState('TIMBRANDO',m[1]);
  else if((m=t.match(/Llamando al nodo (\d+)/)))setState('LLAMANDO',m[1]);
  else if(/CONECTADO con/.test(t)){m=t.match(/(\d+)/);setState('EN LLAMADA',m&&m[1])}
  else if((m=t.match(/^Nodo (\d+) \| ([A-Z ]+?) \|(?:.*con nodo (\d+))?/))){$('#nodo').textContent='Nodo
'+m[1];$('#dest').value=m[1]==1?2:1;setState(m[2],m[3])}
  else if(/En reposo|colg|terminad|rechazad|Cancelando/i.test(t))setState('REPOSO');
}

/* ---------- Interfaz ---------- */
function log(t,c){const
l=$('#log'),d=document.createElement('div');d.textContent=t;if(c)d.className=c;l.append(d);while(l.childElementCount>30
0)l.firstChild.remove();l.scrollTop=l.scrollHeight}
```

```
function timbre(){if(!ctx)return;[0,.3].forEach(t=>{const
o=ctx.createOscillator(),g=ctx.createGain();o.frequency.value=t?480:440;g.gain.value=.15;o.connect(g).connect(ctx.desti
nation);o.start(ctx.currentTime+t);o.stop(ctx.currentTime+t+.2)})}
function setState(s,p){
  estado=s; document.body.dataset.s=s; $('#estado').textContent=s;
  if(p)$('#pareja').textContent='nodo '+p; if(s==='REPOSO'||s==='DESCONECTADO')$('#pareja').textContent='—';
  // Con el firmware ADPCM los botones siguen el estado; con el de base64 quedan todos activos porque no avisa de todos
los cambios
  const on=activo, estricto=modo()==='adpcm';
  $('#bLlamar').disabled=!on||(estricto&&s!=='REPOSO');
  $('#bCont').disabled=$('#bRech').disabled=!on||(estricto&&s!=='TIMBRANDO');
  $('#bCol').disabled=!on||(estricto&&!['LLAMANDO','TIMBRANDO','EN LLAMADA'].includes(s));
  if(s!=='EN LLAMADA')vaciarCola();
  clearInterval(ringT); if(s==='TIMBRANDO'){timbre();ringT=setInterval(timbre,1500)}
}
async function conectar(){
  $('#bCon').disabled=true;
  try{
    try{port=await navigator.serial.requestPort()}
    catch(e){log(e.name==='NotFoundError'?'No elegiste ningún puerto.':'No se pudo abrir el selector:
'+e.message,'err');port=null;return}
    avisado=false;
    ctx=new AudioContext({sampleRate:8000,latencyHint:'interactive'}); ctx.resume().catch(()=>{});
    await port.open({baudRate:+$('#baud').value,bufferSize:8192});
    writer=port.writable.getWriter(); activo=true; leyendo=leer();
    $('#fw').disabled=$('#baud').disabled=true;
    $('#bCon').textContent='Desconectar'; $('#bCon').classList.add('sec');
    log('ESP32 conectado a '+$('#baud').value+' baudios.');
    setState('REPOSO');
    try{await iniciarAudio()}catch(e){log('Audio: '+e.message+' (los comandos siguen funcionando)','err')}
    activarAudio(); setTimeout(activarAudio,3000);   // el ESP32 puede reiniciarse al abrir el puerto
  }catch(e){
    log('No se pudo conectar: '+e.message+(e.name==='NetworkError'?'  (cierra el Monitor Serial de Arduino u otra
pestaña que use el puerto)':''),'err');
    await desconectar();
  }finally{$('#bCon').disabled=false}
}
async function desconectar(){
  if(cerrando)return; cerrando=true;
  const hubo=activo; activo=false;
  try{if(writer&&hubo&&modo()==='adpcm'){cmd('/audio 0',1);await esperar(120)}}catch{}
  try{await reader?.cancel()}catch{}
  try{await Promise.race([leyendo,esperar(1000)])}catch{}
  try{if(writer)await Promise.race([writer.close(),esperar(400)])}catch{}
  try{writer?.releaseLock()}catch{}
  try{if(port)await Promise.race([port.close(),esperar(1500)])}catch{}
  try{mic?.getTracks().forEach(t=>t.stop())}catch{}
  try{play?.disconnect();cap?.disconnect()}catch{}
  try{await ctx?.close()}catch{}
  port=writer=reader=leyendo=play=cap=gain=ctx=mic=null; fase=0;pos=0;ln=[];nPcm=nU8=0;
  $('#fw').disabled=$('#baud').disabled=false;
  $('#bCon').textContent='Conectar ESP32'; $('#bCon').classList.remove('sec');
  $('#nodo').textContent='Sin nodo'; setState('DESCONECTADO');
  cerrando=false;
}
$('#bCon').onclick=()=>activo?desconectar():conectar();
$('#fw').onchange=()=>{$('#baud').value=modo()==='b64'?'500000':'460800'};
$('#bLlamar').onclick=()=>cmd('/llamar '+(+$('#dest').value));
$('#bCont').onclick=()=>cmd('/contestar');
$('#bRech').onclick=()=>cmd(modo()==='adpcm'?'/rechazar':'/colgar');
$('#bCol').onclick=()=>cmd('/colgar');
$('#bMute').onclick=e=>{silencio=!silencio;e.target.textContent=silencio?'Activar micrófono':'Silenciar micrófono'};
$('#vol').oninput=e=>{if(gain)gain.gain.value=e.target.value};
$('#fCmd').onsubmit=e=>{e.preventDefault();const v=$('#cmd').value.trim();if(v&&activo)cmd(v);$('#cmd').value=''};
if(!('serial' in navigator)){$('#aviso').style.display='block';$('#bCon').disabled=true}
else navigator.serial.addEventListener('disconnect',()=>{if(activo){log('Se desconectó el cable
USB.','err');desconectar()}});
setInterval(()=>{$('#mL').style.width=Math.min(100,lvlL*130)+'%';$('#mR').style.width=Math.min(100,lvlR*130)+'%';lvlL*=
.6;lvlR*=.6},60);
setInterval(()=>{if(activo)$('#stats').textContent=`Enviando ${tx} paq/s · recibiendo ${rx} paq/s · búfer de
reproducción ${bufMs|0} ms`;tx=rx=0},1000);
</script>
```

```
</body>
</html>
```

A.11. Versión v6: firmware final con voz (Nodo_Telefono_Voz.ino)

```
/*
  NODO TELEFONO CON VOZ PROPIA  (ESP32 DevKit V1 + nRF24L01+ PA+LNA)
  ------------------------------------------------------------------------
  Usa el MICROFONO y la BOCINA DEL CIRCUITO (no los de la PC):

     Microfono (GPIO34) -> ADPCM 4 bits -> radio nRF24 -> otro nodo
     otro nodo -> radio nRF24 -> ADPCM -> GPIO25 (DAC) -> capacitor 100 uF -> PAM8403 -> bocina

  Conexiones:
     nRF24   : CE=GPIO22 CSN=GPIO4 SCK=GPIO18 MISO=GPIO19 MOSI=GPIO23, VCC=3V3 (+100 uF), GND
     Microfono: OUT=GPIO34, VDD=3V3 (MAX9814: GAIN a VDD), GND
     PAM8403 : L <- GPIO25 (capacitor 100 uF, + hacia GPIO25), G=GND, "+"=VIN (5 V, +100 uF), "-"=GND
     LED     : GPIO12 -> resistencia 220-330 ohm -> LED -> GND
     Boton   : BOOT (GPIO0) = "hablar" si PUSH_TO_TALK = 1

  Monitor Serie a 115200, "Nueva linea". Comandos:
     /llamar N     /contestar    /rechazar    /colgar    /estado
     /tono         tono de prueba de 2 s en la bocina del circuito
     /nivel        pico del microfono desde la ultima vez (0 a 32767)
     /vol N        volumen de la bocina, 1 a 8
     /gan N        ganancia del microfono, 1 a 8
     cualquier otro texto dentro de una llamada se manda como mensaje

  Formato de voz (cada paquete de radio, 32 bytes, sin ACK):
     [VOZ][origen][secuencia][indice ADPCM][predictor lo][predictor hi][26 bytes = 52 muestras]
  Ojo: es ADPCM IMA propio de este firmware; no es compatible con la pagina HTML
  de la rama silviorodriguez. Todos los nodos de la llamada deben usar ESTE firmware.
*/
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define MI_ID 38                // <-- distinto en cada nodo: 1, 2, 3 ...

// ===== RED: IDENTICA A LA DE LA CENTRAL =====
//  Central rama equipoqeso (v2/v3):     NOMBRE_RED "NODE"  CANAL_CONTROL 80
//  Central rama silviorodriguez (v2.2): NOMBRE_RED "ITA1"  CANAL_CONTROL 76
#define NOMBRE_RED     "NODE"    // exactamente 4 letras
#define CANAL_CONTROL  80
#define ID_CENTRAL     0

// ===== AUDIO =====
#define PUSH_TO_TALK   1         // 1 = hablas manteniendo BOOT (evita chillido y choques de radio)
                                 // 0 = ambos hablan siempre (full duplex)
#define FS             8000      // muestras por segundo
#define MUESTRAS       52        // muestras por paquete de voz (26 bytes)
#define AMP_MAX        20        // tope del DAC (+-). El PAM8403 amplifica x16: no subir mucho
#define AMP_TONO       12        // amplitud del /tono
#define PRELLENADO     208       // muestras que se juntan antes de empezar a sonar (26 ms)
#define LATENCIA_MAX   1200      // si se acumula mas que esto, se descarta (evita retraso)

#if !defined(CONFIG_IDF_TARGET_ESP32)
  #error "Este firmware necesita un ESP32 DevKit (DAC en GPIO25)"
#endif
#define PIN_CE    22
#define PIN_CSN   4
#define PIN_SCK   18
#define PIN_MISO  19
#define PIN_MOSI  23
#define PIN_LED   12
#define PIN_BOOT  0
#define PIN_MIC   34
#define PIN_DAC   25
#define LED_ON    HIGH

enum Tipo : uint8_t {
```

```
  REGISTRO = 1, LLAMAR, CONTESTAR, RECHAZAR, LIBERAR,
  ENTRANTE, CONECTAR, RESPUESTA, TEXTO, KEEPALIVE, COLGAR,
  AVISO, VOZ
};
enum Codigo : uint8_t {
  R_TIMBRANDO = 1, R_OCUPADO, R_NO_DISPONIBLE, R_RECHAZADO,
  R_NO_CONTESTA, R_SIN_CANALES, R_CANCELADO
};
struct Paquete {
  uint8_t tipo;
  uint8_t origen;
  uint8_t destino;
  uint8_t canal;
  uint8_t dato;
  uint8_t secuencia;
  char    texto[26];
};
static_assert(sizeof(Paquete) == 32, "El paquete debe medir 32 bytes");

enum Estado { REPOSO, LLAMANDO, TIMBRANDO, EN_LLAMADA };

struct Adpcm { int16_t pred; int8_t idx; };   // estado del compresor ADPCM (debe ir antes de las funciones)

void direccionDe(uint8_t id, uint8_t *dir) {
  dir[0] = id;
  dir[1] = NOMBRE_RED[0]; dir[2] = NOMBRE_RED[1]; dir[3] = NOMBRE_RED[2]; dir[4] = NOMBRE_RED[3];
}

RF24 radio(PIN_CE, PIN_CSN);
uint8_t secuencia = 0;

Estado estado = REPOSO;
uint8_t pareja = 0;
uint8_t canalActual = CANAL_CONTROL;
unsigned long tEstado = 0;
unsigned long tRegistro = 0;
bool centralPerdida = false;
uint8_t nivelRemoto = 0;
#define REGISTRO_CADA_MS 5000UL

const char *nombreEstado[] = {"REPOSO", "LLAMANDO", "TIMBRANDO", "EN LLAMADA"};

// ============================ ADPCM (IMA, 4 bits) ============================
static const int8_t tablaIdx[16] = {-1,-1,-1,-1,2,4,6,8,-1,-1,-1,-1,2,4,6,8};
static const int16_t tablaPaso[89] = {
  7,8,9,10,11,12,13,14,16,17,19,21,23,25,28,31,34,37,41,45,50,55,60,66,73,80,88,97,107,118,130,
  143,157,173,190,209,230,253,279,307,337,371,408,449,494,544,598,658,724,796,876,963,1060,1166,
  1282,1411,1552,1707,1878,2066,2272,2499,2749,3024,3327,3660,4026,4428,4871,5358,5894,6484,7132,
  7845,8630,9493,10442,11487,12635,13899,15289,16818,18500,20350,22385,24623,27086,29794,32767 };

Adpcm codif = {0, 0};            // estado del que comprime (lo que mando)
Adpcm decod = {0, 0};            // estado del que descomprime (lo que recibo)

static inline void aplicar(Adpcm &s, uint8_t code, int vpdiff) {
  int p = s.pred + ((code & 8) ? -vpdiff : vpdiff);
  if (p > 32767) p = 32767;
  if (p < -32768) p = -32768;
  s.pred = (int16_t)p;
  int i = s.idx + tablaIdx[code];
  if (i < 0) i = 0;
  if (i > 88) i = 88;
  s.idx = (int8_t)i;
}

uint8_t comprimirMuestra(Adpcm &s, int16_t m) {
  int paso = tablaPaso[s.idx];
  int dif = m - s.pred;
  uint8_t code = 0;
  if (dif < 0) { code = 8; dif = -dif; }
  int vpdiff = paso >> 3;
  if (dif >= paso) { code |= 4; dif -= paso; vpdiff += paso; }
  paso >>= 1;
```

```
  if (dif >= paso) { code |= 2; dif -= paso; vpdiff += paso; }
  paso >>= 1;
  if (dif >= paso) { code |= 1; vpdiff += paso; }
  aplicar(s, code, vpdiff);
  return code;
}

int16_t descomprimirMuestra(Adpcm &s, uint8_t code) {
  int paso = tablaPaso[s.idx];
  int vpdiff = paso >> 3;
  if (code & 4) vpdiff += paso;
  if (code & 2) vpdiff += paso >> 1;
  if (code & 1) vpdiff += paso >> 2;
  aplicar(s, code, vpdiff);
  return s.pred;
}

// 52 muestras -> 29 bytes [indice][pred lo][pred hi][26 bytes]
void empacar(const int16_t *m, uint8_t *out) {
  out[0] = (uint8_t)codif.idx;
  out[1] = (uint8_t)(codif.pred & 0xFF);
  out[2] = (uint8_t)((uint16_t)codif.pred >> 8);
  for (int i = 0; i < MUESTRAS; i += 2) {
    uint8_t a = comprimirMuestra(codif, m[i]);
    uint8_t b = comprimirMuestra(codif, m[i + 1]);
    out[3 + i / 2] = a | (b << 4);
  }
}

void desempacar(const uint8_t *in, int16_t *m) {
  decod.idx = (int8_t)constrain((int)in[0], 0, 88);
  decod.pred = (int16_t)(in[1] | (in[2] << 8));
  for (int i = 0; i < MUESTRAS; i += 2) {
    uint8_t d = in[3 + i / 2];
    m[i]     = descomprimirMuestra(decod, d & 0x0F);
    m[i + 1] = descomprimirMuestra(decod, d >> 4);
  }
}

// ============ AUDIO EN TIEMPO REAL (tarea en el nucleo 0, 8 kHz exactos) ============
#define MIC_N  1024
#define SPK_N  2048
int16_t micBuf[MIC_N];
int16_t spkBuf[SPK_N];
volatile uint16_t micHead = 0, micTail = 0;     // escribe la tarea, lee el loop
volatile uint16_t spkHead = 0, spkTail = 0;     // escribe el loop, lee la tarea
volatile bool micActivo = false;                // el loop pide capturar
volatile bool spkFlush = false;                 // el loop pide vaciar la cola de sonido
volatile bool bocinaActiva = false;             // hay llamada: reproducir
volatile uint32_t tonoFin = 0;
volatile int volumen = 2;                       // 1..8
volatile int gananciaMic = 2;                   // 1..8
volatile int16_t micPico = 0;

SemaphoreHandle_t semTick;

void IRAM_ATTR alTick() {
  BaseType_t hp = pdFALSE;
  xSemaphoreGiveFromISR(semTick, &hp);
  if (hp) portYIELD_FROM_ISR();
}

void tareaAudio(void *) {
  float media = 2048;
  float faseTono = 0;
  bool sonando = false;
  for (;;) {
    xSemaphoreTake(semTick, portMAX_DELAY);

    // ---- microfono ----
    int v = analogRead(PIN_MIC);
    media += (v - media) * 0.002f;                    // quita el nivel fijo (DC)
```

```
    int x = (int)((v - media) * 16.0f * gananciaMic / 2.0f);
    if (x > 32767) x = 32767;
    if (x < -32768) x = -32768;
    int ax = x < 0 ? -x : x;
    if (ax > micPico) micPico = ax;
    if (micActivo) {
      uint16_t h = micHead;
      uint16_t sig = (h + 1) & (MIC_N - 1);
      if (sig != micTail) { micBuf[h] = (int16_t)x; micHead = sig; }   // si esta llena, se pierde
    }

    // ---- bocina ----
    if (spkFlush) { spkTail = spkHead; sonando = false; spkFlush = false; }
    int salida = 128;
    if (tonoFin && (int32_t)(tonoFin - millis()) > 0) {
      faseTono += 2.0f * PI * 660.0f / FS;
      if (faseTono > 2.0f * PI) faseTono -= 2.0f * PI;
      salida = 128 + (int)(AMP_TONO * sinf(faseTono));
    } else if (bocinaActiva) {
      uint16_t cuenta = (spkHead - spkTail) & (SPK_N - 1);
      if (!sonando && cuenta >= PRELLENADO) sonando = true;
      if (sonando && cuenta == 0) sonando = false;
      if (sonando) {
        int s = spkBuf[spkTail];
        spkTail = (spkTail + 1) & (SPK_N - 1);
        int d = (s * volumen) >> 9;
        if (d > AMP_MAX) d = AMP_MAX;
        if (d < -AMP_MAX) d = -AMP_MAX;
        salida = 128 + d;
      }
    } else {
      sonando = false;
    }
    dacWrite(PIN_DAC, salida);
  }
}

uint16_t micDisponibles() { return (micHead - micTail) & (MIC_N - 1); }
uint16_t spkOcupadas()    { return (spkHead - spkTail) & (SPK_N - 1); }

// ============================ RADIO ============================
bool enviar(uint8_t destino, uint8_t tipo, uint8_t dato, const char *texto) {
  Paquete p = {};
  p.tipo = tipo; p.origen = MI_ID; p.destino = destino;
  p.canal = canalActual; p.dato = dato; p.secuencia = secuencia++;
  if (texto) strncpy(p.texto, texto, sizeof(p.texto) - 1);
  uint8_t dir[5]; direccionDe(destino, dir);
  radio.stopListening();
  radio.openWritingPipe(dir);
  bool ok = radio.write(&p, sizeof(p));
  radio.startListening();
  return ok;
}

void enviarVoz(const uint8_t *cuerpo29) {
  uint8_t b[32];
  b[0] = VOZ; b[1] = MI_ID; b[2] = secuencia++;
  memcpy(b + 3, cuerpo29, 29);
  uint8_t dir[5]; direccionDe(pareja, dir);
  radio.stopListening();
  radio.openWritingPipe(dir);
  radio.write(b, 32, true);               // sin ACK: si se pierde, no se reintenta
  radio.startListening();
}

void cambiarCanal(uint8_t c) {
  radio.stopListening();
  radio.setChannel(c);
  radio.startListening();
  canalActual = c;
}
```

```
void cambiarEstado(Estado e) {
  estado = e;
  tEstado = millis();
  nivelRemoto = 0;
  bool enLlamada = (e == EN_LLAMADA);
  bocinaActiva = enLlamada;
  spkFlush = true;
  micTail = micHead;
  codif = {0, 0};
}

void ledBrillo(uint8_t v) { analogWrite(PIN_LED, LED_ON == HIGH ? v : 255 - v); }
uint8_t gamma8(uint8_t v) { return (uint16_t)v * v / 255; }

void volverAReposo(bool avisarCentral) {
  if (canalActual != CANAL_CONTROL) cambiarCanal(CANAL_CONTROL);
  if (avisarCentral) enviar(ID_CENTRAL, LIBERAR, 0, nullptr);
  pareja = 0;
  cambiarEstado(REPOSO);
  enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
  tRegistro = millis();
  Serial.println("En reposo.");
}

// ============================ COMANDOS ============================
void procesarLinea(String l) {
  l.trim();
  if (l.length() == 0) return;

  if (l.startsWith("/llamar")) {
    if (estado != REPOSO) { Serial.println("[ERR] Ocupado"); return; }
    int destino = l.substring(7).toInt();
    if (destino < 1 || destino > 254 || destino == MI_ID) { Serial.println("Uso: /llamar N"); return; }
    if (!enviar(ID_CENTRAL, LLAMAR, destino, nullptr)) {
      Serial.println("[ERR] La central no responde, no se pudo llamar");
      return;
    }
    pareja = destino;
    cambiarEstado(LLAMANDO);
    Serial.printf("Llamando al nodo %u\n", pareja);
  }
  else if (l == "/contestar") {
    if (estado != TIMBRANDO) return;
    enviar(ID_CENTRAL, CONTESTAR, 0, nullptr);
  }
  else if (l == "/rechazar") {
    if (estado != TIMBRANDO) return;
    enviar(ID_CENTRAL, RECHAZAR, 0, nullptr);
    volverAReposo(false);
  }
  else if (l == "/colgar") {
    if (estado == EN_LLAMADA) {
      enviar(pareja, COLGAR, 0, nullptr);
      volverAReposo(true);
    } else if (estado == LLAMANDO || estado == TIMBRANDO) {
      volverAReposo(true);
    }
  }
  else if (l == "/estado") {
    Serial.printf("Nodo %u | %s | canal %u | red %s", MI_ID, nombreEstado[estado], canalActual, NOMBRE_RED);
    if (pareja) Serial.printf(" | con nodo %u", pareja);
    Serial.printf(" | vol %d | gan %d | %s\n", (int)volumen, (int)gananciaMic,
                  PUSH_TO_TALK ? "push-to-talk (BOOT)" : "full duplex");
  }
  else if (l == "/tono") {
    tonoFin = millis() + 2000;
    Serial.println("Tono de prueba en la bocina del circuito (2 s)");
  }
  else if (l == "/nivel") {
    int p = micPico; micPico = 0;
    Serial.printf("Pico del microfono: %d de 32767\n", p);
  }
```

```
  else if (l.startsWith("/vol")) {
    volumen = constrain(l.substring(4).toInt(), 1, 8);
    Serial.printf("Volumen de la bocina: %d\n", (int)volumen);
  }
  else if (l.startsWith("/gan")) {
    gananciaMic = constrain(l.substring(4).toInt(), 1, 8);
    Serial.printf("Ganancia del microfono: %d\n", (int)gananciaMic);
  }
  else if (l.startsWith("/")) {
    Serial.println("Comandos: /llamar N /contestar /rechazar /colgar /estado /tono /nivel /vol N /gan N");
  }
  else {
    if (estado != EN_LLAMADA) return;
    String t = l.substring(0, 25);
    enviar(pareja, TEXTO, 0, t.c_str());
    Serial.printf("[YO] %s\n", t.c_str());
  }
}

void procesarPaquete(Paquete &p) {
  p.texto[sizeof(p.texto) - 1] = '\0';

  if (p.origen == ID_CENTRAL) {
    centralPerdida = false;
    switch (p.tipo) {
      case AVISO:
        Serial.printf("[CENTRAL] %s\n", p.texto);
        break;
      case ENTRANTE:
        if (estado != REPOSO) break;
        pareja = p.dato;
        cambiarEstado(TIMBRANDO);
        Serial.printf("El nodo %u te llama. Escribe /contestar o /rechazar\n", pareja);
        break;
      case CONECTAR:
        if (estado != LLAMANDO && estado != TIMBRANDO) break;
        pareja = p.dato;
        cambiarCanal(p.canal);
        cambiarEstado(EN_LLAMADA);
        Serial.printf("CONECTADO con nodo %u (canal %u)%s\n", pareja, p.canal,
                      PUSH_TO_TALK ? ". Mantén BOOT para hablar" : "");
        break;
      case RESPUESTA: {
        if (p.dato == R_TIMBRANDO) { Serial.println("Timbrando en el otro nodo..."); break; }
        const char *m[] = {"?", "timbrando", "OCUPADO (uno de los dos ya esta en llamada)",
                           "NO DISPONIBLE (la central no ve al destino: ID mal o apagado)",
                           "RECHAZADO", "NO CONTESTA", "SIN CANALES", "CANCELADO"};
        Serial.printf("[LLAMADA FALLIDA] %s\n", p.dato < 8 ? m[p.dato] : "codigo desconocido");
        volverAReposo(false);
        break;
      }
    }
    return;
  }

  if (estado != EN_LLAMADA || p.origen != pareja) return;
  switch (p.tipo) {
    case TEXTO:
      Serial.printf("[MSG %u] %s\n", p.origen, p.texto);
      break;
    case COLGAR:
      volverAReposo(true);
      break;
  }
}

// Voz que llega del otro nodo: se descomprime y se mete a la cola de la bocina
void recibirVoz(const uint8_t *buf) {
  if (estado != EN_LLAMADA || buf[1] != pareja) return;
  nivelRemoto = constrain(buf[3] * 3, 0, 255);
  if (spkOcupadas() > LATENCIA_MAX) return;                // demasiado retraso: se descarta
  int16_t m[MUESTRAS];
```

```
  desempacar(buf + 3, m);
  uint16_t h = spkHead;
  for (int i = 0; i < MUESTRAS; i++) { spkBuf[h] = m[i]; h = (h + 1) & (SPK_N - 1); }
  spkHead = h;
}

// Voz propia: cada 52 muestras del microfono se comprimen y salen por radio
void transmitirVoz() {
  bool hablar = (estado == EN_LLAMADA);
#if PUSH_TO_TALK
  hablar = hablar && (digitalRead(PIN_BOOT) == LOW);
#endif
  if (hablar != micActivo) {
    micActivo = hablar;
    micTail = micHead;
    if (hablar) codif = {0, 0};
  }
  if (!micActivo) return;
  int paquetes = 0;
  while (micDisponibles() >= MUESTRAS && paquetes < 2) {
    int16_t m[MUESTRAS];
    uint16_t t = micTail;
    for (int i = 0; i < MUESTRAS; i++) { m[i] = micBuf[t]; t = (t + 1) & (MIC_N - 1); }
    micTail = t;
    uint8_t cuerpo[29];
    empacar(m, cuerpo);
    enviarVoz(cuerpo);
    paquetes++;
  }
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(PIN_LED, OUTPUT);
  ledBrillo(0);
  pinMode(PIN_BOOT, INPUT_PULLUP);

  analogReadResolution(12);
  analogSetPinAttenuation(PIN_MIC, ADC_11db);
  dacWrite(PIN_DAC, 128);

  SPI.begin(PIN_SCK, PIN_MISO, PIN_MOSI, PIN_CSN);
  if (!radio.begin(&SPI)) {
    Serial.println("ERROR: el nRF24 no responde. Revisa cables, 3V3 y capacitor.");
    while (true) { ledBrillo(255); delay(100); ledBrillo(0); delay(100); }
  }
  radio.setChannel(CANAL_CONTROL);
  radio.setDataRate(RF24_1MBPS);
  radio.setPALevel(RF24_PA_LOW);            // para mas alcance: RF24_PA_HIGH (ver capacitor)
  radio.setPayloadSize(sizeof(Paquete));
  radio.setAutoAck(true);
  radio.enableDynamicAck();                 // permite mandar la voz sin ACK
  radio.setRetries(3 + (MI_ID % 10), 15);
  uint8_t dir[5]; direccionDe(MI_ID, dir);
  radio.openReadingPipe(1, dir);
  radio.startListening();

  // Reloj de audio de 8 kHz y tarea en el nucleo 0
  semTick = xSemaphoreCreateBinary();
  xTaskCreatePinnedToCore(tareaAudio, "audio", 4096, nullptr, 5, nullptr, 0);
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  hw_timer_t *tm = timerBegin(1000000);
  timerAttachInterrupt(tm, &alTick);
  timerAlarm(tm, 1000000 / FS, true, 0);
#else
  hw_timer_t *tm = timerBegin(0, 80, true);
  timerAttachInterrupt(tm, &alTick, true);
  timerAlarmWrite(tm, 1000000 / FS, true);
  timerAlarmEnable(tm);
#endif
```

```
  tRegistro = millis() - REGISTRO_CADA_MS;
  Serial.printf("Nodo telefono %u listo (ESP32 DevKit) red %s canal %u\n", MI_ID, NOMBRE_RED, CANAL_CONTROL);
  Serial.println("Escribe /llamar N para llamar, o /tono para probar la bocina.");
}

void loop() {
  static String linea = "";
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') { procesarLinea(linea); linea = ""; }
    else if (linea.length() < 100) linea += c;
  }

  while (radio.available()) {
    uint8_t buf[32];
    radio.read(buf, sizeof(buf));
    if (buf[0] == VOZ) {
      recibirVoz(buf);
    } else {
      Paquete p;
      memcpy(&p, buf, sizeof(p));
      procesarPaquete(p);
    }
  }

  transmitirVoz();

  unsigned long ahora = millis();
  if (estado == REPOSO && ahora - tRegistro >= REGISTRO_CADA_MS) {
    tRegistro = ahora;
    bool ok = enviar(ID_CENTRAL, REGISTRO, 0, nullptr);
    if (ok) {
      if (centralPerdida) Serial.println("[OK] Central de nuevo en linea");
      centralPerdida = false;
    } else if (!centralPerdida) {
      centralPerdida = true;
      Serial.println("[ERR] La central no responde (revisa NOMBRE_RED y CANAL_CONTROL)");
    }
  }
  if (estado == LLAMANDO && ahora - tEstado > 30000UL) {
    Serial.println("[LLAMADA FALLIDA] Sin respuesta de la central (30 s)");
    volverAReposo(true);
  }

  uint8_t brillo = 0;
  if (estado == TIMBRANDO)  brillo = ((ahora / 150) % 2) ? 255 : 0;
  if (estado == LLAMANDO)   brillo = ((ahora / 600) % 2) ? 255 : 0;
  if (estado == EN_LLAMADA) brillo = micActivo ? 255 : max((uint8_t)6, gamma8(nivelRemoto));
  ledBrillo(brillo);
}
```

A.12. Prueba del micrófono (Prueba_Microfono.ino)

```
/*
  PRUEBA DE MICROFONO -> BOCINA DE LA PC
  Microfono: OUT -> GPIO34, VDD -> 3V3, GND -> GND (con MAX9814: GAIN a VDD = 40 dB)
  Manda el audio por USB a la pagina Prueba_Audio.html (Chrome o Edge), que lo
  reproduce en la bocina de la PC. NO necesita el nRF24.

  Uso:
   1) Sube este programa. Cierra el Monitor Serie (la pagina necesita el puerto).
   2) Abre Prueba_Audio.html, pulsa "Conectar" y elige el puerto del ESP32.
   3) Habla cerca del microfono. Usa audifonos o volumen bajo para evitar chillido.

  Si prefieres solo ver numeros: pon MODO_TEXTO en 1 y abre Herramientas ->
  Serial Plotter a 460800 baudios.
*/
#define PIN_MIC     34
#define MODO_TEXTO  0          // 0 = audio para la pagina, 1 = numeros para el Plotter
#define FS          8000       // muestras por segundo
#define VELOCIDAD   460800
```

```
void setup() {
  Serial.begin(VELOCIDAD);
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_MIC, ADC_11db);   // rango ~0-3.1 V
  pinMode(PIN_MIC, INPUT);
}

void loop() {
  static unsigned long prox = micros();
  static uint8_t cuenta = 0;
  static long suma = 0;

  while ((long)(micros() - prox) < 0) { }
  prox += 1000000UL / FS;

  int v = analogRead(PIN_MIC);                // 0..4095
#if MODO_TEXTO
  Serial.println(v);
#else
  uint8_t m = v >> 4;                         // 0..255
  if (m > 250) m = 250;                       // 255 queda reservado como inicio de bloque
  if (cuenta == 0) Serial.write((uint8_t)255);
  Serial.write(m);
  cuenta = (cuenta + 1) % 64;
#endif
  (void)suma;
}
```

A.13. Prueba de la bocina del circuito (Prueba_Bocina_Circuito.ino)

```
/*
  PRUEBA DE LA BOCINA DEL CIRCUITO (ESP32 -> PAM8403 -> bocina)
  Audio: GPIO25 (DAC) -> capacitor 100 uF (+ hacia GPIO25) -> pin L del PAM8403
  PAM: "+" a VIN (5 V), "-" y G a GND. Bocina en L OUT (- y +).
  Microfono (solo modo 2): OUT -> GPIO34, VDD -> 3V3, GND -> GND.

  MODO 1: tonos de prueba (440, 880 y 660 Hz), suenan en la bocina del circuito.
  MODO 2: lo que capta el microfono sale por la bocina del circuito.
          Mantén el microfono LEJOS de la bocina o chillara (retroalimentacion).

  Monitor Serie a 115200. Empieza con AMPLITUD baja; la bocina es de 0.5 W.
*/
#define MODO        1       // 1 = tonos, 2 = microfono -> bocina
#define PIN_DAC     25
#define PIN_MIC     34
#define AMPLITUD    30      // 0..127  (sube de 10 en 10 si se oye muy bajo)
#define GANANCIA    2.0f    // solo modo 2

#define FS          16000   // muestras por segundo
#define TONO_MS     400

const float tonos[] = {440.0f, 880.0f, 660.0f};

void setup() {
  Serial.begin(115200);
  delay(500);
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_MIC, ADC_11db);
  dacWrite(PIN_DAC, 128);                   // punto medio = silencio
#if MODO == 1
  Serial.println("Modo 1: tonos en la bocina del circuito");
#else
  Serial.println("Modo 2: microfono -> bocina del circuito");
#endif
}

void loop() {
  static unsigned long prox = micros();
  while ((long)(micros() - prox) < 0) { }
  prox += 1000000UL / FS;

#if MODO == 1
```

```
  static float fase = 0;
  static unsigned long tCambio = millis();
  static uint8_t paso = 0;                  // 0..5: tono, silencio, tono, ...
  unsigned long ahora = millis();
  if (ahora - tCambio >= TONO_MS) {
    tCambio = ahora;
    paso = (paso + 1) % 6;
    if (paso % 2 == 0) Serial.printf("Tono %.0f Hz\n", tonos[paso / 2]);
  }
  if (paso % 2 == 0) {
    fase += 2.0f * PI * tonos[paso / 2] / FS;
    if (fase > 2.0f * PI) fase -= 2.0f * PI;
    dacWrite(PIN_DAC, 128 + (int)(AMPLITUD * sinf(fase)));
  } else {
    dacWrite(PIN_DAC, 128);
  }
#else
  static float media = 2048;
  int v = analogRead(PIN_MIC);
  media += (v - media) * 0.002f;            // quita el nivel fijo del microfono
  float x = (v - media) / 16.0f * GANANCIA;  // 12 bits -> aprox. +-128
  if (x > AMPLITUD) x = AMPLITUD;
  if (x < -AMPLITUD) x = -AMPLITUD;
  dacWrite(PIN_DAC, 128 + (int)x);
#endif
}
```

A.14. Prueba dúplex (Prueba_Audio_Circuito.ino)

```
/*
  PRUEBA COMPLETA DE AUDIO DEL CIRCUITO (trabaja con Prueba_Audio_Circuito.html)

    Microfono  (GPIO34)  --USB-->  PC  -->  bocina de la PC
    PC (sonido generado) --USB-->  ESP32 -->  GPIO25 (DAC) -> capacitor 100 uF -> PAM8403 -> bocina del circuito

  Conexiones:
    Microfono : OUT -> GPIO34, VDD -> 3V3 (+ GAIN a VDD si es MAX9814), GND -> GND
    PAM8403   : L <- GPIO25 (con capacitor 100 uF, + hacia GPIO25), G -> GND,
                "+" -> VIN (5 V), "-" -> GND, bocina en L OUT (- y +)

  1) Sube este programa.   2) CIERRA el Monitor Serie.
  3) Abre Prueba_Audio_Circuito.html en Chrome o Edge y pulsa "Conectar".
  Este programa NO escribe texto por el puerto: solo manda y recibe audio.
*/
#define PIN_MIC     34
#define PIN_DAC     25
#define FS          8000        // muestras por segundo (igual que la pagina)
#define VELOCIDAD   460800      // igual que la pagina
#define AMP_MAX     60          // tope de seguridad para la bocina de 0.5 W
#define BUF_SIZE    4096
#define PRELLENADO  300         // muestras que espera antes de empezar a sonar

uint8_t  cola[BUF_SIZE];
uint16_t cabeza = 0, colaIdx = 0, lleno = 0;
bool sonando = false;

uint8_t bloque[65];             // [255][64 muestras del microfono]
uint8_t nBloque = 0;

void setup() {
  Serial.setRxBufferSize(4096);
  Serial.begin(VELOCIDAD);
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_MIC, ADC_11db);
  dacWrite(PIN_DAC, 128);       // silencio
  bloque[0] = 255;
}

void loop() {
  static unsigned long prox = micros();

  // 1) Todo lo que mande la PC se guarda en una cola
```

```
  while (Serial.available()) {
    uint8_t b = Serial.read();
    if (lleno < BUF_SIZE) {
      cola[cabeza] = b;
      cabeza = (cabeza + 1) % BUF_SIZE;
      lleno++;
    }
  }

  // 2) Una vez por muestra (8 kHz)
  if ((long)(micros() - prox) < 0) return;
  prox += 1000000UL / FS;

  // Microfono -> PC (de 12 bits a 0..250; el 255 marca el inicio de bloque)
  uint8_t m = analogRead(PIN_MIC) >> 4;
  if (m > 250) m = 250;
  bloque[1 + nBloque++] = m;
  if (nBloque == 64) {
    Serial.write(bloque, 65);
    nBloque = 0;
  }

  // PC -> bocina del circuito
  if (!sonando && lleno >= PRELLENADO) sonando = true;
  if (sonando && lleno == 0) sonando = false;
  int s = 128;
  if (sonando) {
    int v = cola[colaIdx];
    colaIdx = (colaIdx + 1) % BUF_SIZE;
    lleno--;
    s = constrain(v, 128 - AMP_MAX, 128 + AMP_MAX);
  }
  dacWrite(PIN_DAC, s);
}
```

APÉNDICE B. DATOS DE LOS MÓDULOS

TABLA L
PINES DEL NRF24L01 (MÓDULO DE 8 PINES)

Pin	Nombre	Función
1	GND	Tierra
2	VCC	3.3 V (1.9–3.6 V)
3	CE	Activa RX o TX
4	CSN	Selección SPI
5	SCK	Reloj SPI
6	MOSI	Datos hacia el módulo
7	MISO	Datos desde el módulo
8	IRQ	Interrupción (no usada)

TABLA LI
PINES DE ARRANQUE DEL ESP32 A EVITAR

GPIO	Observación
0	Modo de carga (botón BOOT)
2	Pin de arranque
5	Pin de arranque
6–11	Memoria flash interna: no usar
12	Pin de arranque (voltaje de flash)
15	Pin de arranque

Las hojas de datos relevantes son las del NRF24L01+ [1], del MAX9814 [4] y del PAM8403 [5].

APÉNDICE C. BITÁCORA DE REUNIONES

TABLA LII
REGISTRO DE REUNIONES

Fecha	Asistentes	Acuerdos
≈12 sep	Equipo	Generador de señales con 555 en protoboard.
≈18 sep	Equipo	Enlace TX/RX con botón y LED; corrección del código y de la librería RF24.
sep–oct	Equipo	Diagnóstico de reinicios: pin CE, capacitor de 100 µF y reintentos.
24 sep	Equipo	Creación del repositorio y del documento de tareas.
28 sep–5 oct	Equipo	Nodo concentrador con cifrado XOR.
2 oct	Equipo	Teléfono con central en estrella y central para ESP32-S3.
8 oct	Equipo	Central v2: mensajes y monitoreo de ruido.
9 oct	Equipo	Teléfono con micrófono (nivel de voz) y central v3.
12 oct	Equipo	Doble central redundante.
13–14 oct	Equipo	Audio ADPCM con interfaz web.
15–19 oct	Equipo	Pruebas de micrófono y bocina del circuito; PAM8403; llamadas con la central de otro equipo.
20 oct	Equipo	Firmware final con voz, documentación y apéndices.